FROM-SCRATCH SYSTEMS & AI INFERENCE MASTERY

Python 3.12+ Engineering Curriculum

An 8-month (224-day, 1 hour/day) hands-on program to master production-grade Python from strict typing, data models, and async streaming to CPython internals, zero-copy buffers, strided tensors, native C/GPU interop, and building Mini-vLLM from scratch.

Months 1–4: Base Concepts (112 days) | Months 5–8: Advanced Concepts (112 days)
224 daily lessons | Strictly 1.0 focused hour per day (224 total hours)

Python Mastery for AI Inference Engineering: 8-Month From-Scratch Curriculum¶

1. Curriculum Analysis & Language Selection¶

1.1 Findings from `AI_INFERENCE_ENGINEERING_CURRICULUM_BOOK.html`¶

An exhaustive audit of your AI_INFERENCE_ENGINEERING_CURRICULUM_BOOK.html (and its source AI_INFERENCE_ENGINEERING_CURRICULUM.md) reveals a decisive architectural reality across both **Path A (AI Inference Systems / ML Infrastructure / Model Serving Engineering)** and **Path B (Advanced AI Systems / GPU / Low-Level Inference Engineering)**:

Language Candidate	Role in `AI_INFERENCE_ENGINEERING_CURRICULUM_BOOK.html`	Track 1 (Weeks 1–16) Share	Track 2 (Modules 2A–2G) Role	Verdict as Starting Language to Master
Python (3.12+)	**Primary Control-Plane, Tooling, Serving, Benchmarking, Tensor & DSL Language** (`MOVE EARLIER / KEEP` to Day 1)	**~85% of all daily code** (Typed clients, `asyncio`, SSE streaming, PyTorch tensors/attention, KV-cache math, vLLM/SGLang orchestration, FastAPI gateways, Prometheus metrics, chaos harnesses)	**Primary host & DSL language** for Triton GPU kernels (`@triton.jit`, Module 2D), vLLM scheduler/block-manager internals (Module 2E), multi-GPU orchestration (Module 2F), and paper reproduction (Module 2G)	**#1 MOST PREFERRED STARTING LANGUAGE**
C / Modern C++	Low-level runtime & kernel binding language (`DEFER / REDUCE` in Section 6 Change Log)	**0% required in Track 1** (explicitly deferred so it does not block career entry)	Modules 2A, 2B, 2C (manual memory, RAII, CUDA kernels wrapped into Python via `pybind11`/`nanobind`)	**#2 Language (Master after/via Month 7 Python C/C++ FFI bridge)**
CUDA C / PTX	Hardware-specific GPU kernel language	Profiling awareness only (`nsys` CLI in Week 7)	Module 2C (custom GPU kernel authoring)	**#3 Specialized Kernel Language**
Bash / Linux Shell	Operational inspection glue (`ps`, `/proc`, `docker`, `nvidia-ctk`)	Supporting 30% minor focus	Supporting tooling	**Supporting Operational Tool, not primary programming language**

> **Why Python 3.12+ is the Unquestioned #1 Language to Master First:** 1. **100% of Modern Inference Control Planes are Written in Python:** vLLM, SGLang, TensorRT-LLM's frontend, PyTorch (`torch.compile` / TorchDynamo), Hugging Face Transformers, and OpenAI-compatible gateways are architected in Python. Specifically, vLLM's Continuous Batching Scheduler, PagedAttention Block Manager, Prefix Caching tree, AsyncLLMEngine, ZeroMQ/SharedMemory IPC loop, and API server are all pure Systems Python. 2. **Triton GPU Kernels are Written in Python:** In Track 2 (Module 2D), high-performance fused GPU kernels (like FlashAttention, fused RMSNorm, and quantized matmuls) are written using OpenAI's **Triton** language—a Python AST/decorator DSL (`@triton.jit`). 3. **Python is the Bridge to C, C++, and CUDA:** By mastering Python deeply (up through the C-level Buffer Protocol, `memoryview`, `ctypes`, `pybind11`/`nanobind`, and DLPack in Months 6–7 of this curriculum), you automatically learn how memory layouts, pointers, strides, and GIL-free native C/C++/CUDA extensions work.

1.2 The "Quality Code From Scratch" Philosophy (No Tutorial Copy-Pasting, No Basic Syntax Fluff)¶

Because you **already know basic programming concepts** (variables, loops, conditionals, basic functions, lists/dicts), this curriculum **skips elementary syntax** on Day 1. Instead, you will spend 8 months writing **production-grade, strictly-typed, tested, high-performance Python code completely by hand**:

1. **Build Before You Import:** Before using external frameworks like Pydantic, FastAPI, NumPy, PyTorch, Prometheus, or vLLM, you will **implement clean working versions of their core mechanisms yourself from scratch** using the Python standard library (`NanoSchema`, `NanoServe`, `StridedTensor`, `AutoTensor`, `TokenPrefixTrie`, `PagedKVBlockAllocator`, `ContinuousBatchScheduler`, `SamplingFlamegraphProfiler`, `MetricsRegistry`, and `Mini-vLLM`). 2. **Zero-Compromise Engineering Standards on Every File:**

- **100% Static Type Annotations:** Every function, method, generator, and class attribute must pass `mypy --strict` (no untyped defs, no implicit `Any`, no `# type: ignore` shortcuts).
- **Zero Linter or Formatting Warnings:** Every file passes `ruff check` and `ruff format`.
- **Deterministic Resource & Memory Hygiene:** Every file, socket, subprocess, `SharedMemory` segment, and async task is managed via context managers (`with` / `async with` / `ExitStack`) and verified for zero leaks using `tracemalloc`.
- **Self-Written Automated Tests:** Every single day includes a `pytest` verification task covering normal behavior, boundary conditions, and a deliberate failure/chaos test.

1.3 The 8-Month (224-Day, 1 Hour/Day) Architecture & Daily Time Contract¶

- **Total Duration: 8 Months** = **32 Weeks** = **224 Daily Lessons** (7 focused study/build days per week, **1.0 Hour per Day** = **224 Total Hours**).
- **Phase 1 — Base Concepts Mastery (Months 1–4 | Weeks 1–16 | Days 1–112 | 112 Hours):**

Transforms basic Python familiarity into rock-solid **Production Core & Intermediate Systems Python**: modern packaging (`pyproject.toml`), strict static typing (`Protocol`, `Generic`, `TypeGuard`), frozen/slotted dataclasses, defensive context management (`ExitStack`), lazy O(1)-memory generator pipelines, the Python Data Model (dunder protocols, `__slots__`, `weakref`), closures/decorators/descriptors, algorithmic data structures (`deque`, `heapq`, `bisect`, O(1) LRU Cache, Prefix Radix Trie), binary/mmap SafeTensors I/O, deep `asyncio` structured concurrency (`TaskGroup`, cancellation, dynamic batching, raw TCP/SSE/ASGI streaming), `multiprocessing` + `shared_memory`, and the **Month 4 Base Capstone (`pyserve-core`)**.

- **Phase 2 — Advanced Concepts Mastery (Months 5–8 | Weeks 17–32 | Days 113–224 | 112 Hours):**

Elevates you to **Senior/Staff Systems & AI Inference Python**: runtime metaprogramming (`__init_subclass__`, metaclasses, AST transformations, `dis` bytecode inspection, `sys.monitoring`), authoring a Pydantic + FastAPI framework from scratch (`NanoSchema` + `NanoServe`), CPython memory internals (`PyObject`, refcounting, cyclic GC tuning, Buffer Protocol, zero-copy `memoryview`), building a strided N-dimensional Tensor, Quantizer (INT8/INT4), Autograd engine, and KV-cached Transformer (`NanoLLM`) from scratch, implementing vLLM's PagedAttention Block Manager & Continuous Batching Scheduler, deterministic/statistical flamegraph profiling, roofline analysis, Free-Threaded Python (PEP 703), native C/C++/CUDA/Triton/DLPack interoperability, distributed multi-process IPC, and the **Month 8 Advanced Capstone (`Mini-vLLM`)**.

The 60-Minute Daily Workload Contract¶

Every daily lesson is strictly bounded to **1.0 Hour (60 minutes)** so you can sustain a 100% streak alongside your other commitments:

Time Block	Duration	Weight	Activity
1. Targeted Concept & Book Retrieval	**15 mins**	Core Reading	Read the exact mapped subsection from the recommended book/docs and inspect how the mechanism works under the hood.
2. From-Scratch Quality Implementation	**30 mins**	**[CORE - 70%]**	Implement the day's module from a blank file in `src/pymastery/` with strict type annotations, docstrings, and clean architecture.
3. Verification, Static Audit & Failure Test	**15 mins**	**[MINOR - 30%]**	Write & run `pytest`, `mypy --strict`, and `ruff check`, plus execute Task 5 (break one assumption, observe the exact failure, and restore).

---

2. Comprehensive Topic Mastery Matrix (Months 1–8)¶

Month	Phase	Weeks (Days)	Core Engineering Theme	Major From-Scratch Implementations
Month 1	**Base (Part 1)**	Weeks 1–4 (Days 1–28)	**Modern Packaging, Strict Typing, Data Modeling, Context Safety & Lazy Generators**	`pyproject.toml` layout, `Result[T, E]` union, `RingBuffer[T]`, `ExitStack` transactional writer, O(1)-memory streaming JSONL benchmark log analyzer CLI
Month 2	**Base (Part 2)**	Weeks 5–8 (Days 29–56)	**Python Data Model, `__slots__`, Descriptors, Algorithmic Collections & Binary/Mmap I/O**	Slotted cache keys, `PureProperty` descriptor, `@retry` typed decorator, `FairRequestScheduler` (`heapq`), O(1) `LRUCache`, `TokenPrefixTrie`, `mmap` `.safetensors` binary parser
Month 3	**Base (Part 3)**	Weeks 9–12 (Days 57–84)	**Deep `asyncio` Concurrency, Dynamic Batching, Raw TCP/SSE/ASGI & Multiprocessing IPC**	`TaskGroup` supervisor, `AsyncDynamicBatcher`, length-prefixed TCP RPC, zero-dependency `SSEParser`, raw ASGI streaming server with graceful drain, `SharedMemory` zero-copy IPC
Month 4	**Base (Part 4)**	Weeks 13–16 (Days 85–112)	**Pythonic Design Patterns, Property Testing, SQLite Persistence & Base Mastery Capstone**	Strategy/Registry samplers, property-based state fuzzer, SQLite WAL benchmark repo, and `pyserve-core` **(Month 4 Base Mastery Capstone)**
Month 5	**Advanced (Part 1)**	Weeks 17–20 (Days 113–140)	**Metaprogramming, Metaclasses, AST/Bytecode Compilation, DI & Framework Authoring**	`MiniModule` (`nn.Module` clone), AST linter & constant folder, bytecode inspector, symbolic graph tracer, `NanoSchema` **(Pydantic clone)** & `NanoServe` **(FastAPI clone)**
Month 6	**Advanced (Part 2)**	Weeks 21–24 (Days 141–168)	**CPython Memory Internals, Buffer Protocol, Strided Tensors, Autograd, LLM & Paged KV Cache**	`SlabArenaAllocator` (`memoryview`), `StridedTensor` (zero-copy views & broadcasting), INT8/INT4 quantizer, `AutoTensor` autograd, `NanoLLM` (RoPE, GQA, KV Cache, BPE), **PagedAttention Block Manager & Continuous Batching Scheduler**
Month 7	**Advanced (Part 3)**	Weeks 25–28 (Days 169–196)	**Profiling, Flamegraphs, Roofline, GIL/Free-Threading & Native C/C++/CUDA/Triton Interop**	`SamplingFlamegraphProfiler`, `AllocationHotspotAnalyzer`, `RooflineModel`, zero-copy `ctypes` C kernels (`fast_ops.c`), DLPack bridge, `MetaTensor` 0-byte verifier, Tiled FlashAttention & CUDA Graph simulator
Month 8	**Advanced (Part 4)**	Weeks 29–32 (Days 197–224)	**Distributed IPC, Tensor Parallelism, Prometheus Engine, vLLM Architecture & Final Capstone**	Multi-rank TP worker group, Prometheus exposition engine, `PrefixAwareLoadBalancer`, guided JSON logits masker, LoRA cache manager, and `Mini-vLLM` **(Month 8 Advanced Mastery Capstone)**

3. Phase 1 — Base Concepts Mastery (Months 1–4: Days 1–112)¶

Every day below is a strictly bounded **1.0-hour engineering block** (15m reading, 30m from-scratch implementation, 15m testing/typing/failure diagnosis). By Day 112 (end of Month 4), you will have implemented every foundational and intermediate Python concept by hand to production quality.

Month 1: Modern Packaging, Strict Typing, Error Safety & Lazy Generators (Weeks 1–4, Days 1–28)¶

Week 1 - Modern Packaging, Strict Typing & Tooling Workshop¶

Day 1: Reproducible `pyproject.toml`, `src/` layout & virtual environments

CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Initialize a modern `src/` package layout with `pyproject.toml` (PEP 621), configure `ruff`, `mypy --strict`, and `pytest`, and write a zero-dependency package entrypoint.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day001/`, and review the mapped book section for `Reproducible pyproject.toml, src/ layout & virtual environments`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `py-mastery/` with `pyproject.toml`, `src/pymastery/__init__.py`, and `src/pymastery/env_info.py` that inspects

interpreter version, platform, and active prefix cleanly.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Reproducible` pyproject.toml, src/ `layout & virtual environments` and save the test log beside `pyproject.toml`, `src/pymastery/env_info.py`, and `tests/test_env_info.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day01/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Reproducible` pyproject.toml, src/ `layout & virtual environments`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day001/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** A fresh virtual environment installs the package in editable mode and passes `pytest`, `ruff`, and `mypy --strict` with zero warnings.

- **Tangible Deliverable:** `pyproject.toml`, `src/pymastery/env_info.py`, and `tests/test_env_info.py`
- **STOP WHEN:** A fresh virtual environment installs the package in editable mode and passes `pytest`, `ruff`, and `mypy --strict` with zero warnings.
- **Do Not Study Yet:** C-extension build backends, PyPI publishing workflows, and setuptools legacy `setup.py`.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 2: Primitive types, immutability, references & memory identity

CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master Python object identity (`id`), value equality (`==`), mutability hazards, interning, and defensive copying without external libraries.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day002/`, and review the mapped book section for `Primitive types, immutability, references & memory identity`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/refs.py` demonstrating deep vs. shallow copy mechanics on nested inference config structures and proving absence of aliasing bugs.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Primitive types, immutability, references & memory identity` and save the test log beside `src/pymastery/refs.py` and `tests/test_refs.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day02/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Primitive types, immutability, references & memory identity`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day002/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** All aliasing and immutability assertions pass and mutated nested structures never leak across state boundaries.

- **Tangible Deliverable:** `src/pymastery/refs.py` and `tests/test_refs.py`
- **STOP WHEN:** All aliasing and immutability assertions pass and mutated nested structures never leak across state boundaries.
- **Do Not Study Yet:** CPython `PyObject` C-struct headers and garbage collector generation tuning (covered in Month 6).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 3: Function signatures, positional-only, keyword-only & variadic typing CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Design unambiguous APIs using `/` (positional-only), `*` (keyword-only), `*args`, `**kwargs`, `Unpack`, and `TypedDict`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day003/`, and review the mapped book section for `Function signatures, positional-only, keyword-only & variadic typing`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/signatures.py` with a strict request-dispatch function enforcing positional-only tensor inputs and keyword-only generation hyperparameters.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Function signatures, positional-only, keyword-only & variadic typing` and save the test log beside `src/pymastery/signatures.py` and `tests/test_signatures.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day03/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Function signatures, positional-only, keyword-only & variadic typing`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day003/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** Misused positional/keyword arguments fail both at `mypy --strict` time and at runtime with clear `TypeError` diagnostics.

- **Tangible Deliverable:** `src/pymastery/signatures.py` and `tests/test_signatures.py`
- **STOP WHEN:** Misused positional/keyword arguments fail both at `mypy --strict` time and at runtime with clear `TypeError` diagnostics.
- **Do Not Study Yet:** Bytecode frame inspection and dynamic signature rewriting.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 4: Strict union types, `Optional`, `Literal`, `Final` & `TypeGuard`

CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate `NoneType` crashes and stringly-typed flags using `Literal`, `Final`, `TypeAlias`, and user-defined `TypeGuard` narrowing.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day004/`, and review the mapped book section for `Strict union types, Optional, Literal, Final & TypeGuard``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/type_narrowing.py` that validates raw JSON-like dicts into narrowed typed structures using `TypeGuard` functions.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Strict union types, Optional, Literal, Final & TypeGuard` and save the test log beside src/pymastery/type_narrowing.py `and` tests/test_type_narrowing.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day04/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Strict union types, Optional, Literal, Final & TypeGuard`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day004/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** `mypy --strict` proves narrowed types in both branches and runtime tests cover malformed inputs.

- **Tangible Deliverable:** `src/pymastery/type_narrowing.py` and `tests/test_type_narrowing.py`
- **STOP WHEN:** `mypy --strict` proves narrowed types in both branches and runtime tests cover malformed inputs.
- **Do Not Study Yet:** Metaclass-driven schema compilers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 5: Enums (`Enum`, `IntEnum`, `StrEnum`, `Flag`) & exhaustive pattern matching

CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Model finite state machines and engine status codes using `Enum`/`StrEnum` and structural `match/case` with `assert_never` exhaustiveness checking.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day005/`, and review the mapped book section for `Enums (Enum, IntEnum, StrEnum, Flag) & exhaustive pattern matching`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/states.py` modeling an inference request lifecycle (`QUEUED`, `PREFILL`, `DECODE`, `COMPLETED`, `FAILED`, `CANCELLED`) with exhaustive transition validation.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Enums (Enum, IntEnum, StrEnum, Flag) & exhaustive pattern matching` and save the test log beside `src/pymastery/states.py` and `tests/test_states.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day05/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Enums (Enum, IntEnum, StrEnum, Flag) & exhaustive pattern matching`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day005/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** Adding an unhandled enum member causes an immediate `mypy --strict` error via `typing.assert_never`.

- **Tangible Deliverable:** `src/pymastery/states.py` and `tests/test_states.py`
- **STOP WHEN:** Adding an unhandled enum member causes an immediate `mypy --strict` error via `typing.assert_never`.
- **Do Not Study Yet:** Async state machine executors.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 6: CLI engineering with `argparse`, exit codes & `stdin`/`stdout`/`stderr` streams CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build POSIX-compliant command-line tools with subcommands, typed arguments, structured error formatting to `stderr`, and deterministic exit codes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a

GPU.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day006/`, and review the mapped book section for `CLI engineering with` argparse`, exit codes &` stdin/stdout/stderr `streams`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/cli.py` and register a console script in `pyproject.toml` that validates config files and returns exit code `0` on valid input and `2` on validation failure.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `CLI engineering with` argparse`, exit codes &` stdin/stdout/stderr `streams` and save the test log beside `src/pymastery/cli.py` and `tests/test_cli.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day06/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `CLI engineering with` argparse`, exit codes &` stdin/stdout/stderr `streams`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day006/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** Subprocess tests verify `stdout`, `stderr`, and exit codes for valid, invalid, and missing-file inputs.

- **Tangible Deliverable:** `src/pymastery/cli.py` and `tests/test_cli.py`
- **STOP WHEN:** Subprocess tests verify `stdout`, `stderr`, and exit codes for valid, invalid, and missing-file inputs.
- **Do Not Study Yet:** Third-party CLI frameworks (`click`, `typer`).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 7: Week 1 Gate: Clean rebuild, zero-warning audit & bug repair

CORE: Base Track (Months 1-4) `[MINOR: Static typing (mypy --strict) and linter (ruff) enforcement]`¶

> **The Big Picture:** Set up a zero-compromise engineering environment (`pyproject.toml`, `uv`/`venv`, `ruff`, `mypy --strict`, `pytest`) where every line of code is verified automatically.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Consolidate Week 1 packaging, strict typing, enum state modeling, and CLI streams without consulting notes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Static typing (`mypy --strict`) and linter (`ruff`) enforcement — because the type checker and linter catch bugs before your code ever touches a GPU.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 8 & Robust Python (Viafore, O'Reilly) Ch. 1–3

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day007/`, and review the mapped book section for `Week 1 Gate: Clean rebuild, zero-warning audit & bug repair`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Delete `.venv`, recreate from scratch, run the full test and type-check suite, and fix a seeded mutable-default-argument and unnarrowed-`Optional` bug.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 1 Gate: Clean rebuild, zero-warning audit & bug repair` and save the test log beside `gates/week01-gate.md` and tagged clean commit.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Static typing (`mypy --strict`) and linter (`ruff`) enforcement: Run `mypy --strict src tests` and `ruff check src tests`; record zero errors in `artifacts/day07/quality.log`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 1 Gate: Clean rebuild, zero-warning audit & bug repair`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day007/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not use `Any`, `# type: ignore`, or untyped function signatures. Untyped code hides interface bugs until runtime.

> **Daily Gate (STOP WHEN):** Clean environment setup, `pytest`, `mypy --strict`, and `ruff check` all pass in under 10 seconds.

- **Tangible Deliverable:** `gates/week01-gate.md` and tagged clean commit
- **STOP WHEN:** Clean environment setup, `pytest`, `mypy --strict`, and `ruff check` all pass in under 10 seconds.
- **Do Not Study Yet:** Async I/O or network sockets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 2 - Data Modeling, Dataclasses, Generics & Structural Subtyping¶

Day 8: `dataclass` deep dive: `frozen`, `slots`, `kw_only`, `field(default_factory=...)` CORE: Base Track (Months 1-4) `[MINOR: Memory footprint (sys.getsizeof) and immutability verification]`¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build immutable, low-overhead domain records using `@dataclass(frozen=True, slots=True, kw_only=True)` and custom `__post_init__` validation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day008/`, and review the mapped book section for `` `dataclass` deep dive: `frozen`, `slots`, `kw_only`, `field(default_factory=...)` ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `InferenceRequest`, `SamplingParams`, and `TokenUsage` in `src/pymastery/models.py` with invariant checks in `__post_init__` (using `object.__setattr__` where normalization is needed).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `dataclass` deep dive: `frozen`, `slots`, `kw_only`, `field(default_factory=...) and save the test log beside` src/pymastery/models.py `and` tests/test_models.py ``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `dataclass` deep dive: `frozen`, `slots`, `kw_only`, `field(default_factory=...), verify ``

`that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day008/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** Invalid temperature, top_p, or max_tokens raise `ValueError` at construction time, and instances reject dynamic attribute creation.

- **Tangible Deliverable:** `src/pymastery/models.py` and `tests/test_models.py`
- **STOP WHEN:** Invalid temperature, top_p, or max_tokens raise `ValueError` at construction time, and instances reject dynamic attribute creation.
- **Do Not Study Yet:** Pydantic C-core internals.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 9: `NamedTuple` vs `TypedDict` vs `dataclass`: tradeoffs & conversion

CORE: Base Track (Months 1-4) `[MINOR: Memory footprint (sys.getsizeof) and immutability verification]`¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Choose the right record representation for binary/tuple unpacking (`NamedTuple`), external JSON boundaries (`TypedDict`), and rich domain models (`dataclass`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day009/`, and review the mapped book section for `NamedTuple` vs `TypedDict` vs `dataclass`: tradeoffs & conversion`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement lossless, strictly-typed converters between `TypedDict` wire payloads, `NamedTuple` metric rows, and `dataclass` domain objects in `src/pymastery/records.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `NamedTuple vs TypedDict vs dataclass: tradeoffs & conversion and save the test log beside src/pymastery/records.py and tests/test_records.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `NamedTuple vs TypedDict vs dataclass: tradeoffs & conversion, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day009/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** Round-trip serialization preserves all types and `sys.getsizeof` comparison is documented.

- **Tangible Deliverable:** `src/pymastery/records.py` and `tests/test_records.py`
- **STOP WHEN:** Round-trip serialization preserves all types and `sys.getsizeof` comparison is documented.
- **Do Not Study Yet:** Protocol Buffers or FlatBuffers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 10: Structural subtyping with `typing.Protocol` & `@runtime_checkable`

CORE: Base Track (Months 1-4) `[MINOR: Memory footprint (sys.getsizeof) and immutability verification]`¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Decouple components using duck-typed static interfaces (`Protocol`) instead of rigid inheritance hierarchies.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day010/`, and review the mapped book section for `Structural subtyping with` typing.Protocol `&` @runtime_checkable``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Define `TokenizerProtocol`, `SamplerProtocol`, and `MetricsSinkProtocol` in `src/pymastery/protocols.py` and implement two independent concrete implementations without subclassing.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Structural subtyping with` typing.Protocol `&` @runtime_checkable` `and save the test log beside` src/pymastery/protocols.py `and` tests/test_protocols.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Structural subtyping with` typing.Protocol `&` @runtime_checkable`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day010/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** `mypy --strict` verifies structural compatibility and rejected implementations fail static checks.

- **Tangible Deliverable:** `src/pymastery/protocols.py` and `tests/test_protocols.py`
- **STOP WHEN:** `mypy --strict` verifies structural compatibility and rejected implementations fail static checks.
- **Do Not Study Yet:** ABC metaclass internals.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 11: Abstract Base Classes (`abc.ABC`, `@abstractmethod`) vs `Protocol` CORE: Base Track (Months 1-4) [MINOR: Memory footprint (**sys.getsizeof**) and immutability verification]¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand nominal subtyping (`ABC`) for shared template-method logic vs structural subtyping (`Protocol`) for loose coupling.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day011/`, and review the mapped book section for `Abstract Base Classes (`abc.ABC`, `@abstractmethod`) vs` Protocol``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a base `BatchSchedulerABC` with shared invariant enforcement around an abstract `_select_next_batch` hook in `src/pymastery/scheduler_abc.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Abstract Base Classes (`abc.ABC`, `@abstractmethod`) vs` Protocol` `and save the test log beside` src/pymastery/scheduler_abc.py `and` tests/test_scheduler_abc.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Abstract Base Classes (`abc.ABC`, `@abstractmethod`) vs` Protocol`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day011/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** Incomplete subclasses fail instantiation with `TypeError` and concrete subclasses pass all invariant tests.

- **Tangible Deliverable:** `src/pymastery/scheduler_abc.py` and `tests/test_scheduler_abc.py`

- **STOP WHEN:** Incomplete subclasses fail instantiation with `TypeError` and concrete subclasses pass all invariant tests.
- **Do Not Study Yet:** Multiple inheritance diamond hierarchies.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 12: Generics (`TypeVar`, `Generic[T]`, variance, `ParamSpec`) from scratch

CORE: Base Track (Months 1-4) `[MINOR: Memory footprint (sys.getsizeof) and immutability verification]`¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write reusable, type-safe containers and higher-order wrappers using Python 3.12+ generic syntax (`class RingBuffer[T]:`) and `ParamSpec`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day012/`, and review the mapped book section for `Generics (TypeVar, Generic[T], variance, ParamSpec) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a generic bounded `RingBuffer[T]` and a `timed_call[P, R]` wrapper in `src/pymastery/generics.py` that preserves exact parameter and return types.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Generics (TypeVar, Generic[T], variance, ParamSpec) from scratch` and save the test log beside `src/pymastery/generics.py` and `tests/test_generics.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Generics (TypeVar, Generic[T], variance, ParamSpec) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day012/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** `RingBuffer[int]` rejects `str` in `mypy --strict` and `RingBuffer` handles wrap-around indexing accurately.

- **Tangible Deliverable:** `src/pymastery/generics.py` and `tests/test_generics.py`
- **STOP WHEN:** `RingBuffer[int]` rejects `str` in `mypy --strict` and `RingBuffer` handles wrap-around indexing accurately.
- **Do Not Study Yet:** C++ templates.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 13: Algebraic data modeling (Tagged Unions / Result pattern in Python) CORE: Base Track (Months 1-4) [MINOR: Memory footprint (sys.getsizeof) and immutability verification]¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Model recoverable operations explicitly using a generic `Result[T, E] = Ok[T] | Err[E]` union and structural pattern matching.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day013/`, and review the mapped book section for `Algebraic data modeling (Tagged Unions / Result pattern in Python)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `Ok[T]` and `Err[E]` frozen dataclasses in `src/pymastery/result.py` and refactor config parsing to return `Result[InferenceRequest, ValidationError]`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Algebraic data modeling (Tagged Unions / Result pattern in Python)` and save the test log beside `src/pymastery/result.py` and `tests/test_result.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Algebraic data modeling (Tagged Unions / Result pattern in Python)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day013/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** Callers must unpack `Ok` and `Err` branches explicitly and all edge cases are tested.

- **Tangible Deliverable:** `src/pymastery/result.py` and `tests/test_result.py`
- **STOP WHEN:** Callers must unpack `Ok` and `Err` branches explicitly and all edge cases are tested.
- **Do Not Study Yet:** Monadic category theory libraries.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 14: Week 2 Gate: Domain model & generic container sprint

CORE: Base Track (Months 1-4) `[MINOR: Memory footprint (sys.getsizeof) and immutability verification]`¶

> **The Big Picture:** Replace fragile dictionaries with self-validating, memory-efficient `dataclass` models, `Protocol` interfaces, and generic containers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a complete typed request/response/batch domain model and generic bounded queue from scratch without notes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Memory footprint (`sys.getsizeof`) and immutability verification — because data models sit on the hot path of every request and response.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 5, 13 & Robust Python (Viafore, O'Reilly) Ch. 4–10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day014/`, and review the mapped book section for `Week 2 Gate: Domain model & generic container sprint`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a mini batch-request validator and generic priority buffer, passing strict typing and 100% branch test coverage.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 2 Gate: Domain model & generic container sprint` and save the test log beside `gates/week02-gate.md` and `tests/test_week02_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Memory footprint (`sys.getsizeof`) and immutability verification: Verify `dataclass(frozen=True, slots=True)` prevents attribute mutation and check `__slots__` presence in unit tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 2 Gate: Domain model & generic container sprint`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day014/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not pass raw `dict[str, Any]` across module boundaries. Use frozen, slotted dataclasses or `NamedTuple` so invalid states are unrepresentable.

> **Daily Gate (STOP WHEN):** All static checks and unit tests pass from a clean checkout.

- **Tangible Deliverable:** `gates/week02-gate.md` and `tests/test_week02_gate.py`
- **STOP WHEN:** All static checks and unit tests pass from a clean checkout.
- **Do Not Study Yet:** Async coroutines.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 3 - Defensive Error Handling, Context Managers & Testing Craftsmanship¶

Day 15: Custom exception hierarchies, exception chaining (`from e`) & `ExceptionGroup` CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Design a structured domain exception tree (`InferenceError` -> `ValidationError`, `CapacityExceededError`, `TimeoutError`) with contextual metadata and Python 3.11+ `ExceptionGroup` / `except*`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day015/`, and review the mapped book section for `Custom exception hierarchies, exception chaining (from e) & ExceptionGroup``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/errors.py` with structured error codes, cause chaining, and a multi-worker error aggregator using `ExceptionGroup` and `except*`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Custom exception hierarchies, exception chaining (from e) & ExceptionGroup` and save the test log beside src/pymastery/errors.py and tests/test_errors.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Custom exception hierarchies, exception chaining (from e) & ExceptionGroup`, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day015/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** `__cause__` is preserved on wrapped exceptions and `except*` cleanly separates timeout errors from validation errors.

- **Tangible Deliverable:** `src/pymastery/errors.py` and `tests/test_errors.py`
- **STOP WHEN:** `__cause__` is preserved on wrapped exceptions and `except*` cleanly separates timeout errors from validation errors.
- **Do Not Study Yet:** Asyncio task group cancellation (covered in Month 3).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 16: Context managers (`__enter__`, `__exit__`) & `@contextlib.contextmanager` CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Guarantee deterministic setup and teardown of locks, temp directories, timers, and state transitions using the context manager protocol.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day016/`, and review the mapped book section for `Context managers (__enter__, __exit__) & @contextlib.contextmanager``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement both a class-based `StopwatchContext` and a generator-based `@contextmanager atomic_file_writer` in `src/pymastery/contexts.py` that cleans up temp files on exception.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Context managers (__enter__, __exit__) & @contextlib.contextmanager` and save the test log beside src/pymastery/contexts.py `and` tests/test_contexts.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Context managers (__enter__, __exit__) & @contextlib.contextmanager`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day016/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** Simulated mid-write exceptions leave zero partial temp files on disk and suppress nothing unless explicitly configured.

- **Tangible Deliverable:** `src/pymastery/contexts.py` and `tests/test_contexts.py`
- **STOP WHEN:** Simulated mid-write exceptions leave zero partial temp files on disk and suppress nothing unless explicitly configured.
- **Do Not Study Yet:** Async context managers (`__aenter__`).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 17: `contextlib.ExitStack` for dynamic resource management

CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Manage an arbitrary runtime-determined number of resources (files, sockets, locks, callbacks) safely using `ExitStack`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day017/`, and review the mapped book section for `` `contextlib.ExitStack` for dynamic resource management` ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a multi-shard log writer in `src/pymastery/exitstack_demo.py` that opens N shard files cleanly and guarantees all N close even if shard K fails on open.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `contextlib.ExitStack` for dynamic resource management and save the test log beside src/pymastery/exitstack_demo.py and tests/test_exitstack_demo.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `contextlib.ExitStack` for dynamic resource management, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day017/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** Fault injection on shard K opening proves shards `0..K-1` are deterministically closed.

- **Tangible Deliverable:** `src/pymastery/exitstack_demo.py` and `tests/test_exitstack_demo.py`
- **STOP WHEN:** Fault injection on shard K opening proves shards `0..K-1` are deterministically closed.
- **Do Not Study Yet:** Distributed two-phase commit.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 18: Assertions, preconditions, invariants & defensive programming

CORE: Base Track (Months 1-4) [MINOR: Exception chaining (raise ... from ...) and resource leak checks]¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Distinguish between programmer assertions (`assert`, which `-O` strips) and runtime preconditions/invariants that must always execute.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.

- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day018/`, and review the mapped book section for `Assertions, preconditions, invariants & defensive programming`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `require_shape`, `require_range`, and `require_non_empty` invariant helpers in `src/pymastery/contracts.py` and test under both normal Python and `python -O`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Assertions, preconditions, invariants & defensive programming` and save the test log beside `src/pymastery/contracts.py` and `tests/test_contracts.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Assertions, preconditions, invariants & defensive programming`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day018/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** Running `python -O -m pytest` still enforces every runtime contract and raises descriptive errors.

- **Tangible Deliverable:** `src/pymastery/contracts.py` and `tests/test_contracts.py`
- **STOP WHEN:** Running `python -O -m pytest` still enforces every runtime contract and raises descriptive errors.
- **Do Not Study Yet:** Formal verification theorem provers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 19: `pytest` mastery: fixtures, `tmp_path`, `monkeypatch`, `caplog` & parametrization CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write expressive, fast, isolated test suites using `pytest.mark.parametrize`, dependency-injected fixtures, and temporary filesystem isolation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day019/`, and review the mapped book section for `` `pytest` mastery: fixtures, `tmp_path`, `monkeypatch`, `caplog` & parametrization``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create a comprehensive test suite `tests/test_parametrized_suite.py` testing boundary conditions, environment variable overrides (`monkeypatch`), and log capture (`caplog`).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `pytest` mastery: fixtures, `tmp_path`, `monkeypatch`, `caplog` & parametrization `and save the test log beside` tests/test_parametrized_suite.py``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `pytest` mastery: fixtures, `tmp_path`, `monkeypatch`, `caplog` & parametrization`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day019/failure-notes.md``.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** 25+ parametrized boundary cases run in < 0.2s with zero shared state between tests.

- **Tangible Deliverable:** `tests/test_parametrized_suite.py`
- **STOP WHEN:** 25+ parametrized boundary cases run in < 0.2s with zero shared state between tests.
- **Do Not Study Yet:** Browser/UI end-to-end testing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 20: Test doubles from scratch vs `unittest.mock`: fakes, stubs & spies

CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prefer lightweight in-memory Fakes implementing a `Protocol` over brittle mock-patching, while using `Mock`/`autospec` surgically when needed.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day020/`, and review the mapped book section for `Test doubles from scratch vs` unittest.mock`: fakes, stubs & spies`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `FakeClock` and `FakeMetricsSink` in `src/pymastery/testing_fakes.py` to test a rate-limiter deterministically without `time.sleep`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Test doubles from scratch vs` unittest.mock`: fakes, stubs & spies` and save the test log beside `src/pymastery/testing_fakes.py` and `tests/test_rate_limiter.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Test doubles from scratch vs` unittest.mock`: fakes, stubs & spies`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day020/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

> **Daily Gate (STOP WHEN):** Rate-limiter tests simulate 60 seconds of token bucket refill in 2 milliseconds using `FakeClock`.

- **Tangible Deliverable:** `src/pymastery/testing_fakes.py` and `tests/test_rate_limiter.py`
- **STOP WHEN:** Rate-limiter tests simulate 60 seconds of token bucket refill in 2 milliseconds using `FakeClock`.
- **Do Not Study Yet:** Heavy mocking frameworks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 21: Week 3 Gate: Fault-tolerant resource manager & test audit

CORE: Base Track (Months 1-4) `[MINOR: Exception chaining (raise ... from ...) and resource leak checks]`¶

> **The Big Picture:** Write resilient code that cleans up resources deterministically on failure and is backed by high-signal unit and edge-case tests.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Demonstrate rock-solid exception handling, `ExitStack` resource safety, and deterministic fake-based testing.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Exception chaining (`raise ... from ...`) and resource leak checks — because swallowed exceptions and leaked file/GPU handles cause silent production outages.
- **Mapped Book & Docs:** Effective Python, 3rd Ed. (Slatkin, Addison-Wesley) Ch. 2, 8 & Python Testing with pytest, 2nd Ed. (Okken, Pragmatic) Ch. 1–6

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day021/`, and review the mapped book section for `Week 3 Gate: Fault-tolerant resource manager & test audit`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Build a transactional checkpoint writer with full fault-injection tests (disk full, permission error, mid-stream abort).

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 3 Gate: Fault-tolerant resource manager & test audit` and save the test log beside `gates/week03-gate.md` and `src/pymastery/checkpoint.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Exception chaining (`raise ... from ...`) and resource leak checks: Verify every custom exception preserves `__cause__` and every resource test passes with `-W error` (warnings as errors).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 3 Gate: Fault-tolerant resource manager & test audit`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day021/failure-notes.md`.

Engineering Trap to Avoid: Never write bare `except:` or `except Exception: pass`. Always catch specific exceptions, attach context, or re-raise cleanly.

Daily Gate (STOP WHEN): All fault-injection tests pass with zero leaked file descriptors and zero unhandled exceptions.

- **Tangible Deliverable:** `gates/week03-gate.md` and `src/pymastery/checkpoint.py`
- **STOP WHEN:** All fault-injection tests pass with zero leaked file descriptors and zero unhandled exceptions.
- **Do Not Study Yet:** Multithreading.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 4 - Iterators, Generators, Lazy Pipelines & Clean Architecture¶

Day 22: The Iterator Protocol (`__iter__`, `__next__`, `StopIteration`) from scratch CORE: Base Track (Months 1-4) `[MINOR: Peak memory complexity (tracemalloc) on streaming data]`¶

The Big Picture: Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.

Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement custom stateful iterators and reusable iterable objects (separating the `Iterable` container from the `Iterator` cursor).

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day022/`, and review the mapped book section for `The Iterator Protocol (__iter__, __next__, StopIteration) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `BatchChunkIterable[T]` and `BatchChunkIterator[T]` in `src/pymastery/iterators.py` that can be iterated multiple times independently without exhausting on the first pass.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The Iterator Protocol (__iter__, __next__, StopIteration) from scratch` and save the test log beside `src/pymastery/iterators.py` and `tests/test_iterators.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The Iterator Protocol (__iter__, __next__, StopIteration) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day022/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** Two simultaneous `for` loops over the same `BatchChunkIterable` instance yield complete, independent sequences.

- **Tangible Deliverable:** `src/pymastery/iterators.py` and `tests/test_iterators.py`
- **STOP WHEN:** Two simultaneous `for` loops over the same `BatchChunkIterable` instance yield complete, independent sequences.
- **Do Not Study Yet:** C-level iterator slots.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 23: Generators (`yield`, `yield from`), lazy evaluation & O(1) memory pipelines CORE: Base Track (Months 1-4) [MINOR: Peak memory complexity (**tracemalloc**) on streaming data]¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Compose multi-stage streaming pipelines (read -> filter -> transform -> batch) using generator functions and `yield from`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day023/`, and review the mapped book section for `Generators (yield, yield from), lazy evaluation & O(1) memory pipelines`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a lazy JSONL benchmark log pipeline in `src/pymastery/lazy_pipeline.py` that streams 100,000 synthetic request records and computes sliding-window latency stats in < 2 MB peak RAM.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Generators (yield, yield from), lazy evaluation & O(1) memory pipelines` and save the test log beside `src/pymastery/lazy_pipeline.py` and `tests/test_lazy_pipeline.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Generators (yield, yield from), lazy evaluation & O(1) memory pipelines`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day023/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** `tracemalloc` test proves peak memory stays below 2 MB regardless of stream length.

- **Tangible Deliverable:** `src/pymastery/lazy_pipeline.py` and `tests/test_lazy_pipeline.py`

- **STOP WHEN:** `tracemalloc` test proves peak memory stays below 2 MB regardless of stream length.
- **Do Not Study Yet:** Distributed Spark/Ray datasets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 24: `itertools` mastery: `islice`, `chain`, `groupby`, `batched`, `tee`, `accumulate`

CORE: Base Track (Months 1-4) `[MINOR: Peak memory complexity (tracemalloc) on streaming data]`¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine standard library C-speed iterator building blocks (`itertools.batched`, `islice`, `groupby`, `chain.from_iterable`) for high-throughput data manipulation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day024/`, and review the mapped book section for `itertools mastery: islice, chain, groupby, batched, tee, accumulate`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement token-stream padding, dynamic micro-batching, and sorted run-length grouping in `src/pymastery/itertools_ops.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `itertools mastery: islice, chain, groupby, batched, tee, accumulate and save the test log beside` src/pymastery/itertools_ops.py `and` tests/test_itertools_ops.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `itertools mastery: islice, chain, groupby, batched, tee, accumulate, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day024/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** All edge cases (empty stream, partial final batch, single-element stream) pass strict tests.

- **Tangible Deliverable:** `src/pymastery/itertools_ops.py` and `tests/test_itertools_ops.py`
- **STOP WHEN:** All edge cases (empty stream, partial final batch, single-element stream) pass strict tests.
- **Do Not Study Yet:** NumPy vectorization (Month 6).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 25: Two-way generators (`send`, `throw`, `close`) & generator cleanup

CORE: Base Track (Months 1-4) `[MINOR: Peak memory complexity (tracemalloc) on streaming data]`¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how `yield` expressions receive values, handle injected exceptions (`throw`), and run `try/finally` on `close()` or garbage collection.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day025/`, and review the mapped book section for `Two-way generators (send, throw, close) & generator cleanup`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a streaming running-percentile accumulator coroutine in `src/pymastery/coro_accumulator.py` with guaranteed flush-on-close semantics.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Two-way generators (send, throw, close) & generator cleanup` and save the test log beside `src/pymastery/coro_accumulator.py` and `tests/test_coro_accumulator.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Two-way generators (send, throw, close) & generator cleanup`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day025/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** Closing the generator triggers final summary emission and prevents `GeneratorExit` swallowing bugs.

- **Tangible Deliverable:** `src/pymastery/coro_accumulator.py` and `tests/test_coro_accumulator.py`
- **STOP WHEN:** Closing the generator triggers final summary emission and prevents `GeneratorExit` swallowing bugs.
- **Do Not Study Yet:** Legacy `@asyncio.coroutine`.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 26: Clean Architecture in Python: Domain vs Infrastructure & Dependency Inversion CORE: Base Track (Months 1-4) [MINOR: Peak memory complexity (tracemalloc) on streaming data]¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Structure multi-module Python packages with pure domain logic at the core and I/O adapters injected via `Protocol` ports.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day026/`, and review the mapped book section for `Clean Architecture in Python: Domain vs Infrastructure & Dependency Inversion`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Refactor Week 1–4 components into `domain/`, `ports/`, `adapters/`, and `cli/` subpackages with zero upward imports from `domain/` to `adapters/`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Clean Architecture in Python: Domain vs Infrastructure & Dependency Inversion` and save the test log beside `src/pymastery/domain/`, `src/pymastery/ports/`, `src/pymastery/adapters/`, and `tests/test_architecture.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Clean Architecture in Python: Domain vs Infrastructure & Dependency Inversion`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day026/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** An AST import-linter test proves `domain/` imports nothing from `adapters/` or external I/O libraries.

- **Tangible Deliverable:** `src/pymastery/domain/`, `src/pymastery/ports/`, `src/pymastery/adapters/`, and `tests/test_architecture.py`
- **STOP WHEN:** An AST import-linter test proves `domain/` imports nothing from `adapters/` or external I/O libraries.
- **Do Not Study Yet:** Enterprise Inversion-of-Control containers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 27: Month 1 Milestone Project: Streaming Benchmark Log Analyzer CLI CORE: Base Track (Months 1-4) [MINOR: Peak memory complexity (**tracemalloc**) on streaming data]¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a complete, packaged, strictly-typed, O(1)-memory CLI application that ingests multi-megabyte JSONL inference logs, filters anomalies, and outputs p50/p95/p99 TTFT and ITL tables.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day027/`, and review the mapped book section for `Month 1 Milestone Project: Streaming Benchmark Log Analyzer CLI`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement the end-to-end analyzer in `src/pymastery/log_analyzer.py` with CLI flags, `ExitStack` file handling, `Result` error reporting, and `tracemalloc` budget verification.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Month 1 Milestone Project: Streaming Benchmark Log Analyzer CLI` and save the test log beside `src/pymastery/log_analyzer.py`, `tests/test_log_analyzer.py`, and `artifacts/month01-report.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Month 1 Milestone Project: Streaming Benchmark Log Analyzer CLI`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day027/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** Processes 250,000 JSONL lines in under 1.5 seconds with < 5 MB peak heap and passes all strict checks.

- **Tangible Deliverable:** `src/pymastery/log_analyzer.py`, `tests/test_log_analyzer.py`, and `artifacts/month01-report.md`
- **STOP WHEN:** Processes 250,000 JSONL lines in under 1.5 seconds with < 5 MB peak heap and passes all strict checks.
- **Do Not Study Yet:** Web frameworks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 28: Week 4 Gate (Month 1 Review): Code quality & architecture defense CORE: Base Track (Months 1-4) [MINOR: Peak memory complexity (tracemalloc) on streaming data]¶

> **The Big Picture:** Process gigabytes of logs, token streams, and datasets in O(1) memory using Python's iterator protocol and generator pipelines.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Audit Month 1 codebase for 100% `mypy --strict` compliance, zero `ruff` warnings, clean architecture boundaries, and test coverage.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Peak memory complexity (`tracemalloc`) on streaming data — because materializing giant lists in memory causes OOM kills in serving pipelines.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 17 & Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 1–4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day028/`, and review the mapped book section for `Week 4 Gate (Month 1 Review): Code quality & architecture defense`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a cold-start verification script, introduce and catch an architectural boundary violation, and complete a 10-question closed-book recall test.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 4 Gate (Month 1 Review): Code quality & architecture defense` and save the test log beside `gates/month01-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Peak memory complexity (`tracemalloc`) on streaming data: Wrap generator pipelines in `tracemalloc.start()` / `get_traced_memory()` to verify O(1) peak memory across 100,000 items.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 4 Gate (Month 1 Review): Code quality & architecture defense`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day028/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not call `list(...)` on an iterator unless you actually need random indexing or multiple passes. Keep pipelines lazy.

> **Daily Gate (STOP WHEN):** All 4 weeks of tests pass in a single run, coverage > 90%, and zero static analysis warnings.

- **Tangible Deliverable:** `gates/month01-gate.md`
- **STOP WHEN:** All 4 weeks of tests pass in a single run, coverage > 90%, and zero static analysis warnings.
- **Do Not Study Yet:** Metaclasses.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 2: Python Data Model, Descriptors, Algorithmic Structures & Binary I/O (Weeks 5–8, Days 29–56)¶

Week 5 - The Python Data Model, Object Internals & `__slots__`¶

Day 29: Object representation (`__repr__`, `__str__`, `__format__`, `__bytes__`)

CORE: Base Track (Months 1-4) `[MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write unambiguous developer-facing `__repr__` (round-trippable where practical) and custom format specifiers (`__format__`) for engineering telemetry.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day029/`, and review the mapped book section for `Object representation (__repr__, __str__, __format__, __bytes__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `TensorShape` and `LatencyStat` in `src/pymastery/datamodel_repr.py` supporting `f'{stat:.2f_ms}'` and `f'{stat:compact}'` custom format specs.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Object representation (__repr__, __str__, __format__, __bytes__)` and save the test log beside `src/pymastery/datamodel_repr.py` and `tests/test_datamodel_repr.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Object representation (__repr__, __str__, __format__, __bytes__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day029/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** `repr()`, `str()`, and custom `format()` specs match exact expected strings.

- **Tangible Deliverable:** `src/pymastery/datamodel_repr.py` and `tests/test_datamodel_repr.py`
- **STOP WHEN:** `repr()`, `str()`, and custom `format()` specs match exact expected strings.
- **Do Not Study Yet:** Locale-specific internationalization.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 30: Equality, ordering (`__eq__`, `@total_ordering`) & hashing (`__hash__`) invariants CORE: Base Track (Months 1-4) `[MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement polymorphic equality returning `NotImplemented` for unknown types, `@functools.total_ordering` for priority scheduling, and composite tuple hashing.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day030/`, and review the mapped book section for `Equality, ordering (__eq__, @total_ordering) & hashing (__hash__) invariants`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `RequestPriorityKey` and `KVCacheBlockKey` in `src/pymastery/hashing.py` usable as `dict` keys and in `heapq` priority queues.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Equality, ordering (__eq__, @total_ordering) & hashing (__hash__) invariants` and save the test log beside `src/pymastery/hashing.py` and `tests/test_hashing.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Equality, ordering (__eq__, @total_ordering) & hashing (__hash__) invariants`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day030/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** Hash consistency holds across 10,000 keys and incompatible type comparisons safely return `False` without raising `TypeError`.

- **Tangible Deliverable:** `src/pymastery/hashing.py` and `tests/test_hashing.py`
- **STOP WHEN:** Hash consistency holds across 10,000 keys and incompatible type comparisons safely return `False` without raising `TypeError`.
- **Do Not Study Yet:** Cryptographic hashing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 31: Emulating sequences & mappings (`__len__`, `__getitem__`, `slice`, `__contains__`) CORE: Base Track (Months 1-4) `[MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build custom sequence-like and mapping-like objects that support negative indexing, `slice` objects (`start:stop:step`), and O(1) membership checks.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day031/`, and review the mapped book section for `Emulating sequences & mappings (__len__, __getitem__, slice, __contains__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `TokenSequenceView` in `src/pymastery/sequences.py` supporting zero-copy window slicing (`seq[10:50]`) without duplicating the backing token array.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Emulating sequences & mappings (__len__, __getitem__, slice, __contains__)` and save the test log beside `src/pymastery/sequences.py` and `tests/test_sequences.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Emulating sequences & mappings (__len__, __getitem__, slice, __contains__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day031/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** Indexing, negative indices, slices, `len()`, and `in` work identically to standard sequences without copying the backing buffer.

- **Tangible Deliverable:** `src/pymastery/sequences.py` and `tests/test_sequences.py`

- **STOP WHEN:** Indexing, negative indices, slices, `len()`, and `in` work identically to standard sequences without copying the backing buffer.
- **Do Not Study Yet:** C-level buffer protocol (covered in Month 6).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 32: Operator overloading (`__add__`, `__radd__`, `__iadd__`, `__matmul__`)

CORE: Base Track (Months 1-4) `[MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement clean, type-safe arithmetic and matrix-multiplication (`@`) operators with proper reverse (`__r*__`) and in-place (`__i*__`) semantics.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day032/`, and review the mapped book section for `Operator overloading (__add__, __radd__, __iadd__, __matmul__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MetricVector` and `Shape2D` in `src/pymastery/operators.py` supporting `+`, `+=`, scalar `*`, and `@` dimension contraction checks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Operator overloading (__add__, __radd__, __iadd__, __matmul__)` and save the test log beside `src/pymastery/operators.py` and `tests/test_operators.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Operator overloading (__add__, __radd__, __iadd__, __matmul__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day032/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** Commutative operations with scalars, in-place mutation rules, and dimension mismatch errors all pass unit tests.

- **Tangible Deliverable:** `src/pymastery/operators.py` and `tests/test_operators.py`
- **STOP WHEN:** Commutative operations with scalars, in-place mutation rules, and dimension mismatch errors all pass unit tests.
- **Do Not Study Yet:** Full BLAS linear algebra.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 33: `__slots__`, instance `__dict__` elimination & memory layout benchmarking CORE: Base Track (Months 1-4) `[MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how CPython stores attributes in per-instance `__dict__` hash tables vs compact fixed-offset `__slots__` arrays.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day033/`, and review the mapped book section for `__slots__, instance __dict__ elimination & memory layout benchmarking`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Benchmark 200,000 instances of `SlottedTokenMeta` vs `DictTokenMeta` in `src/pymastery/slots_bench.py` using `tracemalloc`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `__slots__, instance __dict__ elimination & memory layout benchmarking and save the test log`

beside src/pymastery/slots_bench.py, tests/test_slots_bench.py, and artifacts/day33-memory.md`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `__slots__, instance __dict__ elimination & memory layout benchmarking, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day033/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** `SlottedTokenMeta` demonstrates >50% total memory reduction over `DictTokenMeta` for 200,000 objects.

- **Tangible Deliverable:** `src/pymastery/slots_bench.py`, `tests/test_slots_bench.py`, and `artifacts/day33-memory.md`
- **STOP WHEN:** `SlottedTokenMeta` demonstrates >50% total memory reduction over `DictTokenMeta` for 200,000 objects.
- **Do Not Study Yet:** Custom C malloc arenas.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 34: Weak references (`weakref.ref`, `WeakValueDictionary`, `finalize`) & cache eviction CORE: Base Track (Months 1-4) [MINOR: Object memory size (sys.getsizeof + __slots__) and hash invariants]¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Avoid reference cycles and memory leaks in object registries and parent-child trees using `weakref`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day034/`, and review the mapped book section for `Weak references (`weakref.ref`,` WeakValueDictionary`,` finalize`) & cache eviction`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Implement an active session registry using `WeakValueDictionary` and deterministic cleanup callbacks with `weakref.finalize` in `src/pymastery/weak_registry.py`.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Weak references (`weakref.ref`,` WeakValueDictionary`,` finalize`) & cache eviction` and save the test log beside `src/pymastery/weak_registry.py` and `tests/test_weak_registry.py`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Weak references (`weakref.ref`,` WeakValueDictionary`,` finalize`) & cache eviction`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day034/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** Dropping strong references to session objects automatically removes them from the registry without manual deregistration.

- **Tangible Deliverable:** `src/pymastery/weak_registry.py` and `tests/test_weak_registry.py`
- **STOP WHEN:** Dropping strong references to session objects automatically removes them from the registry without manual deregistration.
- **Do Not Study Yet:** Cyclic GC C-API internals.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 35: Week 5 Gate: Custom sequence & slotted cache key review

CORE: Base Track (Months 1-4) `[MINOR: Object memory size (`**sys.getsizeof + __slots__**`) and hash invariants]`¶

> **The Big Picture:** Master how Python objects behave under the hood by implementing special dunder methods, custom hashing, `__slots__`, and weak references.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine dunder protocols, slicing, hashing, `__slots__`, and weak references into a unified cache-index component.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants — because custom tensor wrappers, cache keys, and batch containers rely on the Python Data Model.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 1, 11, 12, 16

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day035/`, and review the mapped book section for `Week 5 Gate: Custom sequence & slotted cache key review`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Build a slotted, hashable prefix-tree node and sliceable token view, passing all memory and correctness checks.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 5 Gate: Custom sequence & slotted cache key review` and save the test log beside `gates/week05-gate.md` and `tests/test_week05_gate.py`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Object memory size (`sys.getsizeof` + `__slots__`) and hash invariants: Verify `a == b` implies `hash(a) == hash(b)` and benchmark memory reduction of `__slots__` over 100,000 instances.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 5 Gate: Custom sequence & slotted cache key review`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day035/failure-notes.md`.

> **Engineering Trap to Avoid:** Never implement `__hash__` on a mutable object or override `__eq__` without considering hashability and `NotImplemented` return semantics.

> **Daily Gate (STOP WHEN):** Zero memory leaks under `gc.collect()` tests and 100% pass rate.

- **Tangible Deliverable:** `gates/week05-gate.md` and `tests/test_week05_gate.py`
- **STOP WHEN:** Zero memory leaks under `gc.collect()` tests and 100% pass rate.
- **Do Not Study Yet:** Descriptors.

- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 6 - First-Class Functions, Closures, Decorators & Descriptors¶

Day 36: First-class functions, closures, `nonlocal`, cell objects & late-binding traps CORE: Base Track (Months 1-4) `[MINOR: Function signature preservation (functools.wraps + ParamSpec)]`¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand lexical scoping, `__closure__` cells, and the classic loop-variable late-binding pitfall in lambdas/closures.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day036/`, and review the mapped book section for `First-class functions, closures, nonlocal, cell objects & late-binding traps`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement stateful closure-based counters, moving-average filters, and safe factory functions in `src/pymastery/closures.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `First-class functions, closures, nonlocal, cell objects & late-binding traps` and save the test log beside `src/pymastery/closures.py` and `tests/test_closures.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `First-class functions, closures, nonlocal, cell objects & late-binding traps`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day036/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** Closure state is isolated per factory call and loop-created callbacks bind the exact iteration value.

- **Tangible Deliverable:** `src/pymastery/closures.py` and `tests/test_closures.py`
- **STOP WHEN:** Closure state is isolated per factory call and loop-created callbacks bind the exact iteration value.
- **Do Not Study Yet:** Bytecode cellvars disassembly.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 37: Typed decorators (`ParamSpec`, `TypeVar`, `functools.wraps`) & parameterized decorators CORE: Base Track (Months 1-4) `[MINOR: Function signature preservation (functools.wraps + ParamSpec)]`¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write both bare (`@retry`) and parameterized (`@retry(max_attempts=3, backoff=0.01)`) decorators with 100% static type preservation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day037/`, and review the mapped book section for `Typed decorators (ParamSpec, TypeVar, functools.wraps) & parameterized decorators`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `@retry`, `@trace_execution`, and `@enforce_timeout_budget` in `src/pymastery/decorators.py` supporting both `@dec` and `@dec(...)` syntax.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Typed decorators (ParamSpec, TypeVar, functools.wraps) & parameterized decorators` and save the test log beside `src/pymastery/decorators.py` and `tests/test_decorators.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Typed decorators (ParamSpec, TypeVar, functools.wraps) & parameterized decorators`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day037/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** `mypy --strict` catches wrong argument types on decorated functions and `__wrapped__` allows bypassing the decorator in tests.

- **Tangible Deliverable:** `src/pymastery/decorators.py` and `tests/test_decorators.py`
- **STOP WHEN:** `mypy --strict` catches wrong argument types on decorated functions and `__wrapped__` allows bypassing the decorator in tests.
- **Do Not Study Yet:** Async decorators (covered in Month 3).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 38: `functools` deep dive: `partial`, `lru_cache`, `cache`, `singledispatch`

CORE: Base Track (Months 1-4) `[MINOR: Function signature preservation (functools.wraps + ParamSpec)]`¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use `functools.singledispatch` for clean type-based polymorphism and `lru_cache` with cache-info inspection (`hits`, `misses`, `currsize`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day038/`, and review the mapped book section for `functools deep dive: partial, lru_cache, cache, singledispatch`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a polymorphic tensor/payload serializer using `@singledispatch` and a bounded prompt-tokenization cache in `src/pymastery/functools_ops.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `functools deep dive: partial, lru_cache, cache, singledispatch and save the test log beside src/pymastery/functools_ops.py and tests/test_functools_ops.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `functools deep dive: partial, lru_cache, cache, singledispatch, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day038/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** Dispatch handles `int`, `float`, `list`, and custom classes cleanly, and `cache_info()` proves expected hit/miss counts.

- **Tangible Deliverable:** `src/pymastery/functools_ops.py` and `tests/test_functools_ops.py`
- **STOP WHEN:** Dispatch handles `int`, `float`, `list`, and custom classes cleanly, and `cache_info()` proves expected hit/miss counts.
- **Do Not Study Yet:** Distributed Redis caching.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 39: The Descriptor Protocol (`__get__`, `__set__`, `__set_name__`) from scratch CORE: Base Track (Months 1-4) `[MINOR: Function signature preservation (functools.wraps + ParamSpec)]`¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master the mechanism behind Python's dot-operator (`obj.attr`): data vs non-data descriptors and automatic attribute naming via `__set_name__`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day039/`, and review the mapped book section for `The Descriptor Protocol (__get__, __set__, __set_name__) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement reusable validated descriptors (`BoundedFloat`, `PositiveInt`, `NonEmptyString`) in `src/pymastery/descriptors.py` that store values in per-instance private storage.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The Descriptor Protocol (__get__, __set__, __set_name__) from scratch` and save the test log beside `src/pymastery/descriptors.py` and `tests/test_descriptors.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The Descriptor Protocol (__get__, __set__, __set_name__) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day039/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** Multiple instances of a class using `BoundedFloat` maintain independent values and reject out-of-range assignments immediately.

- **Tangible Deliverable:** `src/pymastery/descriptors.py` and `tests/test_descriptors.py`
- **STOP WHEN:** Multiple instances of a class using `BoundedFloat` maintain independent values and reject out-of-range assignments immediately.
- **Do Not Study Yet:** Metaclass descriptor synthesis (Month 5).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 40: How `property`, `@classmethod`, `@staticmethod` & bound methods actually work CORE: Base Track (Months 1-4) `[MINOR: Function signature preservation (functools.wraps + ParamSpec)]`¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Re-implement Python's built-in `Property`, `ClassMethod`, and `StaticMethod` in pure Python using the descriptor protocol to demystify method binding.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day040/`, and review the mapped book section for `How property, @classmethod, @staticmethod & bound methods actually work`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write `PureProperty`, `PureClassMethod`, and `PureStaticMethod` in `src/pymastery/pure_descriptors.py` and prove identical behavior to built-ins.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `How property, @classmethod, @staticmethod & bound methods actually work` and save the test log beside `src/pymastery/pure_descriptors.py` and `tests/test_pure_descriptors.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `How property, @classmethod, @staticmethod & bound methods actually work`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day040/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** Accessing a function on a class returns the function; accessing on an instance returns a bound method wrapper.

- **Tangible Deliverable:** `src/pymastery/pure_descriptors.py` and `tests/test_pure_descriptors.py`
- **STOP WHEN:** Accessing a function on a class returns the function; accessing on an instance returns a bound method wrapper.
- **Do Not Study Yet:** C-level method descriptor tables.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 41: Lazy attributes (`@cached_property`) & attribute lookup chain (`__getattribute__` vs `__getattr__`) CORE: Base Track (Months 1-4) [MINOR: Function signature preservation (functools.wraps + ParamSpec)]¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Control dynamic attribute fallback (`__getattr__`) without breaking normal lookup or causing infinite recursion in `__getattribute__`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day041/`, and review the mapped book section for `Lazy attributes (@cached_property) & attribute lookup chain (__getattribute__ vs __getattr__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a zero-overhead `LazyConfigProxy` and a custom `@slotted_cached_property` in `src/pymastery/attr_access.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Lazy attributes (@cached_property) & attribute lookup chain (__getattribute__ vs __getattr__)` and save the test log beside `src/pymastery/attr_access.py` and `tests/test_attr_access.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Lazy attributes (@cached_property) & attribute lookup chain (__getattribute__ vs __getattr__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day041/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** Expensive computation runs exactly once on first access and subsequent reads are O(1) attribute lookups.

- **Tangible Deliverable:** `src/pymastery/attr_access.py` and `tests/test_attr_access.py`
- **STOP WHEN:** Expensive computation runs exactly once on first access and subsequent reads are O(1) attribute lookups.
- **Do Not Study Yet:** ORM lazy-loading proxies.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 42: Week 6 Gate: Descriptor-powered configuration & telemetry library CORE: Base Track (Months 1-4) [MINOR: Function signature preservation (functools.wraps + ParamSpec)]¶

> **The Big Picture:** Understand how functions, closures, decorators, and the descriptor protocol power Python's attribute lookup, methods, properties, and validation libraries.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine typed decorators, `singledispatch`, and custom descriptors into a cohesive self-validating model config library.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Function signature preservation (`functools.wraps` + `ParamSpec`) — because decorators and descriptors are the backbone of PyTorch (`@torch.no_grad`), FastAPI, and pytest.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 7, 9, 23

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day042/`, and review the mapped book section for `Week 6 Gate: Descriptor-powered configuration & telemetry library`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build a declarative `EngineConfig` base class using descriptors and a `@logged_operation` decorator, verified with unit tests.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 6 Gate: Descriptor-powered configuration & telemetry library` and save the test log beside

`gates/week06-gate.md` and `tests/test_week06_gate.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Function signature preservation (`functools.wraps` + `ParamSpec`): Verify decorated functions preserve `__name__`, `__doc__`, `__wrapped__`, and full `mypy --strict` parameter types.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 6 Gate: Descriptor-powered configuration & telemetry library`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day042/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write a decorator that strips type annotations or loses `functools.wraps(fn)` metadata.

> **Daily Gate (STOP WHEN):** All descriptor validations, decorator metadata checks, and strict type checks pass.

- **Tangible Deliverable:** `gates/week06-gate.md` and `tests/test_week06_gate.py`
- **STOP WHEN:** All descriptor validations, decorator metadata checks, and strict type checks pass.
- **Do Not Study Yet:** AST rewriting.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 7 - Collections, Algorithmic Data Structures & Complexity in Python¶

Day 43: `collections.deque`, `defaultdict`, `Counter` & `ChainMap` internals CORE: Base Track (Months 1-4) [MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master doubly-linked block queues (`deque(maxlen=N)`), multi-maps (`defaultdict`), multiset token counting (`Counter`), and layered config scopes (`ChainMap`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day043/`, and review the mapped book section for `collections.deque, defaultdict, Counter & ChainMap internals`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a sliding-window token-rate tracker (`deque`) and a layered CLI -> Env -> Default configuration resolver (`ChainMap`) in `src/pymastery/collections_ops.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `collections.deque, defaultdict, Counter & ChainMap internals and save the test log beside src/pymastery/collections_ops.py and tests/test_collections_ops.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `collections.deque, defaultdict, Counter & ChainMap internals, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day043/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** `deque.popleft()` is proven >50x faster than `list.pop(0)` at N=50,000 in benchmark test.

- **Tangible Deliverable:** `src/pymastery/collections_ops.py` and `tests/test_collections_ops.py`
- **STOP WHEN:** `deque.popleft()` is proven >50x faster than `list.pop(0)` at N=50,000 in benchmark test.
- **Do Not Study Yet:** Lock-free C++ ring buffers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 44: Priority queues (`heapq`) & custom tie-breaking schedulers

CORE: Base Track (Months 1-4) `[MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]¶`

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build O(log N) priority queues using `heapq` with monotonic sequence tie-breakers and lazy deletion (tombstoning) for dynamic priority updates.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day044/`, and review the mapped book section for `Priority queues (heapq) & custom tie-breaking schedulers`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `FairRequestScheduler` in `src/pymastery/priority_scheduler.py` supporting `push`, `pop_highest_priority`, and `cancel(request_id)` in O(log N) / O(1) amortized time.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Priority queues (heapq) & custom tie-breaking schedulers` and save the test log beside `src/pymastery/priority_scheduler.py` and `tests/test_priority_scheduler.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Priority queues (heapq) & custom tie-breaking schedulers`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day044/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** Equal-priority requests dequeue in strict FIFO order and cancelled requests are never returned.

- **Tangible Deliverable:** `src/pymastery/priority_scheduler.py` and `tests/test_priority_scheduler.py`
- **STOP WHEN:** Equal-priority requests dequeue in strict FIFO order and cancelled requests are never returned.
- **Do Not Study Yet:** Linux CFS red-black tree kernel code.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 45: Binary search (`bisect_left`, `bisect_right`, `insort`) & histogram bucketing CORE: Base Track (Months 1-4) `[MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]`¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use binary search (`bisect`) for O(log B) Prometheus-style latency histogram bucket assignment and sorted timestamp range queries.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day045/`, and review the mapped book section for `Binary search (bisect_left, bisect_right, insort) & histogram bucketing`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `LatencyHistogram` in `src/pymastery/histogram.py` with configurable upper bounds (`le`), `observe(value_ms)`, and quantile estimation.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Binary search (bisect_left, bisect_right, insort) & histogram bucketing` and save the test log beside `src/pymastery/histogram.py` and `tests/test_histogram.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Binary search (bisect_left, bisect_right, insort) & histogram bucketing`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day045/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** Bucket counts match Prometheus cumulative histogram semantics across 10,000 observations.

- **Tangible Deliverable:** `src/pymastery/histogram.py` and `tests/test_histogram.py`
- **STOP WHEN:** Bucket counts match Prometheus cumulative histogram semantics across 10,000 observations.
- **Do Not Study Yet:** HDRHistogram C bindings.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 46: Implementing an O(1) LRU Cache from scratch (Doubly Linked List + Dict) CORE: Base Track (Months 1-4) [MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a production-grade O(1) `get`/`put` LRU Cache with capacity eviction and hit/miss telemetry without using `functools.lru_cache`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day046/`, and review the mapped book section for `Implementing an O(1) LRU Cache from scratch (Doubly Linked List + Dict)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `LRUCache[K, V]` in `src/pymastery/lru_cache.py` using a sentinel-bounded doubly linked list of slotted nodes and a hash map.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing an O(1) LRU Cache from scratch (Doubly Linked List + Dict)` and save the test log beside `src/pymastery/lru_cache.py` and `tests/test_lru_cache.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing an O(1) LRU Cache from scratch (Doubly Linked List + Dict)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day046/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** All `get`, `put`, update-existing, and capacity-eviction operations execute in O(1) time and pass invariant checks on list links.

- **Tangible Deliverable:** `src/pymastery/lru_cache.py` and `tests/test_lru_cache.py`
- **STOP WHEN:** All `get`, `put`, update-existing, and capacity-eviction operations execute in O(1) time and pass invariant checks on list links.
- **Do Not Study Yet:** Distributed cache invalidation.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 47: Implementing a Prefix Radix/Trie Tree for KV-Cache Prefix Matching CORE: Base Track (Months 1-4) [MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how modern LLM engines (vLLM Automatic Prefix Caching / SGLang RadixAttention) match shared token prefixes using prefix trees.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day047/`, and review the mapped book section for `Implementing a Prefix Radix/Trie Tree for KV-Cache Prefix Matching`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `TokenPrefixTrie` in `src/pymastery/prefix_trie.py` that inserts token ID sequences, finds the longest cached prefix match, and tracks reference counts.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing a Prefix Radix/Trie Tree for KV-Cache Prefix Matching` and save the test log beside `src/pymastery/prefix_trie.py` and `tests/test_prefix_trie.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing a Prefix Radix/Trie Tree for KV-Cache Prefix Matching`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day047/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** Longest-prefix lookup accurately returns matched block IDs and unmatched suffix tokens.

- **Tangible Deliverable:** `src/pymastery/prefix_trie.py` and `tests/test_prefix_trie.py`
- **STOP WHEN:** Longest-prefix lookup accurately returns matched block IDs and unmatched suffix tokens.
- **Do Not Study Yet:** GPU page table kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 48: Graph & DAG algorithms in Python: topological sort (`graphlib`) & cycle detection CORE: Base Track (Months 1-4) `[MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]`¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Model computation graphs and task execution dependencies as Directed Acyclic Graphs (DAGs) with cycle detection and parallel ready-set extraction.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day048/`, and review the mapped book section for `Graph & DAG algorithms in Python: topological sort (graphlib) & cycle detection`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `TaskDAG[T]` in `src/pymastery/dag.py` with Kahn's algorithm from scratch and compare with `graphlib.TopologicalSorter`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Graph & DAG algorithms in Python: topological sort (graphlib) & cycle detection` and save the test log beside `src/pymastery/dag.py` and `tests/test_dag.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Graph & DAG algorithms in Python: topological sort (graphlib) & cycle detection`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day048/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** Cyclic dependencies raise `CycleError` with the exact cycle path, and independent nodes are grouped into parallelizable stages.

- **Tangible Deliverable:** `src/pymastery/dag.py` and `tests/test_dag.py`
- **STOP WHEN:** Cyclic dependencies raise `CycleError` with the exact cycle path, and independent nodes are grouped into parallelizable stages.
- **Do Not Study Yet:** Compiler SSA register allocation.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 49: Week 7 Gate: Algorithmic data structures benchmark & stress test

CORE: Base Track (Months 1-4) `[MINOR: Time complexity (Big-O) and benchmark verification (timeit / perf_counter_ns)]`¶

> **The Big Picture:** Select and implement the exact data structure (`deque`, `heapq`, `bisect`, `Counter`, LRU cache, Prefix Trie) required for low-latency scheduling and caching.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Integrate `FairRequestScheduler`, `LRUCache`, `TokenPrefixTrie`, and `LatencyHistogram` into a simulated KV-cache request router.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`) — because an O(N) `list.pop(0)` in a request queue destroys tail latency under high concurrency.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 2, 3 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day049/`, and review the mapped book section for `Week 7 Gate: Algorithmic data structures benchmark & stress test`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a 50,000-request synthetic trace through the router and record prefix cache hit rate, queue wait time, and throughput.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 7 Gate: Algorithmic data structures benchmark & stress test` and save the test log beside `gates/week07-gate.md` and `tests/test_week07_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Time complexity (Big-O) and benchmark verification (`timeit` / `perf_counter_ns`): Measure 10,000-element operations with `time.perf_counter_ns()` to prove O(1) vs O(N) scaling empirically.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 7 Gate: Algorithmic data structures benchmark & stress test`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day049/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `list.pop(0)` or `list.insert(0, x)` for FIFO queues—always use `collections.deque`.

> **Daily Gate (STOP WHEN):** 50,000 requests route in < 0.5 seconds with zero invariant failures.

- **Tangible Deliverable:** `gates/week07-gate.md` and `tests/test_week07_gate.py`
- **STOP WHEN:** 50,000 requests route in < 0.5 seconds with zero invariant failures.
- **Do Not Study Yet:** Network sockets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 8 - Filesystems, Binary I/O, Serialization & Structured Telemetry¶

Day 50: Unicode, UTF-8 byte encoding/decoding & tokenizer byte-fallback fundamentals CORE: Base Track (Months 1-4) [MINOR: Zero-copy binary parsing (struct / memoryview) and structured log schema]¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master the exact distinction between Unicode code points (`str`) and byte sequences (`bytes`/`bytearray`), surrogate pairs, and streaming UTF-8 decoding across chunk boundaries.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day050/`, and review the mapped book section for `Unicode, UTF-8 byte encoding/decoding & tokenizer byte-fallback fundamentals`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `IncrementalUTF8Decoder` in `src/pymastery/utf8_stream.py` that safely decodes multi-byte UTF-8

characters (e.g., emojis/CJK) split across arbitrary network byte chunks.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Unicode, UTF-8 byte encoding/decoding & tokenizer byte-fallback fundamentals` and save the test log beside `src/pymastery/utf8_stream.py` and `tests/test_utf8_stream.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Unicode, UTF-8 byte encoding/decoding & tokenizer byte-fallback fundamentals`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day050/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Splitting a 4-byte UTF-8 sequence across 1-byte chunks never raises `UnicodeDecodeError` and yields the exact string.

- **Tangible Deliverable:** `src/pymastery/utf8_stream.py` and `tests/test_utf8_stream.py`
- **STOP WHEN:** Splitting a 4-byte UTF-8 sequence across 1-byte chunks never raises `UnicodeDecodeError` and yields the exact string.
- **Do Not Study Yet:** Full BPE tokenizer training.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 51: Modern `pathlib`, atomic file writes, temporary files & directory traversal CORE: Base Track (Months 1-4) `[MINOR: Zero-copy binary parsing (struct / memoryview) and structured log schema]`¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use `pathlib.Path`, `tempfile.NamedTemporaryFile`, and `os.replace` to perform crash-safe atomic file updates.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on

disk as binary and structured text formats.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day051/`, and review the mapped book section for `Modern pathlib, atomic file writes, temporary files & directory traversal`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AtomicJsonStore` in `src/pymastery/atomic_io.py` with write-to-temp, `fsync`, and `os.replace` semantics.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Modern pathlib, atomic file writes, temporary files & directory traversal` and save the test log beside `src/pymastery/atomic_io.py` and `tests/test_atomic_io.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Modern pathlib, atomic file writes, temporary files & directory traversal`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day051/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Simulated crash during write leaves the previous valid file intact and never exposes a half-written file.

- **Tangible Deliverable:** `src/pymastery/atomic_io.py` and `tests/test_atomic_io.py`
- **STOP WHEN:** Simulated crash during write leaves the previous valid file intact and never exposes a half-written file.
- **Do Not Study Yet:** Distributed consensus filesystems.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 52: Binary data packing (`struct`), endianness & parsing SafeTensors headers from scratch CORE: Base Track (Months 1-4) `[MINOR: Zero-copy binary parsing (struct / memoryview) and structured log schema]`¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use Python's `struct` module (`<Q`, `<f`, `<I`) to read and write binary headers with explicit little-endian byte order.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day052/`, and review the mapped book section for `Binary data packing (struct), endianness & parsing SafeTensors headers from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a pure-Python `.safetensors` header parser and writer in `src/pymastery/safetensors_io.py` (8-byte little-endian uint64 length + UTF-8 JSON header + aligned byte offsets).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Binary data packing (struct), endianness & parsing SafeTensors headers from scratch` and save the test log beside `src/pymastery/safetensors_io.py` and `tests/test_safetensors_io.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Binary data packing (struct), endianness & parsing SafeTensors headers from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day052/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Extracts tensor names, dtypes, shapes, and byte offsets from a synthetic `.safetensors` file without loading tensor weights into RAM.

- **Tangible Deliverable:** `src/pymastery/safetensors_io.py` and `tests/test_safetensors_io.py`
- **STOP WHEN:** Extracts tensor names, dtypes, shapes, and byte offsets from a synthetic `.safetensors` file without loading tensor weights into RAM.
- **Do Not Study Yet:** CUDA weight loading.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 53: Memory-mapped files (`mmap`) & zero-copy binary slicing (`memoryview`) CORE: Base Track (Months 1-4) `[MINOR: Zero-copy binary parsing (struct / memoryview) and structured log schema]`¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Map large binary files directly into virtual address space with `mmap` and slice tensor byte ranges in O(1) memory using `memoryview`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day053/`, and review the mapped book section for `Memory-mapped files (mmap) & zero-copy binary slicing (memoryview)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Extend `safetensors_io.py` with `MmapTensorReader` that maps a binary file and returns zero-copy `memoryview` slices for requested tensors.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Memory-mapped files (mmap) & zero-copy binary slicing (memoryview)` and save the test log beside `src/pymastery/safetensors_io.py` and `tests/test_mmap_reader.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Memory-mapped files (mmap) & zero-copy binary slicing (memoryview)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day053/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Reading a 1 KB tensor slice from a 50 MB file touches only the requested pages and copies zero background bytes.

- **Tangible Deliverable:** `src/pymastery/safetensors_io.py` and `tests/test_mmap_reader.py`
- **STOP WHEN:** Reading a 1 KB tensor slice from a 50 MB file touches only the requested pages and copies zero background bytes.
- **Do Not Study Yet:** `madvise` kernel tuning.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 54: Custom JSON serialization (`JSONEncoder`/`default`), JSONL streaming & security (`pickle` hazards) CORE: Base Track (Months 1-4) [MINOR: Zero-copy binary parsing (struct / memoryview) and structured log schema]¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Serialize dataclasses, `Enum`, `datetime`, `Path`, and `UUID` cleanly to JSON/JSONL and understand why `pickle` allows arbitrary code execution.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day054/`, and review the mapped book section for `Custom JSON serialization (JSONEncoder/default), JSONL streaming & security (pickle hazards)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `StructuredJSONCodec` in `src/pymastery/json_codec.py` with deterministic key sorting, custom type hooks, and streaming JSONL reader/writer.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Custom JSON serialization (`JSONEncoder/default`), JSONL streaming & security (`pickle `hazards)` and save the test log beside `src/pymastery/json_codec.py` and `tests/test_json_codec.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Custom JSON serialization (`JSONEncoder/default`), JSONL streaming & security (`pickle `hazards)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day054/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Round-trips complex nested dataclasses and enums cleanly and rejects NaN/Infinity when strict RFC-8259 mode is enabled.

- **Tangible Deliverable:** `src/pymastery/json_codec.py` and `tests/test_json_codec.py`
- **STOP WHEN:** Round-trips complex nested dataclasses and enums cleanly and rejects NaN/Infinity when strict RFC-8259 mode is enabled.
- **Do Not Study Yet:** Cython JSON parsers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 55: Structured logging (`logging.Handler`, `Formatter`, `Filter` & `ContextVar` correlation) CORE: Base Track (Months 1-4) `[MINOR: Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema]`¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Configure Python's `logging` subsystem to emit machine-parseable JSON lines enriched with `contextvars.ContextVar` request IDs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day055/`, and review the mapped book section for `Structured logging (`logging.Handler`,` Formatter`,` Filter `&` ContextVar `correlation)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `JsonLogFormatter` and `RequestIdFilter` in `src/pymastery/telemetry_log.py` that automatically injects `request_id` into every log record.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Structured logging (`logging.Handler`,` Formatter`,` Filter `&` ContextVar `correlation)` and save the test log beside `src/pymastery/telemetry_log.py` and `tests/test_telemetry_log.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Structured logging (`logging.Handler`,` Formatter`,` Filter `&` ContextVar `correlation)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day055/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** Logs emitted inside a `request_id_context('req-123')` block contain valid JSON with `request_id: 'req-123'`.

- **Tangible Deliverable:** `src/pymastery/telemetry_log.py` and `tests/test_telemetry_log.py`
- **STOP WHEN:** Logs emitted inside a `request_id_context('req-123')` block contain valid JSON with `request_id: 'req-123'`.
- **Do Not Study Yet:** OpenTelemetry collector deployment.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 56: Week 8 Gate (Month 2 Review): SafeTensors Inspector & Caching Engine CORE: Base Track (Months 1-4) [MINOR: Zero-copy binary parsing (**struct** / **memoryview**) and structured log schema]¶

> **The Big Picture:** Handle files, binary headers (like SafeTensors/GGUF), streaming JSONL, and structured JSON logging with production-level safety.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Deliver a complete Month 2 capstone combining custom dunder models, descriptors, LRU/Prefix caching, `mmap` SafeTensors inspection, and structured JSON logging.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema — because model weights, tokenizers, and audit logs live on disk as binary and structured text formats.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 4 & Python Cookbook, 3rd Ed. (Beazley & Jones, O'Reilly) Ch. 5, 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day056/`, and review the mapped book section for `Week 8 Gate (Month 2 Review): SafeTensors Inspector & Caching Engine`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build `safetensors-inspect` CLI that reads `.safetensors` files via `mmap`, validates tensor metadata, caches lookups in `LRUCache`, and logs structured telemetry.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 8 Gate (Month 2 Review): SafeTensors Inspector & Caching Engine` and save the test log beside `gates/month02-gate.md` and `src/pymastery/safetensors_cli.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy binary parsing (`struct` / `memoryview`) and structured log schema: Verify every file open specifies explicit `encoding='utf-8'` (for text) or `'rb'`/`'wb'` (for binary) and passes cross-platform path checks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 8 Gate (Month 2 Review): SafeTensors Inspector & Caching Engine`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day056/failure-notes.md`.

> **Engineering Trap to Avoid:** Never rely on platform-default text encodings (`open(path)` without `encoding='utf-8'`) or use `pickle` on untrusted data.

> **Daily Gate (STOP WHEN):** All Month 1 & Month 2 tests (56 days of material) pass in one clean run.

- **Tangible Deliverable:** `gates/month02-gate.md` and `src/pymastery/safetensors_cli.py`
- **STOP WHEN:** All Month 1 & Month 2 tests (56 days of material) pass in one clean run.
- **Do Not Study Yet:** Asyncio event loops.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 3: Deep Asyncio, Streaming Protocols, ASGI & Multiprocessing Systems (Weeks 9–12, Days 57–84)¶

Week 9 - Asynchronous Programming (`asyncio`) Foundations & Structured Concurrency¶

Day 57: Event loop mechanics, coroutines (`async def`, `await`) & cooperative scheduling CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how an event loop schedules coroutines at `await` suspension points and how to detect event-loop blocking stalls.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day057/`, and review the mapped book section for `Event loop mechanics, coroutines (async def, await) & cooperative scheduling`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement an event-loop lag monitor and concurrent coroutine trace in `src/pymastery/async_basics.py` that measures scheduling jitter.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Event loop mechanics, coroutines (async def, await) & cooperative scheduling` and save the test log beside `src/pymastery/async_basics.py` and `tests/test_async_basics.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Event loop mechanics, coroutines (async def, await) & cooperative scheduling`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day057/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** 100 concurrent simulated 10ms I/O coroutines complete in ~12ms total instead of 1,000ms.

- **Tangible Deliverable:** `src/pymastery/async_basics.py` and `tests/test_async_basics.py`
- **STOP WHEN:** 100 concurrent simulated 10ms I/O coroutines complete in ~12ms total instead of 1,000ms.
- **Do Not Study Yet:** Custom C epoll/kqueue wrappers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 58: `asyncio.Task`, `create_task`, `gather`, `wait`, `as_completed` & strong references CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Manage concurrent task execution and avoid the garbage-collection bug where unreferenced background tasks disappear mid-execution.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day058/`, and review the mapped book section for `` `asyncio.Task`, `create_task`, `gather`, `wait`, `as_completed` & strong references` ``.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `BackgroundTaskTracker` in `src/pymastery/task_tracker.py` using a strong-reference `set[asyncio.Task]` with `add_done_callback(tasks.discard)`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `asyncio.Task, create_task, gather, wait, as_completed & strong references and save the test log beside src/pymastery/task_tracker.py and tests/test_task_tracker.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `asyncio.Task, create_task, gather, wait, as_completed & strong references, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day058/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** All spawned tasks are tracked, exceptions are logged, and `await tracker.drain()` waits for clean completion.

- **Tangible Deliverable:** `src/pymastery/task_tracker.py` and `tests/test_task_tracker.py`
- **STOP WHEN:** All spawned tasks are tracked, exceptions are logged, and `await tracker.drain()` waits for clean completion.
- **Do Not Study Yet:** Celery task queues.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 59: Structured concurrency with `asyncio.TaskGroup` & `except*` error handling CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use Python 3.11+ `asyncio.TaskGroup` so child task failures automatically cancel sibling tasks and propagate via `ExceptionGroup`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day059/`, and review the mapped book section for `Structured concurrency with` asyncio.TaskGroup `&` except* `error handling`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a parallel multi-shard model prefetcher using `asyncio.TaskGroup` in `src/pymastery/taskgroups.py` where one failing shard cleanly aborts the remaining downloads.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Structured concurrency with` asyncio.TaskGroup `&` except* `error handling` and save the test log beside `src/pymastery/taskgroups.py` and `tests/test_taskgroups.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Structured concurrency with` asyncio.TaskGroup `&` except* `error handling`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day059/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** When shard 2 fails at 5ms, shards 1 and 3 are cancelled immediately and cleanup handlers run.

- **Tangible Deliverable:** `src/pymastery/taskgroups.py` and `tests/test_taskgroups.py`
- **STOP WHEN:** When shard 2 fails at 5ms, shards 1 and 3 are cancelled immediately and cleanup handlers run.
- **Do Not Study Yet:** Distributed consensus.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 60: Cancellation semantics (`CancelledError`), `try/finally` cleanup & `asyncio.shield` CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Handle task cancellation safely without swallowing `asyncio.CancelledError` and protect critical commit sections using `asyncio.shield`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day060/`, and review the mapped book section for `Cancellation semantics (CancelledError), try/finally cleanup & asyncio.shield``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a cancellable streaming inference session in `src/pymastery/cancellation.py` that frees KV-cache slots in `finally:` upon cancellation while shielding final audit log writes.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Cancellation semantics (CancelledError), try/finally cleanup & asyncio.shield` and save the test log beside` src/pymastery/cancellation.py `and` tests/test_cancellation.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Cancellation semantics (CancelledError), try/finally cleanup & asyncio.shield`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day060/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** Cancelling a request mid-stream always returns its KV-cache slot count to 100% available and records the cancellation reason.

- **Tangible Deliverable:** `src/pymastery/cancellation.py` and `tests/test_cancellation.py`

- **STOP WHEN:** Cancelling a request mid-stream always returns its KV-cache slot count to 100% available and records the cancellation reason.
- **Do Not Study Yet:** OS-level `SIGKILL` recovery.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 61: Timeouts (`asyncio.timeout`, `timeout_at`, `wait_for`) & deadline propagation CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Enforce both First-Byte (TTFT) and Total Request deadlines using `asyncio.timeout()` context managers and monotonic clock budgets.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day061/`, and review the mapped book section for `Timeouts (asyncio.timeout, timeout_at, wait_for) & deadline propagation`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `DeadlineBudget` and `with_ttft_and_total_timeout` in `src/pymastery/timeouts.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Timeouts (asyncio.timeout, timeout_at, wait_for) & deadline propagation` and save the test log beside `src/pymastery/timeouts.py` and `tests/test_timeouts.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Timeouts (asyncio.timeout, timeout_at, wait_for) & deadline propagation`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day061/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** Slow prefill triggers `TTFTTimeoutError` while slow decode triggers `TotalTimeoutError` with exact elapsed times.

- **Tangible Deliverable:** `src/pymastery/timeouts.py` and `tests/test_timeouts.py`
- **STOP WHEN:** Slow prefill triggers `TTFTTimeoutError` while slow decode triggers `TotalTimeoutError` with exact elapsed times.
- **Do Not Study Yet:** Distributed gRPC wire deadlines.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 62: Bridging blocking code safely: `asyncio.to_thread` & `loop.run_in_executor` CORE: Base Track (Months 1-4)

MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Offload blocking file I/O, tokenization, or CPU work from the event loop onto a worker thread pool without stalling concurrent async tasks.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day062/`, and review the mapped book section for `Bridging blocking code safely: asyncio.to_thread & loop.run_in_executor``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Demonstrate in `src/pymastery/to_thread_demo.py` how running a 50ms CPU/blocking task via `asyncio.to_thread` keeps concurrent heartbeat latencies under 2ms.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Bridging blocking code safely: asyncio.to_thread & loop.run_in_executor` and save the test log beside src/pymastery/to_thread_demo.py and tests/test_to_thread_demo.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Bridging blocking code safely: asyncio.to_thread & loop.run_in_executor`, `verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day062/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** Heartbeat p99 latency stays < 5ms while blocking tasks execute concurrently via `to_thread`.

- **Tangible Deliverable:** `src/pymastery/to_thread_demo.py` and `tests/test_to_thread_demo.py`
- **STOP WHEN:** Heartbeat p99 latency stays < 5ms while blocking tasks execute concurrently via `to_thread`.
- **Do Not Study Yet:** C-extension GIL release (Month 7).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 63: Week 9 Gate: Cancellable concurrent task supervisor

CORE: Base Track (Months 1-4) MINOR: Task lifecycle and cancellation cleanup verification ¶

> **The Big Picture:** Understand how single-threaded cooperative multitasking works in Python: event loops, coroutines, Tasks, `TaskGroup`, timeouts, and cancellation.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a resilient async task supervisor supporting `TaskGroup`, deadlines, cancellation cleanup, and event-loop lag monitoring.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Task lifecycle and cancellation cleanup verification — because every modern LLM serving frontend and HTTP proxy is built on `asyncio`.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 21 & Python Concurrency with asyncio (Fowler, Manning) Ch. 1–5

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day063/`, and review the mapped book section for `Week 9 Gate: Cancellable concurrent task supervisor`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Pass a 20-scenario async fault test (timeouts, simultaneous exceptions, client disconnects, shielded cleanup).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 9 Gate: Cancellable concurrent task supervisor` and save the test log beside `gates/week09-gate.md` and `tests/test_week09_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Task lifecycle and cancellation cleanup verification: Verify every async test leaves zero dangling tasks (`asyncio.all_tasks()`) and handles `asyncio.CancelledError` cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 9 Gate: Cancellable concurrent task supervisor`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day063/failure-notes.md`.

> **Engineering Trap to Avoid:** Never call blocking I/O (`time.sleep`, `requests.get`, heavy CPU loops) directly inside an `async def` coroutine—it freezes the entire event loop!

> **Daily Gate (STOP WHEN):** Zero leaked tasks, zero unawaited coroutine warnings, and 100% deterministic cleanup.

- **Tangible Deliverable:** `gates/week09-gate.md` and `tests/test_week09_gate.py`
- **STOP WHEN:** Zero leaked tasks, zero unawaited coroutine warnings, and 100% deterministic cleanup.
- **Do Not Study Yet:** Raw TCP sockets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 10 - Async Streams, Queues, Backpressure & Dynamic Batching¶

Day 64: Async context managers (`__aenter__`, `__aexit__`, `@asynccontextmanager`, `AsyncExitStack`) CORE: Base Track (Months 1-4) MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Manage async lifecycles (connection pools, background workers, GPU engine sessions) with deterministic startup and teardown.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day064/`, and review the mapped book section for `Async context managers (__aenter__, __aexit__, @asynccontextmanager, AsyncExitStack)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AsyncEngineSession` and `AsyncResourcePool` in `src/pymastery/async_contexts.py` using `AsyncExitStack`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Async context managers (__aenter__, __aexit__, @asynccontextmanager, AsyncExitStack)` and save the test log beside `src/pymastery/async_contexts.py` and `tests/test_async_contexts.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Async context managers (__aenter__, __aexit__, @asynccontextmanager, AsyncExitStack)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day064/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** Startup failures roll back already-opened async resources in reverse order.

- **Tangible Deliverable:** `src/pymastery/async_contexts.py` and `tests/test_async_contexts.py`
- **STOP WHEN:** Startup failures roll back already-opened async resources in reverse order.
- **Do Not Study Yet:** Kubernetes pod lifecycle hooks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 65: Async iterators & async generators (`async for`, `yield` in `async def`, `aclosing`) CORE: Base Track (Months 1-4)

MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Stream generated tokens one by one using async generators and guarantee deterministic cleanup when a caller breaks out of `async for` early.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day065/`, and review the mapped book section for `Async iterators & async generators (async for, yield in async def, aclosing)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `stream_tokens` in `src/pymastery/async_streams.py` and verify early consumer exit with `async with contextlib.aclosing(stream_tokens(...)):`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Async iterators & async generators (async for, yield in async def, aclosing)` and save the test log beside `src/pymastery/async_streams.py` and `tests/test_async_streams.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Async iterators & async generators (async for, yield in async def, aclosing)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day065/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** Breaking after 3 tokens immediately triggers the generator's `finally` cleanup block.

- **Tangible Deliverable:** `src/pymastery/async_streams.py` and `tests/test_async_streams.py`
- **STOP WHEN:** Breaking after 3 tokens immediately triggers the generator's `finally` cleanup block.
- **Do Not Study Yet:** WebSockets protocol framing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 66: `asyncio.Queue`, `PriorityQueue`, `LifoQueue` & producer-consumer worker pools CORE: Base Track (Months 1-4)

MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Coordinate multiple producers and bounded worker pools using `asyncio.Queue(maxsize=N)`, `task_done()`, and `queue.join()`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day066/`, and review the mapped book section for `asyncio.Queue, PriorityQueue, LifoQueue & producer-consumer worker pools`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a multi-worker async pipeline in `src/pymastery/async_queues.py` with graceful poison-pill / drain shutdown.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `asyncio.Queue, PriorityQueue, LifoQueue & producer-consumer worker pools and save the test log beside` src/pymastery/async_queues.py `and` tests/test_async_queues.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `asyncio.Queue, PriorityQueue, LifoQueue & producer-consumer worker pools, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day066/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** All queued items are processed before shutdown completes and bounded capacity blocks or rejects overflow cleanly.

- **Tangible Deliverable:** `src/pymastery/async_queues.py` and `tests/test_async_queues.py`
- **STOP WHEN:** All queued items are processed before shutdown completes and bounded capacity blocks or rejects overflow cleanly.
- **Do Not Study Yet:** RabbitMQ / Kafka brokers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 67: Async synchronization primitives: `Lock`, `Event`, `Condition`, `Semaphore`

CORE: Base Track (Months 1-4) MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Control concurrent access to shared state and limit in-flight concurrency using `asyncio.Semaphore` and `asyncio.Event`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day067/`, and review the mapped book section for `Async synchronization primitives: Lock, Event, Condition, Semaphore``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AdmissionController` in `src/pymastery/admission.py` that caps active concurrent requests at `max_concurrency` and waiting queue at `max_queue`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Async synchronization primitives: Lock, Event, Condition, Semaphore` and save the test log beside` src/pymastery/admission.py `and` tests/test_admission.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Async synchronization primitives: Lock, Event, Condition, Semaphore`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day067/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** Requests exceeding `max_concurrency + max_queue` fail fast in < 1ms with `OverloadedError`.

- **Tangible Deliverable:** `src/pymastery/admission.py` and `tests/test_admission.py`
- **STOP WHEN:** Requests exceeding `max_concurrency + max_queue` fail fast in < 1ms with `OverloadedError`.
- **Do Not Study Yet:** OS futexes.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 68: Dynamic micro-batching in `asyncio` (`Future` per request + batch loop) CORE: Base Track (Months 1-4) MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build the foundational pattern of ML inference serving: individual async callers await an `asyncio.Future` while a background loop collects requests over a time/size window and processes them as a batch.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference

between a server that scales and one that crashes under load.

- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day068/`, and review the mapped book section for `Dynamic micro-batching in` asyncio `(Future per request + batch loop)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AsyncDynamicBatcher[Req, Resp]` in `src/pymastery/dynamic_batcher.py` with `max_batch_size=8` and `max_batch_delay_ms=5.0`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Dynamic micro-batching in` asyncio `(Future per request + batch loop)` and save the test log beside `src/pymastery/dynamic_batcher.py` and `tests/test_dynamic_batcher.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Dynamic micro-batching in` asyncio `(Future per request + batch loop)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day068/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** 16 concurrent requests automatically coalesce into 2 batches of 8 and each caller receives its exact corresponding response or exception.

- **Tangible Deliverable:** `src/pymastery/dynamic_batcher.py` and `tests/test_dynamic_batcher.py`
- **STOP WHEN:** 16 concurrent requests automatically coalesce into 2 batches of 8 and each caller receives its exact corresponding response or exception.
- **Do Not Study Yet:** CUDA stream batching.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 69: Async token-bucket rate limiting & fair per-tenant queueing

CORE: Base Track (Months 1-4) MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prevent a single noisy client from starving other callers using an async token-bucket rate limiter and round-robin tenant queues.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day069/`, and review the mapped book section for `Async token-bucket rate limiting & fair per-tenant queueing`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AsyncTenantFairQueue` in `src/pymastery/fair_queue.py` scheduling across multiple tenant IDs.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Async token-bucket rate limiting & fair per-tenant queueing` and save the test log beside `src/pymastery/fair_queue.py` and `tests/test_fair_queue.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Async token-bucket rate limiting & fair per-tenant queueing`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day069/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** A tenant flooding 100 requests cannot delay a second tenant's single request beyond one batch cycle.

- **Tangible Deliverable:** `src/pymastery/fair_queue.py` and `tests/test_fair_queue.py`

- **STOP WHEN:** A tenant flooding 100 requests cannot delay a second tenant's single request beyond one batch cycle.
- **Do Not Study Yet:** Distributed Envoy rate limiting.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 70: Week 10 Gate: Streaming Dynamic Batching Engine

CORE: Base Track (Months 1-4) MINOR: Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) ¶

> **The Big Picture:** Build the core concurrency primitives of an inference server: async generators (`yield` inside `async def`), bounded queues, semaphores, and dynamic continuous batchers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `AdmissionController`, `AsyncDynamicBatcher`, and async token streaming so batched requests stream individual tokens back to concurrent callers via per-request `asyncio.Queue`s.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL) — because backpressure and dynamic batching are the difference between a server that scales and one that crashes under load.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 6–10 & Fluent Python, 2nd Ed. Ch. 21

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day070/`, and review the mapped book section for `Week 10 Gate: Streaming Dynamic Batching Engine`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build `StreamingBatchEngine` in `src/pymastery/streaming_batch_engine.py` and benchmark 50 concurrent streaming requests.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 10 Gate: Streaming Dynamic Batching Engine` and save the test log beside `gates/week10-gate.md` and `src/pymastery/streaming_batch_engine.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Queue depth, backpressure rejection, and token-stream latency (TTFT / ITL): Test every async generator with `contextlib.aclosing()` to ensure generator `finally:` blocks run when consumers break early.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 10 Gate: Streaming Dynamic Batching Engine`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day070/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use unbounded `asyncio.Queue()` in a network service—always set `maxsize` to enforce backpressure before RAM exhausts.

> **Daily Gate (STOP WHEN):** 50 concurrent streaming callers receive all tokens in order, cancelled callers drop out of the active batch cleanly, and all tests pass.

- **Tangible Deliverable:** `gates/week10-gate.md` and `src/pymastery/streaming_batch_engine.py`
- **STOP WHEN:** 50 concurrent streaming callers receive all tokens in order, cancelled callers drop out of the active batch cleanly, and all tests pass.
- **Do Not Study Yet:** HTTP protocol parsing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 11 - Network Programming, Raw Async TCP, HTTP/1.1 & SSE Streaming¶

Day 71: TCP socket fundamentals & `asyncio.start_server` (`StreamReader`, `StreamWriter`) CORE: Base Track (Months 1-4) `[MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]`¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build an async TCP server and client using `asyncio.start_server` and `asyncio.open_connection`, always calling `await writer.drain()` and `await writer.wait_closed()`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day071/`, and review the mapped book section for `TCP socket fundamentals & asyncio.start_server (StreamReader, StreamWriter)`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a line-delimited JSON-RPC TCP server and client in `src/pymastery/tcp_rpc.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `TCP socket fundamentals &` asyncio.start_server `(`StreamReader`,` StreamWriter`)` and save the test log beside `src/pymastery/tcp_rpc.py` and `tests/test_tcp_rpc.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `TCP socket fundamentals &` asyncio.start_server `(`StreamReader`,` StreamWriter`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day071/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** Handles 20 concurrent client connections on an ephemeral localhost port and shuts down with zero resource warnings.

- **Tangible Deliverable:** `src/pymastery/tcp_rpc.py` and `tests/test_tcp_rpc.py`
- **STOP WHEN:** Handles 20 concurrent client connections on an ephemeral localhost port and shuts down with zero resource warnings.
- **Do Not Study Yet:** Kernel TCP congestion algorithms.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 72: Length-prefixed binary framing over TCP (`struct.pack('>I', len)` + `readexactly`) CORE: Base Track (Months 1-4) `[MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]`¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate TCP packet fragmentation bugs by implementing 4-byte big-endian length-prefixed binary message framing using `reader.readexactly(n)`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day072/`, and review the mapped book section for `Length-prefixed binary framing over TCP (`struct.pack('>I', len) `+` readexactly`)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `FramedStreamChannel` in `src/pymastery/framed_channel.py` that sends and receives arbitrary-size binary/JSON frames safely even when split into 7-byte TCP chunks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Length-prefixed binary framing over TCP (`struct.pack('>I', len) `+` readexactly`)` and save the test log beside `src/pymastery/framed_channel.py` and `tests/test_framed_channel.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Length-prefixed binary framing over TCP (`struct.pack('>I', len) `+` readexactly`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day072/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** Fragmented byte-dripping tests and truncated-stream `IncompleteReadError` tests pass 100%.

- **Tangible Deliverable:** `src/pymastery/framed_channel.py` and `tests/test_framed_channel.py`
- **STOP WHEN:** Fragmented byte-dripping tests and truncated-stream `IncompleteReadError` tests pass 100%.
- **Do Not Study Yet:** TLS handshake cryptography.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 73: HTTP/1.1 wire format & Server-Sent Events (`text/event-stream`) parser from scratch CORE: Base Track (Months 1-4) `[MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]`¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand HTTP request/response headers, status codes, `Transfer-Encoding: chunked`, and the OpenAI-style `data: {...}\n\n` and `data: [DONE]\n\n` SSE streaming protocol.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day073/`, and review the mapped book section for `HTTP/1.1 wire format & Server-Sent Events (`text/event-stream`) parser from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a zero-dependency `SSEParser` and `SSEEncoder` in `src/pymastery/sse_protocol.py` that parses SSE events split across arbitrary byte chunks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `HTTP/1.1 wire format & Server-Sent Events (`text/event-stream`) parser from scratch` and save the test log beside `src/pymastery/sse_protocol.py` and `tests/test_sse_protocol.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `HTTP/1.1 wire format & Server-Sent Events (`text/event-stream`) parser from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day073/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** Correctly parses multi-event chunks, partial lines across chunks, keep-alive comments (`: ping`), and `[DONE]` markers.

- **Tangible Deliverable:** `src/pymastery/sse_protocol.py` and `tests/test_sse_protocol.py`
- **STOP WHEN:** Correctly parses multi-event chunks, partial lines across chunks, keep-alive comments (`: ping`), and `[DONE]` markers.
- **Do Not Study Yet:** HTTP/3 QUIC datagrams.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 74: Building an Async Streaming HTTP Client with connection pooling, retries & TTFT/ITL timing CORE: Base Track (Months 1-4) [MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write a production-grade async streaming client that tracks `ttft_ms`, per-token `itl_ms`, total duration, and handles exponential backoff with jitter on `429`/`503`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day074/`, and review the mapped book section for `Building an Async Streaming HTTP Client with connection pooling, retries & TTFT/ITL timing`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `StreamingInferenceClient` in `src/pymastery/http_client.py` with pluggable transport and full telemetry capture.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Building an Async Streaming HTTP Client with connection pooling, retries & TTFT/ITL timing` and save the test log beside `src/pymastery/http_client.py` and `tests/test_http_client.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Building an Async Streaming HTTP Client with connection pooling, retries & TTFT/ITL timing`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day074/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** Records accurate TTFT, ITL array, token count, and status code across normal streams, 503 retries, and mid-stream timeouts.

- **Tangible Deliverable:** `src/pymastery/http_client.py` and `tests/test_http_client.py`
- **STOP WHEN:** Records accurate TTFT, ITL array, token count, and status code across normal streams, 503 retries, and mid-stream timeouts.
- **Do Not Study Yet:** OAuth2 browser flows.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 75: The ASGI Specification (`scope`, `receive`, `send`) from scratch without frameworks CORE: Base Track (Months 1-4) `[MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]`¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how Uvicorn, Starlette, and FastAPI work under the hood by writing a raw ASGI 3.0 application and lifespan handler in pure Python.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day075/`, and review the mapped book section for `The ASGI Specification (scope, receive, send) from scratch without frameworks`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a raw ASGI inference app in `src/pymastery/raw_asgi.py` supporting `lifespan.startup`/`shutdown`, `/healthz`, `/readyz`, and streaming `/v1/generate` SSE responses.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The ASGI Specification (scope, receive, send) from scratch without frameworks` and save the test log beside `src/pymastery/raw_asgi.py` and `tests/test_raw_asgi.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The ASGI Specification (scope, receive, send) from scratch without frameworks`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day075/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** In-memory ASGI harness verifies lifespan events, HTTP status headers, and multi-chunk `http.response.body` streaming.

- **Tangible Deliverable:** `src/pymastery/raw_asgi.py` and `tests/test_raw_asgi.py`
- **STOP WHEN:** In-memory ASGI harness verifies lifespan events, HTTP status headers, and multi-chunk `http.response.body` streaming.
- **Do Not Study Yet:** WSGI legacy sync servers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 76: Graceful server shutdown: draining in-flight streams, `http.disconnect` & readiness flipping CORE: Base Track (Months 1-4) `[MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]`¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Detect client disconnects mid-stream (`http.disconnect`) to abort wasted token generation and drain active requests cleanly before shutdown.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.

- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day076/`, and review the mapped book section for `Graceful server shutdown: draining in-flight streams,` http.disconnect `& readiness flipping`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Add disconnect detection and graceful drain state (`is_draining`) to `src/pymastery/raw_asgi.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Graceful server shutdown: draining in-flight streams,` http.disconnect `& readiness flipping` and save the test log beside `src/pymastery/raw_asgi.py` and `tests/test_graceful_drain.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Graceful server shutdown: draining in-flight streams,` http.disconnect `& readiness flipping`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day076/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** When `begin_drain()` is called, `/readyz` returns `503`, new requests are rejected, and in-flight streams finish cleanly before shutdown completes.

- **Tangible Deliverable:** `src/pymastery/raw_asgi.py` and `tests/test_graceful_drain.py`
- **STOP WHEN:** When `begin_drain()` is called, `/readyz` returns `503`, new requests are rejected, and in-flight streams finish cleanly before shutdown completes.
- **Do Not Study Yet:** BGP load balancing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 77: Week 11 Gate: End-to-End Streaming Server + Client Benchmark Harness CORE: Base Track (Months 1-4) [MINOR: Wire-level byte framing (StreamReader / StreamWriter) and connection draining]¶

> **The Big Picture:** Demystify web frameworks by building raw async TCP streams, length-prefixed framing, HTTP/1.1 chunked/SSE streaming, and ASGI apps from the ground up.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Connect the Week 10 `StreamingBatchEngine` to the Week 11 ASGI/SSE server and `StreamingInferenceClient`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining — because understanding what happens on the wire lets you debug truncated streams, half-closed sockets, and proxy buffering.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler, Manning) Ch. 3, 8 & Foundations of Python Network Programming, 3rd Ed. (Rhodes & Goerzen, Apress) Ch. 1–4, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day077/`, and review the mapped book section for `Week 11 Gate: End-to-End Streaming Server + Client Benchmark Harness`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a 100-request concurrent streaming benchmark over loopback sockets and output p50/p95/p99 TTFT and ITL metrics.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 11 Gate: End-to-End Streaming Server + Client Benchmark Harness` and save the test log beside `gates/week11-gate.md` and `tests/test_week11_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire-level byte framing (`StreamReader` / `StreamWriter`) and connection draining: Verify every socket/server test binds to `127.0.0.1` port `0` (OS-assigned ephemeral port) and closes all transports cleanly.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 11 Gate: End-to-End Streaming Server + Client Benchmark Harness`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day077/failure-notes.md`.

> **Engineering Trap to Avoid:** Never assume one `sock.recv()` or `reader.read()` returns one complete message—TCP is a byte stream, not a message stream!

> **Daily Gate (STOP WHEN):** 100 streaming requests complete with 0% error rate, accurate SSE framing, and clean server drain.

- **Tangible Deliverable:** `gates/week11-gate.md` and `tests/test_week11_gate.py`

- **STOP WHEN:** 100 streaming requests complete with 0% error rate, accurate SSE framing, and clean server drain.
- **Do Not Study Yet:** Multiprocessing IPC.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 12 - Multithreading, Multiprocessing, Shared Memory & OS Signals¶

Day 78: OS processes, `subprocess.run`/`Popen`, pipes, exit codes & `SIGTERM`/`SIGINT` signals CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Launch, monitor, stream `stdout`/`stderr`, and terminate external processes safely using `subprocess` and `signal` handlers.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day078/`, and review the mapped book section for `OS processes, subprocess.run/Popen, pipes, exit codes & SIGTERM/SIGINT signals`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ManagedSubprocess` in `src/pymastery/process_supervisor.py` that streams logs, enforces startup readiness timeouts, and escalates `SIGTERM` -> `SIGKILL` on shutdown.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `OS processes, subprocess.run/Popen, pipes, exit codes & SIGTERM/SIGINT signals` and save the test log beside `src/pymastery/process_supervisor.py` and `tests/test_process_supervisor.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `OS processes, subprocess.run/Popen, pipes, exit codes & SIGTERM/SIGINT signals`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day078/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** Hung child process is cleanly terminated within the grace period and exit code is captured.

- **Tangible Deliverable:** `src/pymastery/process_supervisor.py` and `tests/test_process_supervisor.py`
- **STOP WHEN:** Hung child process is cleanly terminated within the grace period and exit code is captured.
- **Do Not Study Yet:** Linux cgroups v2 kernel APIs.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 79: `threading` fundamentals, the GIL, race conditions, `Lock`, `RLock`, `Condition` & `Queue` CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand why the GIL protects CPython bytecode pointers but NOT application-level invariants, and build thread-safe components with `threading.Lock` and `queue.Queue`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day079/`, and review the mapped book section for `threading` fundamentals, the GIL, race conditions, `Lock`, `RLock`, `Condition` & `Queue`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Demonstrate a lost-update race condition and build a thread-safe `ThreadSafeTokenCache` and bounded worker pool in

`src/pymastery/threading_ops.py`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `threading` fundamentals, the GIL, race conditions, `Lock`, `RLock`, `Condition` & `Queue and save the test log beside` src/pymastery/threading_ops.py `and` tests/test_threading_ops.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `threading` fundamentals, the GIL, race conditions, `Lock`, `RLock`, `Condition` & `Queue, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day079/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** 10 concurrent threads performing 5,000 cache mutations preserve exact invariant counts with zero deadlocks.

- **Tangible Deliverable:** `src/pymastery/threading_ops.py` and `tests/test_threading_ops.py`
- **STOP WHEN:** 10 concurrent threads performing 5,000 cache mutations preserve exact invariant counts with zero deadlocks.
- **Do Not Study Yet:** Lock-free C11 atomics.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 80: `concurrent.futures`: `ThreadPoolExecutor` vs `ProcessPoolExecutor` & `as_completed` CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Compare thread pools (for I/O or GIL-releasing tasks) against process pools (for pure-Python CPU-bound tasks) using the unified `Executor` interface.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run

across multiple threads and OS processes.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day080/`, and review the mapped book section for `` `concurrent.futures`: `ThreadPoolExecutor` vs `ProcessPoolExecutor` & `as_completed` ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Benchmark a CPU-heavy batch tokenization/hashing workload across sequential, `ThreadPoolExecutor`, and `ProcessPoolExecutor` in `src/pymastery/executors_bench.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `concurrent.futures`: `ThreadPoolExecutor` vs `ProcessPoolExecutor` & `as_completed` and save the test log beside `` src/pymastery/executors_bench.py and tests/test_executors_bench.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `concurrent.futures`: `ThreadPoolExecutor` vs `ProcessPoolExecutor` & `as_completed`, verify that `` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day080/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** Results are identical across all three executors and exception propagation from worker processes is verified.

- **Tangible Deliverable:** `src/pymastery/executors_bench.py` and `tests/test_executors_bench.py`
- **STOP WHEN:** Results are identical across all three executors and exception propagation from worker processes is verified.
- **Do Not Study Yet:** Distributed Ray clusters.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 81: `multiprocessing` with `spawn` context, `Pipe`, `Queue` & pickle serialization overhead CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build multi-process worker architectures using `mp.get_context('spawn')` and measure IPC serialization overhead over pipes and queues.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day081/`, and review the mapped book section for `` `multiprocessing` with `spawn` context, `Pipe`, `Queue` & pickle serialization overhead`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a parent-child worker engine in `src/pymastery/mp_worker.py` with heartbeat checks and clean crash detection (`proc.exitcode`).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `multiprocessing` with `spawn` context, `Pipe`, `Queue` & pickle serialization overhead `and save the test log beside` src/pymastery/mp_worker.py `and` tests/test_mp_worker.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `multiprocessing` with `spawn` context, `Pipe`, `Queue` & pickle serialization overhead`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day081/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** Parent detects simulated worker crash (`sys.exit(1)`) within 100ms and marks pending requests as failed instead of hanging forever.

- **Tangible Deliverable:** `src/pymastery/mp_worker.py` and `tests/test_mp_worker.py`
- **STOP WHEN:** Parent detects simulated worker crash (`sys.exit(1)`) within 100ms and marks pending requests as failed instead of hanging forever.
- **Do Not Study Yet:** MPI collectives.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 82: Zero-copy inter-process communication with `multiprocessing.shared_memory` CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate pickle copying of large byte/tensor buffers between processes using `multiprocessing.shared_memory.SharedMemory` and `memoryview`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day082/`, and review the mapped book section for `Zero-copy inter-process communication with multiprocessing.shared_memory``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `SharedRingBuffer` in `src/pymastery/shm_ipc.py` that transfers megabyte-sized float32 payloads between two processes with zero serialization copies.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Zero-copy inter-process communication with multiprocessing.shared_memory` and save the test log beside src/pymastery/shm_ipc.py and tests/test_shm_ipc.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Zero-copy inter-process communication with multiprocessing.shared_memory`, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day082/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** `SharedMemory` segment is read accurately by child process and `unlink()` is guaranteed via `try/finally`.

- **Tangible Deliverable:** `src/pymastery/shm_ipc.py` and `tests/test_shm_ipc.py`
- **STOP WHEN:** `SharedMemory` segment is read accurately by child process and `unlink()` is guaranteed via `try/finally`.
- **Do Not Study Yet:** CUDA IPC memory handles.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 83: Combining `asyncio` event loops in the frontend with `ProcessPool` / `SharedMemory` workers CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master the real-world architecture of vLLM and SGLang: an `asyncio` HTTP/API frontend communicating asynchronously with separate model-execution worker processes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day083/`, and review the mapped book section for `Combining asyncio event loops in the frontend with ProcessPool / SharedMemory workers`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AsyncMultiprocessEngineBridge` in `src/pymastery/engine_bridge.py` linking async caller `Futures` to a child worker process.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Combining` asyncio `event loops in the frontend with` ProcessPool `/` SharedMemory `workers` and save the test log beside `src/pymastery/engine_bridge.py` and `tests/test_engine_bridge.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Combining` asyncio `event loops in the frontend with` ProcessPool `/` SharedMemory `workers`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day083/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** Async callers await results from the child process without blocking the event loop, and shutdown cleans up both loop and process.

- **Tangible Deliverable:** `src/pymastery/engine_bridge.py` and `tests/test_engine_bridge.py`
- **STOP WHEN:** Async callers await results from the child process without blocking the event loop, and shutdown cleans up both loop and process.
- **Do Not Study Yet:** Multi-node NCCL.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 84: Week 12 Gate (Month 3 Review): Multi-Process Async Streaming Service CORE: Base Track (Months 1-4)

MINOR: Race-condition prevention, GIL awareness, and clean child-process termination ¶

> **The Big Picture:** Understand when to use threads (`threading`), processes (`multiprocessing`), and POSIX shared memory (`shared_memory`) to bypass the GIL and coordinate multi-worker systems.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Integrate Month 3's `asyncio` structured concurrency, dynamic batcher, SSE protocol, and multi-process worker bridge.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Race-condition prevention, GIL awareness, and clean child-process termination — because tokenizers, data loaders, and multi-GPU workers run across multiple threads and OS processes.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 19, 20 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald) Ch. 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day084/`, and review the mapped book section for `Week 12 Gate (Month 3 Review): Multi-Process Async Streaming Service`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a comprehensive stress test including worker crash recovery, client disconnect during stream, and backpressure overload.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 12 Gate (Month 3 Review): Multi-Process Async Streaming Service` and save the test log beside `gates/month03-gate.md` and `tests/test_month03_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Race-condition prevention, GIL awareness, and clean child-process termination: Ensure every spawned thread or child process is joined with an explicit timeout (`join(timeout=5.0)`) and leaves zero zombie processes.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 12 Gate (Month 3 Review): Multi-Process Async Streaming Service`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day084/failure-notes.md`.

> **Engineering Trap to Avoid:** Never use `multiprocessing` default `fork` start method with multithreaded or CUDA code—always use `mp.get_context('spawn')` to prevent deadlocks!

> **Daily Gate (STOP WHEN):** All Month 1–3 tests pass with zero zombie processes or leaked tasks.

- **Tangible Deliverable:** `gates/month03-gate.md` and `tests/test_month03_gate.py`
- **STOP WHEN:** All Month 1–3 tests pass with zero zombie processes or leaked tasks.
- **Do Not Study Yet:** Metaprogramming.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 4: Pythonic Architecture, SQLite Storage & Base Mastery Capstone (Weeks 13–16, Days 85–112)¶

Week 13 - Software Design Patterns, SOLID in Python & Property-Based Testing¶

Day 85: Pythonic Strategy, Command & Registry patterns using Callables and Protocols CORE: Base Track (Months 1-4) [MINOR: Cyclomatic simplicity and

property-based invariant verification (hypothesis)]¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement pluggable sampling strategies (Greedy, Temperature, Top-K, Top-P) and a decorator-based plugin registry without inheritance bloat.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.
- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day085/`, and review the mapped book section for `Pythonic Strategy, Command & Registry patterns using Callables and Protocols`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `SamplerRegistry` and composable logit processor pipelines in `src/pymastery/sampling_patterns.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Pythonic Strategy, Command & Registry patterns using Callables and Protocols` and save the test log beside `src/pymastery/sampling_patterns.py` and `tests/test_sampling_patterns.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Pythonic Strategy, Command & Registry patterns using Callables and Protocols`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day085/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** Logit processors compose cleanly in sequence and custom samplers register via `@register_sampler('top_k')`.

- **Tangible Deliverable:** `src/pymastery/sampling_patterns.py` and `tests/test_sampling_patterns.py`
- **STOP WHEN:** Logit processors compose cleanly in sequence and custom samplers register via `@register_sampler('top_k')`.
- **Do Not Study Yet:** CUDA sampling kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 86: State Machine & Observer/Event-Bus patterns for Engine Lifecycle & Telemetry CORE: Base Track (Months 1-4) [MINOR: Cyclomatic simplicity and property-based invariant verification (hypothesis)]¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Decouple engine execution from metrics, logging, and tracing using a synchronous/asynchronous typed Event Bus and explicit state transition table.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.
- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day086/`, and review the mapped book section for `State Machine & Observer/Event-Bus patterns for Engine Lifecycle & Telemetry`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `TypedEventBus` and `EngineStateMachine` in `src/pymastery/event_bus.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `State Machine & Observer/Event-Bus patterns for Engine Lifecycle & Telemetry` and save the test log beside `src/pymastery/event_bus.py` and `tests/test_event_bus.py`.

- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `State Machine & Observer/Event-Bus patterns for Engine Lifecycle & Telemetry`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day086/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** Subscribers receive strongly-typed events (`BatchStarted`, `TokenGenerated`, `RequestFailed`) and faulty subscribers are isolated.

- **Tangible Deliverable:** `src/pymastery/event_bus.py` and `tests/test_event_bus.py`
- **STOP WHEN:** Subscribers receive strongly-typed events (`BatchStarted`, `TokenGenerated`, `RequestFailed`) and faulty subscribers are isolated.
- **Do Not Study Yet:** Distributed Kafka buses.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 87: Builder, Factory & Configuration validation patterns without external frameworks CORE: Base Track (Months 1-4) `[MINOR: Cyclomatic simplicity and property-based invariant verification (hypothesis)]`¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Construct complex, multi-component systems (`EngineBuilder`) with staged validation, immutable output configs, and clear error accumulation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.

- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day087/`, and review the mapped book section for `Builder, Factory & Configuration validation patterns without external frameworks`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `InferenceServerBuilder` in `src/pymastery/builder.py` that validates all cross-field constraints (e.g., `max_batch_tokens >= max_seq_len`) before instantiation.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Builder, Factory & Configuration validation patterns without external frameworks` and save the test log beside `src/pymastery/builder.py` and `tests/test_builder.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Builder, Factory & Configuration validation patterns without external frameworks`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day087/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** All invalid cross-field combinations are reported together in a single `ConfigurationError`.

- **Tangible Deliverable:** `src/pymastery/builder.py` and `tests/test_builder.py`
- **STOP WHEN:** All invalid cross-field combinations are reported together in a single `ConfigurationError`.
- **Do Not Study Yet:** XML bean configs.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 88: Property-based testing concepts & invariant fuzzing (randomized state-machine testing) CORE: Base Track (Months 1-4) [MINOR: Cyclomatic simplicity and property-based invariant verification (hypothesis)]¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Test code against hundreds of generated edge cases (NaNs, empty lists, Unicode extremes, arbitrary operation sequences) rather than 3 hand-picked examples.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.
- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day088/`, and review the mapped book section for `Property-based testing concepts & invariant fuzzing (randomized state-machine testing)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a deterministic seedable fuzz/property test runner in `src/pymastery/prop_testing.py` (and optional `hypothesis` strategies) to fuzz `LRUCache`, `RingBuffer`, and `SSEParser`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Property-based testing concepts & invariant fuzzing (randomized state-machine testing)` and save the test log beside `src/pymastery/prop_testing.py` and `tests/test_properties.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Property-based testing concepts & invariant fuzzing (randomized state-machine testing)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day088/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** 500 randomized operation sequences against `LRUCache` match a simple reference oracle 100% of the time.

- **Tangible Deliverable:** `src/pymastery/prop_testing.py` and `tests/test_properties.py`
- **STOP WHEN:** 500 randomized operation sequences against `LRUCache` match a simple reference oracle 100% of the time.
- **Do Not Study Yet:** Coverage-guided binary fuzzing (AFL).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 89: Refactoring legacy/messy Python into clean, typed, testable modules CORE: Base Track (Months 1-4) [MINOR: Cyclomatic simplicity and property-based invariant verification (hypothesis)]¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Take a monolithic 200-line untyped script with global variables, bare `except:`, and mixed I/O, and refactor it systematically under a golden-master characterization test.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.
- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day089/`, and review the mapped book section for `Refactoring legacy/messy Python into clean, typed, testable modules`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/refactored_bench.py` replacing a messy benchmark script with pure functions, frozen dataclasses, and injected I/O ports.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Refactoring legacy/messy Python into clean, typed, testable modules` and save the test log beside `src/pymastery/refactored_bench.py` and `tests/test_refactored_bench.py`.

- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Refactoring legacy/messy Python into clean, typed, testable modules`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day089/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** Characterization tests prove identical output while reducing function complexity and achieving 100% `mypy --strict` coverage.

- **Tangible Deliverable:** `src/pymastery/refactored_bench.py` and `tests/test_refactored_bench.py`
- **STOP WHEN:** Characterization tests prove identical output while reducing function complexity and achieving 100% `mypy --strict` coverage.
- **Do Not Study Yet:** Automated AST codemods.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 90: Code documentation, docstrings, doctests & API surface curation (`__all__`) CORE: Base Track (Months 1-4) `[MINOR: Cyclomatic simplicity and property-based invariant verification (hypothesis)]`¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Control public package exports using `__all__`, write runnable doctests (`pytest --doctest-modules`), and document preconditions, complexity, and failure modes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.

- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day090/`, and review the mapped book section for `Code documentation, docstrings, doctests & API surface curation (__all__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Audit and add `__all__` and tested docstrings to core `pymastery` modules.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Code documentation, docstrings, doctests & API surface curation (__all__)` and save the test log beside Updated `src/pymastery/__init__.py` and `tests/test_public_api.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Code documentation, docstrings, doctests & API surface curation (__all__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day090/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** `pytest --doctest-modules` passes and `test_public_api.py` verifies every exported symbol in `__all__` is typed.

- **Tangible Deliverable:** Updated `src/pymastery/__init__.py` and `tests/test_public_api.py`
- **STOP WHEN:** `pytest --doctest-modules` passes and `test_public_api.py` verifies every exported symbol in `__all__` is typed.
- **Do Not Study Yet:** Sphinx theme customization.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 91: Week 13 Gate: Pattern & Invariant Audit CORE: Base Track (Months 1-4) [MINOR: Cyclomatic simplicity and property-based invariant verification (**hypothesis**)]¶

> **The Big Picture:** Write maintainable, extensible systems using Pythonic design patterns (replacing bloated Java-style boilerplate with first-class functions, Protocols, and composition) and Hypothesis property testing.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Demonstrate clean architectural composition and randomized state-machine fuzzing across your data structures and streaming parser.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Cyclomatic simplicity and property-based invariant verification (`hypothesis`) — because complex inference schedulers and routers rot quickly without clean boundaries and invariant tests.
- **Mapped Book & Docs:** Architecture Patterns with Python (Percival & Gregory, O'Reilly) Ch. 4–7 & Fluent Python, 2nd Ed. Ch. 10

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day091/`, and review the mapped book section for `Week 13 Gate: Pattern & Invariant Audit`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run 1,000 seeded randomized operation traces against your scheduler and cache with zero invariant violations.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 13 Gate: Pattern & Invariant Audit` and save the test log beside `gates/week13-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Cyclomatic simplicity and property-based invariant verification (`hypothesis`): Every module must pass `ruff` complexity checks and include invariant tests covering edge inputs.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 13 Gate: Pattern & Invariant Audit`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day091/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy Gang-of-Four Java class hierarchies (`AbstractStrategyFactoryBean`) into Python when a callable, `Protocol`, or dictionary dispatch table solves it in 10 lines.

> **Daily Gate (STOP WHEN):** All property and unit tests pass deterministically from a recorded seed.

- **Tangible Deliverable:** `gates/week13-gate.md`
- **STOP WHEN:** All property and unit tests pass deterministically from a recorded seed.

- **Do Not Study Yet:** C extensions.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 14 - SQLite Persistence, Caching Layers, Configuration & CLI Polish¶

Day 92: Embedded `sqlite3` in Python: WAL mode, transactions, parameterized queries & `Row` factories CORE: Base Track (Months 1-4) MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a fast, thread-safe local persistence layer using `sqlite3` with `PRAGMA journal_mode=WAL`, `PRAGMA foreign_keys=ON`, and typed row mapping.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day092/`, and review the mapped book section for `Embedded` sqlite3 `in Python: WAL mode, transactions, parameterized queries &` Row `factories`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `BenchmarkRepository` in `src/pymastery/sqlite_repo.py` storing benchmark runs, hardware manifests, and per-request latency records.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Embedded` sqlite3 `in Python: WAL mode, transactions, parameterized queries &` Row `factories` and save the test log beside `src/pymastery/sqlite_repo.py` and `tests/test_sqlite_repo.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Embedded` sqlite3 `in Python: WAL mode, transactions, parameterized queries &` Row `factories`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day092/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Failed mid-batch insertions roll back completely and queries return typed `BenchmarkRun` dataclasses.

- **Tangible Deliverable:** `src/pymastery/sqlite_repo.py` and `tests/test_sqlite_repo.py`
- **STOP WHEN:** Failed mid-batch insertions roll back completely and queries return typed `BenchmarkRun` dataclasses.
- **Do Not Study Yet:** Distributed PostgreSQL sharding.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 93: Disk-backed content-addressable caching (`hashlib.sha256` + SQLite/Filesystem) CORE: Base Track (Months 1-4)

MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a deterministic prompt/response and tokenization artifact cache keyed by canonical SHA-256 hashes of `(model_id, params, prompt)`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day093/`, and review the mapped book section for `Disk-backed content-addressable caching (`hashlib.sha256 `+ SQLite/Filesystem)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ContentAddressableCache` in `src/pymastery/disk_cache.py` with TTL expiration and max-bytes eviction.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Disk-backed content-addressable caching (`hashlib.sha256 `+ SQLite/Filesystem)` and save the test log beside `src/pymastery/disk_cache.py` and `tests/test_disk_cache.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Disk-backed content-addressable caching (`hashlib.sha256 `+ SQLite/Filesystem)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day093/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Identical requests hit cache in < 0.5ms, expired entries are purged, and disk quota is strictly enforced.

- **Tangible Deliverable:** `src/pymastery/disk_cache.py` and `tests/test_disk_cache.py`
- **STOP WHEN:** Identical requests hit cache in < 0.5ms, expired entries are purged, and disk quota is strictly enforced.
- **Do Not Study Yet:** CDN edge caching.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 94: TOML configuration (`tomllib`), environment overrides & secret redaction CORE: Base Track (Months 1-4) MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Parse `TOML` configuration files using Python 3.11+ built-in `tomllib`, merge with `ENV` variables, and prevent API keys/secrets from leaking in `__repr__` or logs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.

- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day094/`, and review the mapped book section for `TOML configuration (tomllib), environment overrides & secret redaction`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `SecretStr` and `load_app_config` in `src/pymastery/config_loader.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `TOML configuration (tomllib), environment overrides & secret redaction` and save the test log beside `src/pymastery/config_loader.py` and `tests/test_config_loader.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `TOML configuration (tomllib), environment overrides & secret redaction`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day094/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** `str(secret)` and `repr(config)` always print `'SecretStr(***)'` and never expose the raw secret in logs or tracebacks.

- **Tangible Deliverable:** `src/pymastery/config_loader.py` and `tests/test_config_loader.py`
- **STOP WHEN:** `str(secret)` and `repr(config)` always print `'SecretStr(***)'` and never expose the raw secret in logs or tracebacks.
- **Do Not Study Yet:** HashiCorp Vault clusters.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 95: Statistical analysis in pure Python: `statistics`, exact percentiles, bootstrap & outlier detection CORE: Base Track (Months 1-4)

MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Compute accurate p50, p90, p95, p99 latencies, throughput (tokens/sec), standard deviation, and warm-up exclusion without heavy third-party dependencies.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day095/`, and review the mapped book section for `Statistical analysis in pure Python: statistics, exact percentiles, bootstrap & outlier detection`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `BenchmarkStatsEngine` in `src/pymastery/stats_engine.py` that separates warm-up runs from measured runs and computes full latency distributions.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Statistical analysis in pure Python: statistics, exact percentiles, bootstrap & outlier detection` and save the test log beside `src/pymastery/stats_engine.py` and `tests/test_stats_engine.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Statistical analysis in pure Python: statistics, exact percentiles, bootstrap & outlier detection`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day095/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Matches reference quantile calculations and flags high-variance runs automatically.

- **Tangible Deliverable:** `src/pymastery/stats_engine.py` and `tests/test_stats_engine.py`
- **STOP WHEN:** Matches reference quantile calculations and flags high-variance runs automatically.
- **Do Not Study Yet:** Bayesian MCMC frameworks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 96: Terminal table rendering, progress bars & Markdown/CSV report generation from scratch CORE: Base Track (Months 1-4)

MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Format clean aligned ASCII/Markdown tables and live terminal status lines (`\r` ANSI control) without external UI libraries.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day096/`, and review the mapped book section for `Terminal table rendering, progress bars & Markdown/CSV report generation from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ReportRenderer` in `src/pymastery/reporting.py` outputting Markdown tables, CSV files, and terminal summaries.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Terminal table rendering, progress bars & Markdown/CSV report generation from scratch` and save the test log beside `src/pymastery/reporting.py` and `tests/test_reporting.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Terminal table rendering, progress bars & Markdown/CSV report generation from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day096/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Generates valid Markdown tables and RFC-4180 CSV files from `BenchmarkStatsEngine` outputs.

- **Tangible Deliverable:** `src/pymastery/reporting.py` and `tests/test_reporting.py`
- **STOP WHEN:** Generates valid Markdown tables and RFC-4180 CSV files from `BenchmarkStatsEngine` outputs.
- **Do Not Study Yet:** React frontend dashboards.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 97: Packaging for distribution: wheel building (`python -m build`), entrypoints & clean install test CORE: Base Track (Months 1-4)

MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a standard `.whl` (Wheel) distribution, inspect its contents (`zipfile`), and verify installation in an isolated environment.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day097/`, and review the mapped book section for `Packaging for distribution: wheel`

`building (`python -m build`), entrypoints & clean install test`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build the wheel for `pymastery` and write a verification test in `tests/test_packaging_build.py` that inspects wheel metadata.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Packaging for distribution: wheel building (`python -m build`), entrypoints & clean install test` and save the test log beside `tests/test_packaging_build.py` and `dist/` wheel artifact.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Packaging for distribution: wheel building (`python -m build`), entrypoints & clean install test`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day097/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Wheel contains all `src/pymastery` modules, `py.typed` marker (PEP 561), and working CLI entrypoints.

- **Tangible Deliverable:** `tests/test_packaging_build.py` and `dist/` wheel artifact
- **STOP WHEN:** Wheel contains all `src/pymastery` modules, `py.typed` marker (PEP 561), and working CLI entrypoints.
- **Do Not Study Yet:** C-extension manylinux wheels (Month 7).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 98: Week 14 Gate: Reproducible Benchmark & Storage Suite

CORE: Base Track (Months 1-4) MINOR: Transactional SQLite ACID safety and SQL-injection prevention ¶

> **The Big Picture:** Use Python's built-in `sqlite3` transactional database, configuration layering (`tomllib`), and rich terminal formatting for durable local state and benchmark repositories.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `StreamingInferenceClient`, `BenchmarkStatsEngine`, `BenchmarkRepository` (SQLite), and `ReportRenderer` into a complete CLI tool.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Transactional SQLite ACID safety and SQL-injection prevention — because every benchmark harness, prompt evaluation suite, and local cache benefits from embedded SQLite storage.
- **Mapped Book & Docs:** Python 3.12 Standard Library Docs (`sqlite3`, `tomllib`) & Robust Python (Viafore, O'Reilly) Ch. 15

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day098/`, and review the mapped book section for `Week 14 Gate: Reproducible Benchmark & Storage Suite`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Execute 5 benchmark repetitions against the local async server, persist to SQLite, and generate a Markdown comparison report.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 14 Gate: Reproducible Benchmark & Storage Suite` and save the test log beside `gates/week14-gate.md` and `artifacts/week14-benchmark-report.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Transactional SQLite ACID safety and SQL-injection prevention: Always use parameterized SQL queries (`?` placeholders) and context-managed transactions (`with conn:`).
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 14 Gate: Reproducible Benchmark & Storage Suite`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day098/failure-notes.md`.

> **Engineering Trap to Avoid:** Never format SQL strings using f-strings (`f'SELECT * FROM runs WHERE id = {x}'`)—always use parameterized `?` bindings!

> **Daily Gate (STOP WHEN):** Report is generated reproducibly from SQLite records and all tests pass.

- **Tangible Deliverable:** `gates/week14-gate.md` and `artifacts/week14-benchmark-report.md`
- **STOP WHEN:** Report is generated reproducibly from SQLite records and all tests pass.
- **Do Not Study Yet:** AST metaprogramming.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 15 - Month 4 Base Mastery Capstone — Part 1: Core Engine & Protocol Implementation¶

Day 99: Capstone Architecture, Frozen Domain Models & Strict Request Validation CORE: Base Track (Months 1-4) [MINOR: Strict typing (mypy --strict), test coverage, and zero external web-framework dependencies]¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Design and implement the complete domain layer for `pyserve-core`: `GenerateRequest`, `TokenChunk`, `BatchState`, `ServerConfig`, and `Result`-based validation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day099/`, and review the mapped book section for `Capstone Architecture, Frozen Domain Models & Strict Request Validation`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/capstone_base/models.py` and `validation.py` with frozen slotted dataclasses, descriptors, and exhaustive error types.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Architecture, Frozen Domain Models & Strict Request Validation` and save the test log beside `src/pymastery/capstone_base/models.py`, `validation.py`, and `tests/test_cap_models.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Architecture, Frozen Domain Models & Strict Request Validation`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day099/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone —implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** 100% branch coverage on request validation and zero `mypy --strict` errors.

- **Tangible Deliverable:** `src/pymastery/capstone_base/models.py`, `validation.py`, and `tests/test_cap_models.py`
- **STOP WHEN:** 100% branch coverage on request validation and zero `mypy --strict` errors.
- **Do Not Study Yet:** GPU kernel execution.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 100: Capstone Prefix Cache (`TokenPrefixTrie` + `LRUCache`) & Memory Budget Tracker CORE: Base Track (Months 1-4) `[MINOR: Strict typing (mypy --strict), test coverage, and zero external web-framework dependencies]`¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the token prefix cache and simulated KV-block capacity manager that tracks active and cached tokens against a fixed memory budget.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day100/`, and review the mapped book section for `Capstone Prefix Cache (TokenPrefixTrie + LRUCache) & Memory Budget Tracker`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/capstone_base/cache_manager.py` with O(1) LRU eviction of unreferenced prefix blocks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Prefix Cache (TokenPrefixTrie + LRUCache) & Memory Budget Tracker` and save the test log beside `src/pymastery/capstone_base/cache_manager.py` and `tests/test_cap_cache.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Prefix Cache (TokenPrefixTrie + LRUCache) & Memory Budget Tracker`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day100/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone—implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** Active blocks are never evicted, unreferenced blocks evict in strict LRU order, and capacity invariants hold under fuzzing.

- **Tangible Deliverable:** `src/pymastery/capstone_base/cache_manager.py` and `tests/test_cap_cache.py`
- **STOP WHEN:** Active blocks are never evicted, unreferenced blocks evict in strict LRU order, and capacity invariants hold under fuzzing.
- **Do Not Study Yet:** CUDA PagedAttention kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 101: Capstone Continuous/Dynamic Batching Scheduler & Multi-Process Worker Pool CORE: Base Track (Months 1-4) [MINOR: Strict typing (mypy --strict), test coverage, and zero external web-framework dependencies]¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the async scheduler that pulls requests from a fair priority queue, forms batches within token budgets, and dispatches steps to a `spawn` worker process.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day101/`, and review the mapped book section for `Capstone Continuous/Dynamic Batching Scheduler & Multi-Process Worker Pool`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/capstone_base/scheduler.py` and `worker.py` with cancellation, timeout, and heartbeat

monitoring.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Continuous/Dynamic Batching Scheduler & Multi-Process Worker Pool` and save the test log beside `src/pymastery/capstone_base/scheduler.py`, `worker.py`, and `tests/test_cap_scheduler.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Continuous/Dynamic Batching Scheduler & Multi-Process Worker Pool`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day101/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone—implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** Handles concurrent prefill and decode steps, cancels aborted requests immediately, and recovers from simulated worker crashes.

- **Tangible Deliverable:** `src/pymastery/capstone_base/scheduler.py`, `worker.py`, and `tests/test_cap_scheduler.py`
- **STOP WHEN:** Handles concurrent prefill and decode steps, cancels aborted requests immediately, and recovers from simulated worker crashes.
- **Do Not Study Yet:** Multi-node tensor parallelism.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 102: Capstone Streaming Server (ASGI / Async TCP SSE) with Admission Control & Graceful Drain CORE: Base Track (Months 1-4) [MINOR: Strict typing (**mypy --strict**), test coverage, and zero external web-framework dependencies]¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Expose `/v1/generate` (SSE streaming), `/healthz`, `/readyz`, and `/metrics` over async I/O with bounded admission and graceful shutdown draining.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day102/`, and review the mapped book section for `Capstone Streaming Server (ASGI / Async TCP SSE) with Admission Control & Graceful Drain`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/capstone_base/server.py` and `tests/test_cap_server.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Streaming Server (ASGI / Async TCP SSE) with Admission Control & Graceful Drain` and save the test log beside `src/pymastery/capstone_base/server.py` and `tests/test_cap_server.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Streaming Server (ASGI / Async TCP SSE) with Admission Control & Graceful Drain`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day102/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone—implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** Streams SSE chunks accurately, rejects overload with HTTP 429/503, and drains in-flight streams on shutdown.

- **Tangible Deliverable:** `src/pymastery/capstone_base/server.py` and `tests/test_cap_server.py`
- **STOP WHEN:** Streams SSE chunks accurately, rejects overload with HTTP 429/503, and drains in-flight streams on shutdown.
- **Do Not Study Yet:** TLS termination proxies.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 103: Capstone Async Load Generator, SQLite Results Store & Statistical Report CLI CORE: Base Track (Months 1-4) [MINOR: Strict typing (mypy --

strict), test coverage, and zero external web-framework dependencies]¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build the CLI benchmark runner that drives configurable concurrency against `pyserve-core`, records TTFT/ITL/p50/p95/p99 to SQLite, and prints Markdown reports.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day103/`, and review the mapped book section for `Capstone Async Load Generator, SQLite Results Store & Statistical Report CLI`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/capstone_base/bench_cli.py` and `tests/test_cap_bench.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Async Load Generator, SQLite Results Store & Statistical Report CLI` and save the test log beside `src/pymastery/capstone_base/bench_cli.py` and `tests/test_cap_bench.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Async Load Generator, SQLite Results Store & Statistical Report CLI`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day103/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone —implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** End-to-end benchmark run executes 200 streaming requests across concurrency 1, 4, 8, and 16 and outputs a complete report.

- **Tangible Deliverable:** `src/pymastery/capstone_base/bench_cli.py` and `tests/test_cap_bench.py`
- **STOP WHEN:** End-to-end benchmark run executes 200 streaming requests across concurrency 1, 4, 8, and 16 and outputs a complete report.
- **Do Not Study Yet:** Grafana cloud hosting.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 104: Capstone Chaos & Fault-Injection Test Suite CORE: Base Track (Months 1-4)

`[MINOR: Strict typing (mypy --strict), test coverage, and zero external web-framework dependencies]`¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prove the resilience of your hand-built Python server under slow clients, mid-stream disconnects, malformed payloads, timeout storms, and worker restarts.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day104/`, and review the mapped book section for `Capstone Chaos & Fault-Injection Test Suite`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `tests/test_cap_chaos.py` running 10 deterministic fault scenarios.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Chaos & Fault-Injection Test Suite` and save the test log beside `tests/test_cap_chaos.py` and `artifacts/week15-chaos-report.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Chaos & Fault-Injection Test Suite`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day104/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone—implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** All 10 chaos scenarios pass with zero leaked tasks, zero leaked memory blocks, and accurate error telemetry.

- **Tangible Deliverable:** `tests/test_cap_chaos.py` and `artifacts/week15-chaos-report.md`
- **STOP WHEN:** All 10 chaos scenarios pass with zero leaked tasks, zero leaked memory blocks, and accurate error telemetry.
- **Do Not Study Yet:** Production chaos monkeys.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 105: Week 15 Gate: Capstone Feature Freeze & Code Quality Audit

CORE: Base Track (Months 1-4) `[MINOR: Strict typing (mypy --strict), test coverage, and zero external web-framework dependencies]`¶

> **The Big Picture:** Synthesize every Base Concept from Months 1–4 by building `pyserve-core`: a 100% self-implemented, strictly typed, async streaming inference server & benchmark suite.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Freeze features on `pyserve-core` and audit every module for code clarity, typing, docstrings, and test speed.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies — because building a complete serving & benchmarking stack by hand proves you truly own core Python.
- **Mapped Book & Docs:** All Month 1–4 Core Book Chapters (Ramalho, Viafore, Fowler)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day105/`, and review the mapped book section for `Week 15 Gate: Capstone Feature Freeze & Code Quality Audit`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run `mypy --strict`, `ruff check`, `pytest --cov`, and `tracemalloc` leak checks across the entire repository.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 15 Gate: Capstone Feature Freeze & Code Quality Audit` and save the test log beside `gates/week15-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Strict typing (`mypy --strict`), test coverage, and zero external web-framework dependencies: Every capstone file must pass `mypy --strict`, `ruff check`, and unit tests before moving to the next day.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 15 Gate: Capstone Feature Freeze & Code Quality Audit`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day105/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not import FastAPI, Pydantic, or aiohttp for this Month 4 capstone—implement the models, validation, ASGI/TCP streaming, and dynamic batcher yourself using the Python standard library!

> **Daily Gate (STOP WHEN):** Test suite passes in < 5 seconds with >92% coverage and zero static warnings.

- **Tangible Deliverable:** `gates/week15-gate.md`
- **STOP WHEN:** Test suite passes in < 5 seconds with >92% coverage and zero static warnings.
- **Do Not Study Yet:** Adding unrequested features.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 16 - Month 4 Base Mastery Capstone — Part 2: Verification, Defense & Base Gate¶

Day 106: End-to-End Memory Leak & Complexity Verification (`tracemalloc` + Big-O benchmarks) CORE: Base Track (Months 1-4)

MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Verify that running 5,000 requests through `pyserve-core` exhibits flat steady-state memory usage and O(1) cache/queue operations.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day106/`, and review the mapped book section for `End-to-End Memory Leak & Complexity Verification (`tracemalloc `+ Big-O benchmarks)`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Write and execute `scripts/verify_memory_and_scaling.py` and save the memory growth trace.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `End-to-End Memory Leak & Complexity Verification (`tracemalloc `+ Big-O benchmarks)` and save the test log beside `scripts/verify_memory_and_scaling.py` and `artifacts/day106-memory-trace.md`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `End-to-End Memory Leak & Complexity Verification (`tracemalloc `+ Big-O benchmarks)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day106/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** Memory delta between request 1,000 and request 5,000 is < 100 KB after `gc.collect()`.

- **Tangible Deliverable:** `scripts/verify_memory_and_scaling.py` and `artifacts/day106-memory-trace.md`
- **STOP WHEN:** Memory delta between request 1,000 and request 5,000 is < 100 KB after `gc.collect()`.
- **Do Not Study Yet:** C-level Valgrind.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 107: Clean-Room Wheel Build, Installation & CLI Smoke Test

CORE: Base Track (Months 1-4) MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prove that your package installs cleanly into a brand-new environment from a built `.whl` file and runs all CLI subcommands.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day107/`, and review the mapped book section for `Clean-Room Wheel Build, Installation & CLI Smoke Test`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a clean build script that creates a temporary venv, installs the `.whl`, starts the server, runs a 50-request benchmark, and verifies the SQLite output.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Clean-Room Wheel Build, Installation & CLI Smoke Test` and save the test log beside `artifacts/day107-clean-room.log`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Clean-Room Wheel Build, Installation & CLI Smoke Test`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day107/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** Zero errors from wheel build through benchmark report generation.

- **Tangible Deliverable:** `artifacts/day107-clean-room.log`

- **STOP WHEN:** Zero errors from wheel build through benchmark report generation.
- **Do Not Study Yet:** Docker containerization.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 108: Architectural Documentation & Sequence Diagrams

CORE: Base Track (Months 1-4) MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Document the request lifecycle, class/protocol boundaries, concurrency model, and backpressure invariants of `pyserve-core`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day108/`, and review the mapped book section for `Architectural Documentation & Sequence Diagrams`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write `docs/BASE_CAPSTONE_ARCHITECTURE.md` with Mermaid sequence and component diagrams.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Architectural Documentation & Sequence Diagrams` and save the test log beside `docs/BASE_CAPSTONE_ARCHITECTURE.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Architectural Documentation & Sequence Diagrams`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day108/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** Another engineer can trace a request from socket read to dynamic batcher to worker and back in under 5 minutes.

- **Tangible Deliverable:** `docs/BASE_CAPSTONE_ARCHITECTURE.md`
- **STOP WHEN:** Another engineer can trace a request from socket read to dynamic batcher to worker and back in under 5 minutes.
- **Do Not Study Yet:** Marketing slides.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 109: Closed-Book Base Python Implementation Drill (Timed 60-Minute Coding Challenge) CORE: Base Track (Months 1-4)

MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prove fluency by implementing a typed, slotted, thread-safe/async-safe bounded priority batch queue with timeout cancellation from a blank file in 60 minutes without AI or notes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day109/`, and review the mapped book section for `Closed-Book Base Python Implementation Drill (Timed 60-Minute Coding Challenge)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Complete `src/pymastery/drills/day109_drill.py` and `tests/test_day109_drill.py` under a strict 60-minute timer.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Closed-Book Base Python Implementation Drill (Timed 60-Minute Coding Challenge)` and save the test log beside `src/pymastery/drills/day109_drill.py` and `tests/test_day109_drill.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Closed-Book Base Python Implementation Drill (Timed 60-Minute Coding Challenge)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day109/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** Passes `mypy --strict` and 15 edge-case tests within the 60-minute budget.

- **Tangible Deliverable:** `src/pymastery/drills/day109_drill.py` and `tests/test_day109_drill.py`
- **STOP WHEN:** Passes `mypy --strict` and 15 edge-case tests within the 60-minute budget.
- **Do Not Study Yet:** Looking up syntax.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 110: Closed-Book Base Python Written Defense (25 Core Engineering Questions) CORE: Base Track (Months 1-4)

MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Answer 25 deep conceptual questions covering mutability, descriptors, iterator protocol, `asyncio` cancellation, `TaskGroup`, `spawn` vs `fork`, and `shared_memory`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day110/`, and review the mapped book section for `Closed-Book Base Python Written Defense (25 Core Engineering Questions)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write answers in `gates/month04-written-defense.md` without notes, then grade against the reference rubric.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Closed-Book Base Python Written Defense (25 Core Engineering Questions)` and save the test log beside `gates/month04-written-defense.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Closed-Book Base Python Written Defense (25 Core Engineering Questions)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day110/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** Score >= 90% with any weak spots patched by a concrete code test.

- **Tangible Deliverable:** `gates/month04-written-defense.md`
- **STOP WHEN:** Score >= 90% with any weak spots patched by a concrete code test.
- **Do Not Study Yet:** Trivia memorization.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 111: Refactoring & Bug-Hunt Challenge (Diagnosing 5 Seeded Production Bugs) CORE: Base Track (Months 1-4)

MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Diagnose and fix 5 classic Python systems bugs: late-binding closure, swallowed `CancelledError`, mutable default arg, missing `writer.drain()`, and

unlinked `SharedMemory`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day111/`, and review the mapped book section for `Refactoring & Bug-Hunt Challenge (Diagnosing 5 Seeded Production Bugs)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Fix all 5 bugs in `src/pymastery/drills/bug_hunt.py` and write regression tests proving each fix.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Refactoring & Bug-Hunt Challenge (Diagnosing 5 Seeded Production Bugs)` and save the test log beside `src/pymastery/drills/bug_hunt.py` and `tests/test_bug_hunt.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Refactoring & Bug-Hunt Challenge (Diagnosing 5 Seeded Production Bugs)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day111/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** All 5 regression tests fail before the fix and pass deterministically after the fix.

- **Tangible Deliverable:** `src/pymastery/drills/bug_hunt.py` and `tests/test_bug_hunt.py`
- **STOP WHEN:** All 5 regression tests fail before the fix and pass deterministically after the fix.
- **Do Not Study Yet:** New features.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 112: Day 112 — Month 4 Final Gate: Base Concepts Mastery Certification CORE: Base Track (Months 1-4)

MINOR: Clean-room reproducibility and oral/written defense of Python internals ¶

> **The Big Picture:** Complete the 4-Month Base Concepts Mastery Gate: verify clean-room reproducibility, benchmark performance, complexity guarantees, and closed-book recall.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Final verification of Months 1–4 (Days 1–112, 112 hours): packaging, strict typing, data model, descriptors, algorithmic collections, binary/mmap I/O, `asyncio`, multiprocessing, and `pyserve-core`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room reproducibility and oral/written defense of Python internals — because completing Month 4 means you can write production-grade core Python from a blank file without guessing.
- **Mapped Book & Docs:** Review of Fluent Python, 2nd Ed., Robust Python, and Python Concurrency with asyncio

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day112/`, and review the mapped book section for `Day 112 — Month 4 Final Gate: Base Concepts Mastery Certification`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Execute the full Month 4 gate checklist, tag git release `v1.0.0-base-mastery`, and prepare workspace for Month 5 Advanced Concepts.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Day 112 — Month 4 Final Gate: Base Concepts Mastery Certification` and save the test log beside `gates/month04-final-base-gate.md` and git tag `v1.0.0-base-mastery`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room reproducibility and oral/written defense of Python internals: Run all checks from a freshly cloned directory and fresh virtual environment.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Day 112 — Month 4 Final Gate: Base Concepts Mastery Certification`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day112/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not advance to Month 5 (Advanced Metaprogramming, CPython Internals & C/GPU Extensions) until every Month 4 gate item passes.

> **Daily Gate (STOP WHEN):** All 112 Base Days complete, test suite 100% green, `mypy --strict` 100% clean. Ready for Months 5–8 Advanced Python!

- **Tangible Deliverable:** `gates/month04-final-base-gate.md` and git tag `v1.0.0-base-mastery`
- **STOP WHEN:** All 112 Base Days complete, test suite 100% green, `mypy --strict` 100% clean. Ready for Months 5–8 Advanced Python!
- **Do Not Study Yet:** Nothing—you are now ready to start Month 5!
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

---

4. Phase 2 — Advanced Concepts Mastery (Months 5–8: Days 113–224)¶

Phase 2 takes you from intermediate systems Python to **Senior/Staff AI Inference Python**: metaprogramming, AST/bytecode compilers, CPython memory layout, zero-copy buffer protocol, strided N-dimensional tensors, autograd, KV-cached Transformer execution, PagedAttention scheduling, profiling, native C/C++/GPU interop, and the full `Mini-vLLM` capstone.

Month 5: Metaprogramming, AST/Bytecode & Building FastAPI/Pydantic From Scratch (Weeks 17–20, Days 113–140)¶

Week 17 - Class Construction, `__init_subclass__`, Metaclasses & Dynamic Types¶

Day 113: `type(name, bases, namespace)` & how class statements execute at runtime CORE: Advanced Track (Months 5-8) `[MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]`¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand that classes are instances of `type` and construct classes dynamically without the `class` keyword.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day113/`, and review the mapped book section for `` `type(name, bases, namespace)` & how class statements execute at runtime`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `make_record_class` in `src/pymastery/meta_type.py` that builds slotted, repr-enabled classes dynamically via `type(...)`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `type(name, bases, namespace)` & how class statements execute at runtime `and save the test log beside` src/pymastery/meta_type.py `and` tests/test_meta_type.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `type(name, bases, namespace)` & how class statements execute at runtime`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day113/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** Dynamically created classes behave identically to hand-written classes and enforce `__slots__`.

- **Tangible Deliverable:** `src/pymastery/meta_type.py` and `tests/test_meta_type.py`
- **STOP WHEN:** Dynamically created classes behave identically to hand-written classes and enforce `__slots__`.
- **Do Not Study Yet:** C-level `PyTypeObject` struct fields.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 114: `__new__` vs `__init__`: object allocation, singletons, interning & immutable subclassing CORE: Advanced Track (Months 5-8) [MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Control object allocation before `__init__` runs using `__new__` to implement flyweight interning and subclass immutable types (`tuple`, `int`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`,

`pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day114/`, and review the mapped book section for `__new__ vs __init__: object allocation, singletons, interning & immutable subclassing`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement an interned `DeviceSpec` (`cuda:0`, `cpu`) and an immutable `ShapeTuple(tuple)` in `src/pymastery/new_allocation.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `__new__ vs __init__: object allocation, singletons, interning & immutable subclassing and save the test log beside` src/pymastery/new_allocation.py `and` tests/test_new_allocation.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `__new__ vs __init__: object allocation, singletons, interning & immutable subclassing, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day114/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** `DeviceSpec('cuda:0') is DeviceSpec('cuda:0')` evaluates to `True` (identical memory pointer).

- **Tangible Deliverable:** `src/pymastery/new_allocation.py` and `tests/test_new_allocation.py`
- **STOP WHEN:** `DeviceSpec('cuda:0') is DeviceSpec('cuda:0')` evaluates to `True` (identical memory pointer).
- **Do Not Study Yet:** Custom C slab allocators.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 115: `__init_subclass__` for plugin auto-registration & architectural invariant checks CORE: Advanced Track (Months 5-8) [MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use PEP 487 `__init_subclass__` hooks to automatically register model/kernel backends and validate subclass contracts at import time.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day115/`, and review the mapped book section for `__init_subclass__ for plugin auto-registration & architectural invariant checks`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `KernelBackendBase` in `src/pymastery/init_subclass_hook.py` that auto-registers subclasses into a dispatch table and rejects subclasses missing required attributes.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `__init_subclass__ for plugin auto-registration & architectural invariant checks and save the test log beside` src/pymastery/init_subclass_hook.py `and` tests/test_init_subclass_hook.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `__init_subclass__ for plugin auto-registration & architectural invariant checks, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day115/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** Defining a subclass automatically populates `KernelBackendBase.registry` without manual decorator calls.

- **Tangible Deliverable:** `src/pymastery/init_subclass_hook.py` and `tests/test_init_subclass_hook.py`
- **STOP WHEN:** Defining a subclass automatically populates `KernelBackendBase.registry` without manual decorator calls.
- **Do Not Study Yet:** Import hooks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 116: Class decorators & `@typing.dataclass_transform` for custom model systems CORE: Advanced Track (Months 5-8) `[MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]`¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a custom `@schema_model` class decorator that synthesizes `__init__`, `__repr__`, and field validators while informing static type checkers via `@dataclass_transform()`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day116/`, and review the mapped book section for `Class decorators & @typing.dataclass_transform for custom model systems`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `@schema_model` in `src/pymastery/schema_decorator.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Class decorators & @typing.dataclass_transform for custom model systems` and save the test log beside `src/pymastery/schema_decorator.py` and `tests/test_schema_decorator.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Class decorators & @typing.dataclass_transform for custom model systems`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day116/failure-notes.md`.

Engineering Trap to Avoid: Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

Daily Gate (STOP WHEN): `mypy --strict` recognizes synthesized constructor parameters and runtime validation catches bad types.

- **Tangible Deliverable:** `src/pymastery/schema_decorator.py` and `tests/test_schema_decorator.py`
- **STOP WHEN:** `mypy --strict` recognizes synthesized constructor parameters and runtime validation catches bad types.
- **Do Not Study Yet:** Compiler plugins.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 117: Metaclasses (`class Meta(type):`, `__prepare__`, `__new__`, `__call__`)

CORE: Advanced Track (Months 5-8) `[MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]`¶

The Big Picture: Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.

Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master the full metaclass lifecycle: ordering namespace dicts in `__prepare__`, rewriting class attributes in `__new__`, and intercepting instantiation in `__call__`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day117/`, and review the mapped book section for `Metaclasses (class Meta(type):, __prepare__, __new__, __call__)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ValidatedStructMeta` in `src/pymastery/metaclasses.py` that collects descriptor fields in declaration order and compiles a fast validation tuple.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Metaclasses (class Meta(type):, __prepare__, __new__, __call__)` and save the test log beside `src/pymastery/metaclasses.py` and `tests/test_metaclasses.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Metaclasses (class Meta(type):, __prepare__, __new__, __call__)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day117/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** Declaration order is preserved, descriptor names are bound, and metaclass conflicts are tested.

- **Tangible Deliverable:** `src/pymastery/metaclasses.py` and `tests/test_metaclasses.py`
- **STOP WHEN:** Declaration order is preserved, descriptor names are bound, and metaclass conflicts are tested.
- **Do Not Study Yet:** Multiple conflicting metaclasses in production.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 118: Building a PyTorch-style `Module` parameter & submodule registration system from scratch CORE: Advanced Track (Months 5-8) `[MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]`¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how `torch.nn.Module` tracks `parameters()`, `modules()`, and `state_dict()` by overriding `__setattr__` to intercept `Tensor`/`Parameter` and child `Module` assignments.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day118/`, and review the mapped book section for `Building a PyTorch-style Module parameter & submodule registration system from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MiniModule` and `Parameter` in `src/pymastery/mini_nn_module.py` supporting recursive `named_parameters()` and `state_dict()` extraction.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Building a PyTorch-style Module parameter & submodule registration system from scratch` and save the test log beside `src/pymastery/mini_nn_module.py` and `tests/test_mini_nn_module.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Building a PyTorch-style Module parameter & submodule registration system from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day118/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** Assigning `self.linear = SubModule()` and `self.weight = Parameter(...)` automatically registers them in `named_parameters()`.

- **Tangible Deliverable:** `src/pymastery/mini_nn_module.py` and `tests/test_mini_nn_module.py`
- **STOP WHEN:** Assigning `self.linear = SubModule()` and `self.weight = Parameter(...)` automatically registers them in `named_parameters()`.
- **Do Not Study Yet:** Autograd backward graph (covered in Month 6).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 119: Week 17 Gate: Declarative Schema & `MiniModule` Container

CORE: Advanced Track (Months 5-8) `[MINOR: Metaclass / class-hook safety and static type compatibility (dataclass_transform)]`¶

> **The Big Picture:** Master how Python constructs classes at runtime (`type`, `__new__`, `__prepare__`, `__init_subclass__`) to build declarative frameworks like Pydantic, PyTorch `nn.Module`, and SQLAlchemy.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `__init_subclass__`, descriptors, and `MiniModule` parameter tracking into a declarative model configuration and weight-tree inspector.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`) — because every ML framework (`torch.nn.Module`, `pydantic.BaseModel`, `triton.jit`) uses class-creation hooks under the hood.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. (Ramalho, O'Reilly) Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day119/`, and review the mapped book section for `Week 17 Gate: Declarative Schema &` MiniModule `Container`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build a 3-layer transformer weight skeleton with `MiniModule` and verify recursive `state_dict()` serialization.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 17 Gate: Declarative Schema &` MiniModule `Container` and save the test log beside `gates/week17-gate.md` and `tests/test_week17_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Metaclass / class-hook safety and static type compatibility (`dataclass_transform`): Verify every dynamically constructed or hooked class works with `isinstance`, `issubclass`, and `mypy --strict`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 17 Gate: Declarative Schema &` MiniModule `Container`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day119/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not reach for a custom metaclass when `__init_subclass__` or a class decorator solves the problem in half the code.

> **Daily Gate (STOP WHEN):** All metaclass, `__init_subclass__`, and `MiniModule` tests pass with zero warnings.

- **Tangible Deliverable:** `gates/week17-gate.md` and `tests/test_week17_gate.py`

- **STOP WHEN:** All metaclass, `__init_subclass__`, and `MiniModule` tests pass with zero warnings.
- **Do Not Study Yet:** AST compilation.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 18 - AST Manipulation, Bytecode (`dis`), Frame Inspection & Dynamic Codegen¶

Day 120: Runtime inspection with `inspect`: signatures, source extraction, frames & call stacks CORE: Advanced Track (Months 5-8) [MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use `inspect.signature`, `inspect.getsource`, and frame inspection (`sys._getframe`) to build zero-boilerplate RPC routers and diagnostic tracers.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day120/`, and review the mapped book section for `Runtime inspection with inspect: signatures, source extraction, frames & call stacks`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement an automatic CLI/RPC schema extractor in `src/pymastery/introspection.py` that inspects function type hints and defaults to build JSON schemas.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Runtime inspection with inspect: signatures, source extraction, frames & call stacks` and save the test log beside `src/pymastery/introspection.py` and `tests/test_introspection.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Runtime inspection with inspect: signatures, source extraction, frames & call`

`stacks`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day120/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Generates exact OpenAI-tool-style JSON schemas from typed Python functions.

- **Tangible Deliverable:** `src/pymastery/introspection.py` and `tests/test_introspection.py`
- **STOP WHEN:** Generates exact OpenAI-tool-style JSON schemas from typed Python functions.
- **Do Not Study Yet:** Modifying live C stack pointers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 121: Abstract Syntax Trees (`ast.parse`, `ast.dump`, `ast.NodeVisitor`) for static code analysis CORE: Advanced Track (Months 5-8) `[MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]`¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Parse Python source into an AST and traverse it with `NodeVisitor` to enforce custom architectural and security rules.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day121/`, and review the mapped book section for `Abstract Syntax Trees (ast.parse, ast.dump, ast.NodeVisitor) for static code analysis`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ForbiddenConstructLinter` in `src/pymastery/ast_linter.py` that flags `eval()`, bare `except:`, `time.sleep()` inside `async def`, and forbidden imports.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Abstract Syntax Trees (ast.parse, ast.dump, ast.NodeVisitor) for static code analysis` and save the test log beside `src/pymastery/ast_linter.py` and `tests/test_ast_linter.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Abstract Syntax Trees (ast.parse, ast.dump, ast.NodeVisitor) for static code analysis`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day121/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Accurately reports line numbers and column offsets for all seeded anti-patterns.

- **Tangible Deliverable:** `src/pymastery/ast_linter.py` and `tests/test_ast_linter.py`
- **STOP WHEN:** Accurately reports line numbers and column offsets for all seeded anti-patterns.
- **Do Not Study Yet:** Full type-inference engines.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 122: AST transformation (`ast.NodeTransformer`, `compile`, `ast.unparse`) & expression compilers CORE: Advanced Track (Months 5-8) [MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Rewrite AST nodes with `NodeTransformer` to fuse arithmetic operations or compile declarative filter expressions into fast Python callables.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day122/`, and review the mapped book section for `AST transformation (ast.NodeTransformer, compile, ast.unparse) & expression compilers`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ConstantFolderAndFusionTransformer` in `src/pymastery/ast_transform.py` that folds constant math and rewrites AST expressions before `compile()`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `AST transformation (ast.NodeTransformer, compile, ast.unparse) & expression compilers` and save the test log beside `src/pymastery/ast_transform.py` and `tests/test_ast_transform.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `AST transformation (ast.NodeTransformer, compile, ast.unparse) & expression compilers`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day122/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Transformed AST compiles cleanly and produces identical results with fewer bytecode instructions.

- **Tangible Deliverable:** `src/pymastery/ast_transform.py` and `tests/test_ast_transform.py`
- **STOP WHEN:** Transformed AST compiles cleanly and produces identical results with fewer bytecode instructions.
- **Do Not Study Yet:** LLVM IR codegen.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 123: Generating specialized validation code at class-creation time (How `@dataclass` & Pydantic work) CORE: Advanced Track (Months 5-8) `[MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]`¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate per-field descriptor/loop overhead by generating and compiling a flat `__init__` and `validate()` function string once at class definition time.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day123/`, and review the mapped book section for `Generating specialized validation code at class-creation time (How` @dataclass `& Pydantic work)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `CompiledValidatorMeta` in `src/pymastery/codegen_validator.py` that compiles an unrolled field-validation function per schema class.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Generating specialized validation code at class-creation time (How` @dataclass `& Pydantic work)` and save the test log beside `src/pymastery/codegen_validator.py` and `tests/test_codegen_validator.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Generating specialized validation code at class-creation time (How` @dataclass `& Pydantic work)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day123/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Benchmarks >3x faster than loop-based reflection validation across 100,000 instantiations.

- **Tangible Deliverable:** `src/pymastery/codegen_validator.py` and `tests/test_codegen_validator.py`
- **STOP WHEN:** Benchmarks >3x faster than loop-based reflection validation across 100,000 instantiations.
- **Do Not Study Yet:** JIT machine-code emission.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 124: CPython Bytecode (`dis.dis`, `dis.Bytecode`, code objects `__code__`) & the evaluation stack CORE: Advanced Track (Months 5-8) `[MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]`¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Read CPython stack-machine bytecode (`LOAD_FAST`, `LOAD_ATTR`, `CALL`, `BINARY_OP`, `RETURN_VALUE`) and inspect `co_varnames`, `co_consts`, and `co_names`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day124/`, and review the mapped book section for `CPython Bytecode (dis.dis, dis.Bytecode, code objects __code__) & the evaluation stack`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a bytecode analyzer in `src/pymastery/bytecode_inspector.py` that counts global lookups (`LOAD_GLOBAL`) vs local lookups (`LOAD_FAST`) and detects loop allocations.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `CPython Bytecode (dis.dis, dis.Bytecode, code objects __code__) & the evaluation stack` and

save the test log beside `src/pymastery/bytecode_inspector.py` and `tests/test_bytecode_inspector.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `CPython Bytecode (dis.dis, dis.Bytecode, code objects __code__) & the evaluation stack`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day124/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Correctly identifies `LOAD_GLOBAL` instructions inside hot loops and verifies optimization when bound to local variables.

- **Tangible Deliverable:** `src/pymastery/bytecode_inspector.py` and `tests/test_bytecode_inspector.py`
- **STOP WHEN:** Correctly identifies `LOAD_GLOBAL` instructions inside hot loops and verifies optimization when bound to local variables.
- **Do Not Study Yet:** PEP 659 specializing adaptive interpreter C source.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 125: Execution tracing (`sys.settrace`, `sys.monitoring`) & building a lightweight line profiler/tracer CORE: Advanced Track (Months 5-8) `[MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]`¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use Python 3.12+ low-overhead `sys.monitoring` (or `sys.setprofile`) to trace function calls, returns, and exceptions without modifying source code.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work

by inspecting Python ASTs and bytecode frames!.

- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day125/`, and review the mapped book section for `Execution tracing (sys.settrace, sys.monitoring) & building a lightweight line profiler/tracer`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `CallGraphTracer` in `src/pymastery/call_tracer.py` that captures call counts and inclusive/exclusive execution times.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Execution tracing (sys.settrace, sys.monitoring) & building a lightweight line profiler/tracer` and save the test log beside `src/pymastery/call_tracer.py` and `tests/test_call_tracer.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Execution tracing (sys.settrace, sys.monitoring) & building a lightweight line profiler/tracer`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day125/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Captures exact call hierarchy and restores monitoring state cleanly on context exit.

- **Tangible Deliverable:** `src/pymastery/call_tracer.py` and `tests/test_call_tracer.py`
- **STOP WHEN:** Captures exact call hierarchy and restores monitoring state cleanly on context exit.
- **Do Not Study Yet:** Kernel eBPF probes.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 126: Week 18 Gate: Mini `@jit_trace` Graph Extractor (How `@triton.jit` / TorchFX works) CORE: Advanced Track (Months 5-8) `[MINOR: AST safety (ast.parse / ast.NodeVisitor) and bytecode instruction count (dis)]`¶

> **The Big Picture:** Understand how Python compiles source code into Abstract Syntax Trees (`ast`) and stack-machine bytecode (`dis`), and how compilers like `torch.compile`, Triton, and Pydantic generate specialized fast paths.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a decorator `@trace_computation_graph` that traces symbolic proxy objects or AST nodes to record a Computation DAG (nodes, ops, shapes) from a Python function.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`) — because `torch.compile` (TorchDynamo) and `@triton.jit` work by inspecting Python ASTs and bytecode frames!.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 4–8 & Fluent Python, 2nd Ed. Ch. 24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day126/`, and review the mapped book section for `Week 18 Gate: Mini` @jit_trace `Graph Extractor (How` @triton.jit `/ TorchFX works)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/symbolic_tracer.py` that traces `def attention_block(q, k, v): ...` into an explicit operator DAG and optimizes dead nodes.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 18 Gate: Mini` @jit_trace `Graph Extractor (How` @triton.jit `/ TorchFX works)` and save the test log beside `gates/week18-gate.md` and `src/pymastery/symbolic_tracer.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** AST safety (`ast.parse` / `ast.NodeVisitor`) and bytecode instruction count (`dis`): Never run `eval()` or `exec()` on untrusted user strings; restrict dynamic codegen to trusted internal schema/kernel compilation.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 18 Gate: Mini` @jit_trace `Graph Extractor (How` @triton.jit `/ TorchFX works)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day126/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not mutate AST nodes without calling `ast.fix_missing_locations(tree)` before compiling.

> **Daily Gate (STOP WHEN):** Extracts the exact operator graph from a Python function and passes dead-code elimination tests.

- **Tangible Deliverable:** `gates/week18-gate.md` and `src/pymastery/symbolic_tracer.py`
- **STOP WHEN:** Extracts the exact operator graph from a Python function and passes dead-code elimination tests.
- **Do Not Study Yet:** CUDA PTX emission.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 19 - Import System Internals, Plugin Architectures & Dependency Injection¶

Day 127: How `import` works: `sys.modules`, `sys.meta_path`, finders, loaders & `importlib` CORE: Advanced Track (Months 5-8) `[MINOR: Module import time (python -X importtime) and circular-import prevention]`¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Trace the exact steps CPython takes on `import foo`, cache behavior in `sys.modules`, and dynamic module loading via `importlib.import_module`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day127/`, and review the mapped book section for `How` import `works:` sys.modules`,` sys.meta_path`, finders, loaders &` importlib``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a dynamic backend loader and isolated module reloader in `src/pymastery/import_mechanics.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `How` import `works:` sys.modules`,` sys.meta_path`, finders, loaders &` importlib` `and save the test log beside` src/pymastery/import_mechanics.py `and` tests/test_import_mechanics.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `How import works: sys.modules, sys.meta_path, finders, loaders & importlib`, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day127/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** Loads backend modules on demand by string path and handles missing backends with clear diagnostics.

- **Tangible Deliverable:** `src/pymastery/import_mechanics.py` and `tests/test_import_mechanics.py`
- **STOP WHEN:** Loads backend modules on demand by string path and handles missing backends with clear diagnostics.
- **Do Not Study Yet:** Custom zipimport C hooks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 128: Lazy module loading (`__getattr__` on modules, PEP 562) for instant CLI startup CORE: Advanced Track (Months 5-8) `[MINOR: Module import time (python -X importtime) and circular-import prevention]`¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate multi-second CLI startup delays (caused by eagerly importing `torch` or heavy submodules) using PEP 562 module-level `__getattr__` and `TYPE_CHECKING` guards.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day128/`, and review the mapped book section for `Lazy module loading (__getattr__ on modules, PEP 562) for instant CLI startup`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement lazy submodule exporting in `src/pymastery/lazy_imports.py` so heavy modules load only when accessed while `mypy --strict` still sees full types via `if TYPE_CHECKING:`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Lazy module loading (__getattr__ on modules, PEP 562) for instant CLI startup` and save the test log beside `src/pymastery/lazy_imports.py` and `tests/test_lazy_imports.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Lazy module loading (__getattr__ on modules, PEP 562) for instant CLI startup`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day128/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** Top-level import takes < 2ms and heavy submodule is not in `sys.modules` until first attribute access.

- **Tangible Deliverable:** `src/pymastery/lazy_imports.py` and `tests/test_lazy_imports.py`
- **STOP WHEN:** Top-level import takes < 2ms and heavy submodule is not in `sys.modules` until first attribute access.
- **Do Not Study Yet:** Custom C import hooks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 129: Plugin discovery via `importlib.metadata` entry points & extensible hardware backends CORE: Advanced Track (Months 5-8) `[MINOR: Module import time (python -X importtime) and circular-import prevention]`¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Discover third-party or modular quantizer/sampler plugins dynamically using standard `importlib.metadata.entry_points`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day129/`, and review the mapped book section for `Plugin discovery via` importlib.metadata `entry points & extensible hardware backends`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `PluginManager` in `src/pymastery/plugin_system.py` with protocol verification for loaded plugins.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Plugin discovery via` importlib.metadata `entry points & extensible hardware backends` and save the test log beside `src/pymastery/plugin_system.py` and `tests/test_plugin_system.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Plugin discovery via` importlib.metadata `entry points & extensible hardware backends`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day129/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** Discovers registered plugins, validates they satisfy `BackendProtocol`, and isolates faulty plugins.

- **Tangible Deliverable:** `src/pymastery/plugin_system.py` and `tests/test_plugin_system.py`
- **STOP WHEN:** Discovers registered plugins, validates they satisfy `BackendProtocol`, and isolates faulty plugins.
- **Do Not Study Yet:** OS dynamic linker `dlopen`.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 130: Type-hint-driven Dependency Injection container from scratch (How FastAPI `Depends` works) CORE: Advanced Track (Months 5-8) [MINOR: Module import time (python -X importtime) and circular-import prevention]¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a lightweight dependency injection resolver that inspects function type annotations and `Depends(...)` markers, resolves DAG dependencies, and caches per-request scopes.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day130/`, and review the mapped book section for `Type-hint-driven Dependency Injection container from scratch (How FastAPI Depends works)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `DependencyInjector` in `src/pymastery/di_container.py` supporting both sync and async providers and `AsyncExitStack` teardown.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Type-hint-driven Dependency Injection container from scratch (How FastAPI Depends works)` and save the test log beside `src/pymastery/di_container.py` and `tests/test_di_container.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Type-hint-driven Dependency Injection container from scratch (How FastAPI Depends works)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day130/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** Shared dependencies within one request execute once, and generator dependencies clean up deterministically after the request.

- **Tangible Deliverable:** `src/pymastery/di_container.py` and `tests/test_di_container.py`
- **STOP WHEN:** Shared dependencies within one request execute once, and generator dependencies clean up deterministically after the request.
- **Do Not Study Yet:** Java Spring XML containers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 131: Middleware pipeline architecture (Onion model / ASGI middleware composition) CORE: Advanced Track (Months 5-8) [MINOR: Module import time (python -X importtime) and circular-import prevention]¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Compose cross-cutting concerns (request ID, timing, authentication, rate limiting, exception boundary) as zero-coupling middleware layers.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day131/`, and review the mapped book section for `Middleware pipeline architecture (Onion model / ASGI middleware composition)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MiddlewarePipeline` in `src/pymastery/middleware.py` wrapping async request handlers.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Middleware pipeline architecture (Onion model / ASGI middleware composition)` and save the test log beside `src/pymastery/middleware.py` and `tests/test_middleware.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Middleware pipeline architecture (Onion model / ASGI middleware composition)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day131/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** Execution order follows exact onion ordering (pre-1 -> pre-2 -> handler -> post-2 -> post-1) even when exceptions occur.

- **Tangible Deliverable:** `src/pymastery/middleware.py` and `tests/test_middleware.py`
- **STOP WHEN:** Execution order follows exact onion ordering (pre-1 -> pre-2 -> handler -> post-2 -> post-1) even when exceptions occur.
- **Do Not Study Yet:** Envoy WASM filters.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 132: Configuration hot-reloading, immutable state swapping & thread/task safety CORE: Advanced Track (Months 5-8) `[MINOR: Module import time (python -X importtime) and circular-import prevention]`¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Update routing weights, rate limits, and logging levels live without restarting the server by atomically swapping immutable config snapshots.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day132/`, and review the mapped book section for `Configuration hot-reloading, immutable state swapping & thread/task safety`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AtomicConfigHolder[T]` in `src/pymastery/hot_reload.py` where in-flight requests keep their starting config snapshot while new requests see the new snapshot.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Configuration hot-reloading, immutable state swapping & thread/task safety` and save the test log beside `src/pymastery/hot_reload.py` and `tests/test_hot_reload.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Configuration hot-reloading, immutable state swapping & thread/task safety`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day132/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** In-flight requests never observe torn/partially-updated configuration state during concurrent reloads.

- **Tangible Deliverable:** `src/pymastery/hot_reload.py` and `tests/test_hot_reload.py`
- **STOP WHEN:** In-flight requests never observe torn/partially-updated configuration state during concurrent reloads.
- **Do Not Study Yet:** Distributed etcd watches.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 133: Week 19 Gate: Modular Plugin & DI Micro-Framework

CORE: Advanced Track (Months 5-8) `[MINOR: Module import time (python -X importtime) and circular-import prevention]`¶

> **The Big Picture:** Understand how `import`, `sys.modules`, `importlib`, entry-point plugins, and ASGI middleware pipelines work inside large modular codebases.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine lazy imports, plugin discovery, `DependencyInjector`, and `MiddlewarePipeline` into a cohesive micro-framework.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Module import time (`python -X importtime`) and circular-import prevention — because slow startup times and circular imports plague large ML repositories unless the import system is mastered.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22, 24 & Python 3.12 `importlib` Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day133/`, and review the mapped book section for `Week 19 Gate: Modular Plugin & DI Micro-Framework`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Verify cold import time < 5ms and run a 50-test suite covering DI cleanup and middleware fault isolation.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 19 Gate: Modular Plugin & DI Micro-Framework` and save the test log beside `gates/week19-gate.md` and `tests/test_week19_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Module import time (`python -X importtime`) and circular-import prevention: Verify zero circular imports and keep top-level package import time under 30ms.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 19 Gate: Modular Plugin & DI Micro-Framework`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day133/failure-notes.md`.

> **Engineering Trap to Avoid:** Never execute heavy I/O, GPU initialization, or network calls at module import top-level!

> **Daily Gate (STOP WHEN):** All tests pass and `python -X importtime` confirms lazy loading.

- **Tangible Deliverable:** `gates/week19-gate.md` and `tests/test_week19_gate.py`
- **STOP WHEN:** All tests pass and `python -X importtime` confirms lazy loading.
- **Do Not Study Yet:** WebSockets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 20 - Month 5 Milestone: Authoring a FastAPI + Pydantic Style Micro-Framework from Scratch¶

Day 134: `NanoSchema` Part 1: Metaclass, type-annotation parsing & nested model resolution CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build `BaseModel` in `src/pymastery/nanoschema/model.py` that inspects `__annotations__` (including `Optional`, `Literal`, `list[T]`, `dict[K, V]`, and nested `BaseModel`s).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day134/`, and review the mapped book section for `` `NanoSchema` Part 1: Metaclass, type-annotation parsing & nested model resolution ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `NanoModel` with recursive dict parsing, validation error paths (`loc: ('items', 2, 'temperature')`), and `to_dict()`/`to_json()`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `NanoSchema` Part 1: Metaclass, type-annotation parsing & nested model resolution `and save the test log beside` src/pymastery/nanoschema/model.py `and` tests/test_nanoschema.py ``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `NanoSchema` Part 1: Metaclass, type-annotation parsing & nested model resolution`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day134/failure-notes.md ``.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** Reports exact nested field paths on validation errors and coerces/validates types strictly.

- **Tangible Deliverable:** `src/pymastery/nanoschema/model.py` and `tests/test_nanoschema.py`
- **STOP WHEN:** Reports exact nested field paths on validation errors and coerces/validates types strictly.
- **Do Not Study Yet:** Rust `pydantic-core`.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 135: `NanoSchema` Part 2: Code-generated fast validator & JSON Schema generator CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Compile per-model validation code and generate OpenAPI-compatible JSON Schema definitions directly from `NanoModel` subclasses.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day135/`, and review the mapped book section for `` `NanoSchema` Part 2: Code-generated fast validator & JSON Schema generator``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Add `compile_validator()` and `json_schema()` to `src/pymastery/nanoschema/model.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `NanoSchema` Part 2: Code-generated fast validator & JSON Schema generator `and save the test log beside` src/pymastery/nanoschema/model.py `and` tests/test_nanoschema_codegen.py``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `NanoSchema` Part 2: Code-generated fast validator & JSON Schema generator`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day135/failure-notes.md``.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** Generated JSON Schema matches OpenAI/OpenAPI structure and compiled validator is >2.5x faster than reflective loop.

- **Tangible Deliverable:** `src/pymastery/nanoschema/model.py` and `tests/test_nanoschema_codegen.py`
- **STOP WHEN:** Generated JSON Schema matches OpenAI/OpenAPI structure and compiled validator is >2.5x faster than reflective loop.
- **Do Not Study Yet:** XML schemas.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 136: `NanoServe` Part 1: Decorator route registration (`@app.post`), path parameters & type-driven dispatch CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build `NanoServe` ASGI router that inspects handler signatures: parameters typed as a `NanoModel` parse the JSON request body; primitives parse path/query params; `Depends` resolves DI.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day136/`, and review the mapped book section for `NanoServe Part 1: Decorator route registration (@app.post), path parameters & type-driven dispatch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/nanoserve/app.py` and `router.py`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `NanoServe Part 1: Decorator route registration (@app.post), path parameters & type-driven dispatch and save the test log beside src/pymastery/nanoserve/app.py and tests/test_nanoserve_routing.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `NanoServe Part 1: Decorator route registration (@app.post), path parameters & type-driven dispatch, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day136/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** Invalid JSON bodies automatically return HTTP 422 with structured `NanoSchema` error details.

- **Tangible Deliverable:** `src/pymastery/nanoserve/app.py` and `tests/test_nanoserve_routing.py`
- **STOP WHEN:** Invalid JSON bodies automatically return HTTP 422 with structured `NanoSchema` error details.
- **Do Not Study Yet:** HTML template engines.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 137: `NanoServe` Part 2: Streaming SSE responses, lifespan context & middleware integration CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Support returning `AsyncIterator[NanoModel | str]` from route handlers to automatically stream Server-Sent Events (`text/event-stream`) with disconnect cleanup.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day137/`, and review the mapped book section for `` `NanoServe Part 2: Streaming SSE responses, lifespan context & middleware integration` ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Extend `src/pymastery/nanoserve/app.py` with `EventStreamResponse` and lifespan startup/shutdown hooks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `NanoServe Part 2: Streaming SSE responses, lifespan context & middleware integration and save the test log beside src/pymastery/nanoserve/app.py and tests/test_nanoserve_streaming.py` ``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `NanoServe Part 2: Streaming SSE responses, lifespan context & middleware integration, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day137/failure-notes.md` ``.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** Async generator routes stream SSE frames and trigger generator `finally:` blocks on client disconnect.

- **Tangible Deliverable:** `src/pymastery/nanoserve/app.py` and `tests/test_nanoserve_streaming.py`
- **STOP WHEN:** Async generator routes stream SSE frames and trigger generator `finally:` blocks on client disconnect.
- **Do Not Study Yet:** HTTP/2 server push.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 138: `NanoServe` Part 3: Automatic `/openapi.json` spec generation from routes & `NanoModels` CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Introspect all registered routes, request body `NanoModel`s, response models, and query parameters to serve a live `/openapi.json` specification.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day138/`, and review the mapped book section for `` `NanoServe` Part 3: Automatic `/openapi.json` spec generation from routes & `NanoModel`s ``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/nanoserve/openapi.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `` `NanoServe` Part 3: Automatic `/openapi.json` spec generation from routes & `NanoModel`s `and save the test log beside` src/pymastery/nanoserve/openapi.py `and` tests/test_nanoserve_openapi.py ``.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `` `NanoServe` Part 3: Automatic `/openapi.json` spec generation from routes & `NanoModel`s, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day138/failure-notes.md ``.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** `GET /openapi.json` returns a valid OpenAPI 3.1 JSON document describing all endpoints and schemas.

- **Tangible Deliverable:** `src/pymastery/nanoserve/openapi.py` and `tests/test_nanoserve_openapi.py`

- **STOP WHEN:** `GET /openapi.json` returns a valid OpenAPI 3.1 JSON document describing all endpoints and schemas.
- **Do Not Study Yet:** Swagger UI CSS assets.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 139: Migrating the Month 4 Inference Server onto `NanoServe` & `NanoSchema` CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Replace manual request parsing in your inference server with declarative `@app.post('/v1/completions')` and `NanoModel` schemas.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day139/`, and review the mapped book section for `Migrating the Month 4 Inference Server onto` NanoServe `&` NanoSchema``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/nanoserve/inference_api.py` and run the full benchmark and chaos test suite against it.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Migrating the Month 4 Inference Server onto` NanoServe `&` NanoSchema` `and save the test log beside` src/pymastery/nanoserve/inference_api.py `and` tests/test_nanoserve_inference.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Migrating the Month 4 Inference Server onto` NanoServe `&` NanoSchema`, `verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day139/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** Codebase shrinks by >35% while retaining 100% performance and passing all chaos tests.

- **Tangible Deliverable:** `src/pymastery/nanoserve/inference_api.py` and `tests/test_nanoserve_inference.py`
- **STOP WHEN:** Codebase shrinks by >35% while retaining 100% performance and passing all chaos tests.
- **Do Not Study Yet:** Third-party web frameworks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 140: Week 20 Gate (Month 5 Review): Metaprogramming & Framework Defense CORE: Advanced Track (Months 5-8)

MINOR: Zero external dependencies, 100% static typing, and codegen validation speed ¶

> **The Big Picture:** Prove complete mastery of Python metaprogramming, descriptors, AST/codegen, type introspection, and ASGI by building `NanoServe` & `NanoSchema` from scratch.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Complete Month 5 gate reviewing metaclasses, `__init_subclass__`, AST/bytecode, `MiniModule`, `NanoSchema`, and `NanoServe`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero external dependencies, 100% static typing, and codegen validation speed — because once you have built your own Pydantic and FastAPI from scratch, no Python library is ever 'magic' to you again.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 22–24

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day140/`, and review the mapped book section for `Week 20 Gate (Month 5 Review): Metaprogramming & Framework Defense`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run the full test suite (Days 1–140) and complete the closed-book metaprogramming & bytecode defense.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 20 Gate (Month 5 Review): Metaprogramming & Framework Defense` and save the test log beside `gates/month05-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero external dependencies, 100% static typing, and codegen validation speed: Every component of `NanoSchema` and `NanoServe` must be written in pure Python standard library.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 20 Gate (Month 5 Review): Metaprogramming & Framework Defense`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day140/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not copy library code—build a clean, minimal, well-tested implementation of the core mechanisms.

> **Daily Gate (STOP WHEN):** All 20 weeks of tests pass with 0 errors and `mypy --strict` passes.

- **Tangible Deliverable:** `gates/month05-gate.md`
- **STOP WHEN:** All 20 weeks of tests pass with 0 errors and `mypy --strict` passes.
- **Do Not Study Yet:** C-level buffer protocol (starts tomorrow in Month 6!).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 6: CPython Memory, Zero-Copy Buffers, Strided Tensors, LLM & Paged KV Cache (Weeks 21–24, Days 141–168)¶

Week 21 - CPython Memory Architecture, Refcounting, Cyclic GC & Zero-Copy Buffers¶

Day 141: CPython `PyObject` header (`ob_refcnt`, `ob_type`), `PyVarObject`, boxed integers & list over-allocation CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]`¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand why a Python `int` is 28 bytes (vs 4/8 bytes in C), how `list` stores an array of `PyObject*` pointers with ~12.5% growth over-allocation, and how `dict` uses compact hash tables.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day141/`, and review the mapped book section for `CPython PyObject header (ob_refcnt, ob_type), PyVarObject, boxed integers & list over-allocation`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write `src/pymastery/memory_layout.py` inspecting exact byte sizes (`sys.getsizeof`), reference counts (`sys.getrefcount`), and `list` growth allocation steps.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `CPython PyObject header (ob_refcnt, ob_type), PyVarObject, boxed integers & list over-allocation` and save the test log beside `src/pymastery/memory_layout.py` and `tests/test_memory_layout.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `CPython PyObject header (ob_refcnt, ob_type), PyVarObject, boxed integers & list over-allocation`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day141/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** Empirically maps every `list.append` reallocation boundary and compares `list[float]` vs `array('f')` memory footprint.

- **Tangible Deliverable:** `src/pymastery/memory_layout.py` and `tests/test_memory_layout.py`
- **STOP WHEN:** Empirically maps every `list.append` reallocation boundary and compares `list[float]` vs `array('f')` memory footprint.
- **Do Not Study Yet:** Modifying CPython C source.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 142: Reference counting vs Generational Cyclic Garbage Collector (gc module) & GC freeze/tuning CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand immediate destruction via refcounting (`ob_refcnt == 0`) vs cyclic GC generations (0, 1, 2), reference cycles, and `gc.freeze()` for copy-on-write multi-worker servers.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day142/`, and review the mapped book section for `Reference counting vs Generational Cyclic Garbage Collector (gc module) & GC freeze/tuning`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a cycle detector, GC pause timer callback (`gc.callbacks`), and `gc.freeze()` pre-fork/warmup helper in `src/pymastery/gc_control.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Reference counting vs Generational Cyclic Garbage Collector (gc module) & GC freeze/tuning` and save the test log beside `src/pymastery/gc_control.py` and `tests/test_gc_control.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Reference counting vs Generational Cyclic Garbage Collector (gc module) & GC freeze/tuning`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day142/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** Demonstrates reference cycle collection, measures GC pause duration in microseconds, and proves `gc.get_freeze_count()` moves warmup objects out of GC scans.

- **Tangible Deliverable:** `src/pymastery/gc_control.py` and `tests/test_gc_control.py`
- **STOP WHEN:** Demonstrates reference cycle collection, measures GC pause duration in microseconds, and proves `gc.get_freeze_count()` moves warmup objects out of GC scans.
- **Do Not Study Yet:** JVM G1GC tuning.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 143: Memory profiling with `tracemalloc`: snapshots, diffs, domain filters & leak hunting CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use `tracemalloc.take_snapshot()` and `snapshot2.compare_to(snapshot1, 'lineno')` to pinpoint the exact file and line number leaking memory over time.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day143/`, and review the mapped book section for `Memory profiling with tracemalloc: snapshots, diffs, domain filters & leak hunting`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Build `MemoryLeakDetector` in `src/pymastery/leak_hunter.py` that runs a target callable for N warmup + M measured iterations and asserts zero positive growth per iteration.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Memory profiling with tracemalloc: snapshots, diffs, domain filters & leak hunting` and save the test log beside `src/pymastery/leak_hunter.py` and `tests/test_leak_hunter.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Memory profiling with tracemalloc: snapshots, diffs, domain filters & leak hunting`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day143/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** Catches a seeded 64-byte-per-request cache leak and reports the exact offending line number.

- **Tangible Deliverable:** `src/pymastery/leak_hunter.py` and `tests/test_leak_hunter.py`
- **STOP WHEN:** Catches a seeded 64-byte-per-request cache leak and reports the exact offending line number.
- **Do Not Study Yet:** Linux kernel kmemleak.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 144: Contiguous typed arrays (`array.array`, `bytearray`) vs `list` of boxed objects CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]`¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Store millions of unboxed C primitives (`'f'` float32, `'i'` int32, `'H'` uint16) in contiguous memory using `array.array` and `bytearray`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day144/`, and review the mapped book section for `Contiguous typed arrays (`array.array`, `bytearray`) vs` list `of boxed objects`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Benchmark memory footprint and sequential scan speed of 1,000,000 float32 numbers in `list[float]` vs `array('f')` in `src/pymastery/contiguous_arrays.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Contiguous typed arrays (`array.array`, bytearray`) vs` list `of boxed objects` and save the test log beside `src/pymastery/contiguous_arrays.py` and `tests/test_contiguous_arrays.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Contiguous typed arrays (`array.array`, bytearray`) vs` list `of boxed objects`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day144/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** `array('f')` uses exactly 4,000,000 payload bytes (~8x less RAM than `list[float]`).

- **Tangible Deliverable:** `src/pymastery/contiguous_arrays.py` and `tests/test_contiguous_arrays.py`
- **STOP WHEN:** `array('f')` uses exactly 4,000,000 payload bytes (~8x less RAM than `list[float]`).
- **Do Not Study Yet:** CUDA device malloc.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 145: The Buffer Protocol & `memoryview`: zero-copy slicing, casting (`cast('f')`) & multi-dimensional views CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy verification (`**`memoryview`**` address checks) and GC pause control (`**`gc`**`)]`¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master Python's C-level Buffer Protocol via `memoryview`: reinterpreting raw `bytearray`/`mmap` buffers as typed arrays (`mv.cast('f', shape=[r, c])`) without copying a single byte.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day145/`, and review the mapped book section for `The Buffer Protocol & memoryview: zero-copy slicing, casting (cast('f')) & multi-dimensional views`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ZeroCopyTensorBuffer` in `src/pymastery/buffer_views.py` supporting zero-copy dtype casting, reshaping, and sub-range slicing.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The Buffer Protocol & memoryview: zero-copy slicing, casting (cast('f')) & multi-dimensional views` and save the test log beside `src/pymastery/buffer_views.py` and `tests/test_buffer_views.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The Buffer Protocol & memoryview: zero-copy slicing, casting (cast('f')) & multi-dimensional views`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day145/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** Mutating a slice of the `float32` cast view immediately mutates the underlying raw `bytearray`, proving zero-copy memory sharing.

- **Tangible Deliverable:** `src/pymastery/buffer_views.py` and `tests/test_buffer_views.py`
- **STOP WHEN:** Mutating a slice of the `float32` cast view immediately mutates the underlying raw `bytearray`, proving zero-copy memory sharing.
- **Do Not Study Yet:** RDMA NIC verbs.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 146: Zero-copy network & file I/O (`socket.recv_into`, `file.readinto`) with pre-allocated ring buffers CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate per-read byte allocations on the hot path by reading directly from sockets and files into pre-allocated `bytearray`/`memoryview` slices using `readinto` / `recv_into`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day146/`, and review the mapped book section for `Zero-copy network & file I/O (socket.recv_into, file.readinto) with pre-allocated ring buffers`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `PreallocatedStreamReader` in `src/pymastery/zero_copy_io.py` that processes a 10 MB stream using a single reusable 64 KB buffer.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Zero-copy network & file I/O (socket.recv_into, file.readinto) with pre-allocated ring buffers` and save the test log beside `src/pymastery/zero_copy_io.py` and `tests/test_zero_copy_io.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Zero-copy network & file I/O (socket.recv_into, file.readinto) with pre-allocated ring buffers`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day146/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** `tracemalloc` confirms zero buffer allocations inside the read loop across 10 MB of streaming I/O.

- **Tangible Deliverable:** `src/pymastery/zero_copy_io.py` and `tests/test_zero_copy_io.py`
- **STOP WHEN:** `tracemalloc` confirms zero buffer allocations inside the read loop across 10 MB of streaming I/O.
- **Do Not Study Yet:** Linux `io_uring` C bindings.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 147: Week 21 Gate: Zero-Copy Paged KV-Cache Arena Simulator

CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy verification (memoryview address checks) and GC pause control (gc)]`¶

> **The Big Picture:** Look inside the CPython virtual machine: `PyObject` memory layout, `ob_refcnt`, generational garbage collection pauses, `mmap`, and the C-level Buffer Protocol (`memoryview`).
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a pre-allocated contiguous slab arena (`bytearray` + `memoryview`) that allocates and frees fixed-size KV-cache blocks with zero runtime heap allocations.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`) — because GC pauses and accidental gigabyte buffer copies cause latency spikes and OOM crashes in model servers.
- **Mapped Book & Docs:** CPython Internals (Shaw, Real Python) Ch. 9, 10 & High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day147/`, and review the mapped book section for `Week 21 Gate: Zero-Copy Paged KV-Cache Arena Simulator`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `SlabArenaAllocator` in `src/pymastery/slab_arena.py` and verify zero heap growth across 100,000 block alloc/free cycles.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 21 Gate: Zero-Copy Paged KV-Cache Arena Simulator` and save the test log beside `gates/week21-gate.md` and `src/pymastery/slab_arena.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy verification (`memoryview` address checks) and GC pause control (`gc`): Verify zero-copy operations by comparing underlying memory addresses (`ctypes.addressof` or `memoryview.obj`) and `tracemalloc` bytes allocated.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 21 Gate: Zero-Copy Paged KV-Cache Arena Simulator`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day147/failure-notes.md`.

> **Engineering Trap to Avoid:** Never slice a `bytes` object (`data[1000:]`) in a hot loop—slicing `bytes` copies memory! Wrap it in `memoryview(data)[1000:]` for O(1) zero-copy slicing.

> **Daily Gate (STOP WHEN):** 100,000 block allocations and deallocations perform zero `malloc`s after initialization.

- **Tangible Deliverable:** `gates/week21-gate.md` and `src/pymastery/slab_arena.py`
- **STOP WHEN:** 100,000 block allocations and deallocations perform zero `malloc`s after initialization.
- **Do Not Study Yet:** GPU page faults.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 22 - Strided N-Dimensional Arrays, Vectorization & NumPy/PyTorch Memory Mechanics¶

Day 148: Implementing a Strided N-Dimensional `NDArray` from scratch in pure Python (`buffer`, `offset`, `shape`, `strides`) CORE: Advanced Track (Months 5-8) `[MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]`¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Demystify how tensors map N-dimensional coordinates `(i, j, k)` to a 1D flat memory offset via `offset + sum(idx * stride for idx, stride in zip(indices, strides))`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day148/`, and review the mapped book section for `Implementing a Strided N-Dimensional` NDArray `from scratch in pure Python (`buffer`,` offset`,` shape`,` strides`)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `StridedTensor` over a flat `array('f')` buffer in `src/pymastery/strided_tensor.py` supporting arbitrary rank indexing and row-major (C-contiguous) stride calculation.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing a Strided N-Dimensional` NDArray `from scratch in pure Python (`buffer`,` offset`,` shape`,` strides`)` and save the test log beside `src/pymastery/strided_tensor.py` and `tests/test_strided_tensor.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing a Strided N-Dimensional` NDArray `from scratch in pure Python (`buffer`,` offset`,` shape`,` strides`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day148/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** 3D indexing `t[b, s, d]` maps to the exact flat offset and matches reference values.

- **Tangible Deliverable:** `src/pymastery/strided_tensor.py` and `tests/test_strided_tensor.py`
- **STOP WHEN:** 3D indexing `t[b, s, d]` maps to the exact flat offset and matches reference values.
- **Do Not Study Yet:** CUDA texture memory.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 149: Zero-copy `slice`, `transpose`, `permute`, `squeeze` & `unsqueeze` by manipulating `shape` and `strides` only CORE: Advanced Track (Months 5-8) `[MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]`¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement `transpose(dim0, dim1)`, `permute(*dims)`, and multi-axis slicing on `StridedTensor` purely by returning a new view with swapped `shape`/`strides` and updated `offset`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day149/`, and review the mapped book section for `Zero-copy slice, transpose, permute, squeeze & unsqueeze by manipulating shape and strides only`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Add view operations (`transpose`, `permute`, `unsqueeze`, `is_contiguous()`, `contiguous()`) to `src/pymastery/strided_tensor.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Zero-copy slice, transpose, permute, squeeze & unsqueeze by manipulating shape and strides only` and save the test log beside `src/pymastery/strided_tensor.py` and `tests/test_tensor_views.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Zero-copy slice, transpose, permute, squeeze & unsqueeze by manipulating shape and strides only`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day149/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** `t.transpose(0, 1)` performs O(1) work, copies zero floats, flips `is_contiguous()` to `False`, and `contiguous()` materializes a fresh C-order buffer.

- **Tangible Deliverable:** `src/pymastery/strided_tensor.py` and `tests/test_tensor_views.py`
- **STOP WHEN:** `t.transpose(0, 1)` performs O(1) work, copies zero floats, flips `is_contiguous()` to `False`, and `contiguous()` materializes a fresh C-order buffer.
- **Do Not Study Yet:** Sparse CSR tensors.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 150: The Broadcasting Algorithm from scratch (Right-aligned shapes, stride=0 virtual expansion) CORE: Advanced Track (Months 5-8) [MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand the exact mathematical trick behind NumPy/PyTorch broadcasting: setting `stride = 0` along a size-1 broadcast dimension so repeated reads hit the same memory address without copying!
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day150/`, and review the mapped book section for `The Broadcasting Algorithm from scratch (Right-aligned shapes, stride=0 virtual expansion)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `broadcast_shapes(s1, s2)` and zero-copy `broadcast_to(target_shape)` using `stride = 0` in `src/pymastery/broadcasting.py`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The Broadcasting Algorithm from scratch (Right-aligned shapes, stride=0 virtual expansion)` and save the test log beside `src/pymastery/broadcasting.py` and `tests/test_broadcasting.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The Broadcasting Algorithm from scratch (Right-aligned shapes, stride=0 virtual expansion)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day150/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** Broadcasting `(4, 1)` to `(2, 4, 8)` allocates zero extra floats (`strides` become `(0, 1, 0)`) and rejects incompatible shapes cleanly.

- **Tangible Deliverable:** `src/pymastery/broadcasting.py` and `tests/test_broadcasting.py`
- **STOP WHEN:** Broadcasting `(4, 1)` to `(2, 4, 8)` allocates zero extra floats (`strides` become `(0, 1, 0)`) and rejects incompatible shapes cleanly.
- **Do Not Study Yet:** Distributed all-gather.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 151: Vectorization mechanics, `einsum` notation & eliminating Python loop overhead CORE: Advanced Track (Months 5-8) `[MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]`¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand why Python loop overhead (~50–100ns per iteration) dominates scalar math and implement a general `einsum` (Einstein summation) evaluator and tiled matrix multiply.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come

from misunderstanding strides, contiguity, and broadcasting.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day151/`, and review the mapped book section for `Vectorization mechanics, einsum notation & eliminating Python loop overhead`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a clean `einsum` parser/evaluator and blocked/tiled 2D/batched matmul in `src/pymastery/tensor_math.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Vectorization mechanics, einsum notation & eliminating Python loop overhead` and save the test log beside `src/pymastery/tensor_math.py` and `tests/test_tensor_math.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Vectorization mechanics, einsum notation & eliminating Python loop overhead`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day151/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** Supports `'ij,jk->ik'`, `'bqd,bkd->bqk'`, and `'bhqk,bhkd->bhqd'` batched attention contractions accurately.

- **Tangible Deliverable:** `src/pymastery/tensor_math.py` and `tests/test_tensor_math.py`
- **STOP WHEN:** Supports `'ij,jk->ik'`, `'bqd,bkd->bqk'`, and `'bhqk,bhkd->bhqd'` batched attention contractions accurately.
- **Do Not Study Yet:** Cutlass C++ templates.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 152: Floating-point representation (`IEEE 754` FP32, FP16, BF16, FP8) & numerical stability CORE: Advanced Track (Months 5-8) `[MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]`¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Inspect sign, exponent, and mantissa bits of FP32/FP16/BF16 using `struct`, explain underflow/overflow, and implement numerically stable `logsumexp` and `softmax` (subtracting `max(x)`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day152/`, and review the mapped book section for `Floating-point representation (`IEEE 754 `FP32, FP16, BF16, FP8) & numerical stability`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement IEEE-754 bit inspector, BF16/INT8 quantizer/dequantizer, and stable `softmax` in `src/pymastery/numerics.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Floating-point representation (`IEEE 754 `FP32, FP16, BF16, FP8) & numerical stability` and save the test log beside `src/pymastery/numerics.py` and `tests/test_numerics.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Floating-point representation (`IEEE 754 `FP32, FP16, BF16, FP8) & numerical stability`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day152/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** Naive softmax overflows to `NaN` on `[1000.0, 1001.0]`, while your stable `softmax` returns exact finite probabilities `[0.2689, 0.7311]`.

- **Tangible Deliverable:** `src/pymastery/numerics.py` and `tests/test_numerics.py`

- **STOP WHEN:** Naive softmax overflows to `NaN` on `[1000.0, 1001.0]`, while your stable `softmax` returns exact finite probabilities `[0.2689, 0.7311]`.
- **Do Not Study Yet:** Stochastic rounding hardware.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 153: Symmetric & Asymmetric Quantization (INT8 / INT4) & Per-Channel/Group Scales from scratch CORE: Advanced Track (Months 5-8) [MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement weight-only and activation quantization (`scale`, `zero_point`, `quantize_int8`, `dequantize_int8`, group-wise INT4 packing into `uint8` bytes) in pure Python.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day153/`, and review the mapped book section for `Symmetric & Asymmetric Quantization (INT8 / INT4) & Per-Channel/Group Scales from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `GroupQuantizer` (packing two 4-bit integers per byte via bitwise shifts `& 0x0F`, `<< 4`) in `src/pymastery/quantization.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Symmetric & Asymmetric Quantization (INT8 / INT4) & Per-Channel/Group Scales from scratch` and save the test log beside `src/pymastery/quantization.py` and `tests/test_quantization.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Symmetric & Asymmetric Quantization (INT8 / INT4) & Per-Channel/Group Scales from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day153/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** INT4 packing compresses weight bytes by 8x vs FP32 and dequantized matmul stays within expected SQNR tolerance.

- **Tangible Deliverable:** `src/pymastery/quantization.py` and `tests/test_quantization.py`
- **STOP WHEN:** INT4 packing compresses weight bytes by 8x vs FP32 and dequantized matmul stays within expected SQNR tolerance.
- **Do Not Study Yet:** Marlin CUDA INT4 kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 154: Week 22 Gate: Pure-Python Strided Tensor & Quantized Linear Layer CORE: Advanced Track (Months 5-8) `[MINOR: Stride arithmetic (offset + sum(i_k * stride_k)) and allocation tracking]`¶

> **The Big Picture:** Understand how NumPy and PyTorch tensors actually work in memory: contiguous data buffers, `shape`, `strides`, zero-copy `transpose`/`reshape`, broadcasting rules, and vectorization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `StridedTensor`, `broadcast_to`, `einsum`, stable `softmax`, and `GroupQuantizer` into a working quantized tensor library.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking — because 90% of tensor bugs and hidden performance cliffs come from misunderstanding strides, contiguity, and broadcasting.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 2 & High Performance Python, 2nd Ed. Ch. 6

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day154/`, and review the mapped book section for `Week 22 Gate: Pure-Python Strided Tensor & Quantized Linear Layer`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Pass a 40-case test suite comparing shapes, strides, broadcasting, and numerical tolerances.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 22 Gate: Pure-Python Strided Tensor & Quantized Linear Layer` and save the test log beside

`gates/week22-gate.md` and `tests/test_week22_gate.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Stride arithmetic (`offset + sum(i_k * stride_k)`) and allocation tracking: Verify every stride operation (`transpose`, `slice`, `broadcast_to`) shares the same underlying buffer pointer and copies zero elements unless `contiguous()` is called.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 22 Gate: Pure-Python Strided Tensor & Quantized Linear Layer`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day154/failure-notes.md`.

> **Engineering Trap to Avoid:** Never write nested Python `for i in range(M): for j in range(N):` loops over tensor elements in production when vectorized array operations can run in C/SIMD.

> **Daily Gate (STOP WHEN):** All stride, broadcast, softmax, and INT8/INT4 quantization tests pass.

- **Tangible Deliverable:** `gates/week22-gate.md` and `tests/test_week22_gate.py`
- **STOP WHEN:** All stride, broadcast, softmax, and INT8/INT4 quantization tests pass.
- **Do Not Study Yet:** Training optimizers.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 23 - Building an Autograd Engine & Transformer Inference Runtime from Scratch¶

Day 155: Reverse-Mode Automatic Differentiation (`Value` / `Tensor` computation graph & topological `backward()`) from scratch

CORE: Advanced Track (Months 5-8) MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a dynamic computation graph (`_prev`, `_backward` closures, topological sort) supporting `+`, `*`, `@`, `exp`, `relu`, and `sum` with reverse-mode `backward()`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day155/`, and review the mapped book section for `Reverse-Mode Automatic Differentiation (`Value `/` Tensor `computation graph & topological` backward()`) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AutoTensor` in `src/pymastery/autograd.py` and verify analytical gradients against finite differences `(f(x+eps) - f(x-eps)) / (2*eps)`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Reverse-Mode Automatic Differentiation (`Value `/` Tensor `computation graph & topological` backward()`) from scratch` and save the test log beside `src/pymastery/autograd.py` and `tests/test_autograd.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Reverse-Mode Automatic Differentiation (`Value `/` Tensor `computation graph & topological` backward()`) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day155/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** Analytical gradients match numerical finite-difference gradients within `1e-4` across a 2-layer MLP.

- **Tangible Deliverable:** `src/pymastery/autograd.py` and `tests/test_autograd.py`
- **STOP WHEN:** Analytical gradients match numerical finite-difference gradients within `1e-4` across a 2-layer MLP.
- **Do Not Study Yet:** Higher-order Hessian-vector products.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 156: Implementing RMSNorm, LayerNorm, SiLU/SwiGLU & Residual Connections CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the core building blocks of modern Llama/Mistral/Qwen architectures: `RMSNorm`, gated `SwiGLU` feed-forward network, and pre-norm residual streams.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day156/`, and review the mapped book section for `Implementing RMSNorm, LayerNorm, SiLU/SwiGLU & Residual Connections`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `RMSNorm` and `SwiGLUFFN` in `src/pymastery/transformer_layers.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing RMSNorm, LayerNorm, SiLU/SwiGLU & Residual Connections` and save the test log beside `src/pymastery/transformer_layers.py` and `tests/test_transformer_layers.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing RMSNorm, LayerNorm, SiLU/SwiGLU & Residual Connections`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day156/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** Output shapes, variance normalization invariants, and numerical values match reference formulas.

- **Tangible Deliverable:** `src/pymastery/transformer_layers.py` and `tests/test_transformer_layers.py`
- **STOP WHEN:** Output shapes, variance normalization invariants, and numerical values match reference formulas.
- **Do Not Study Yet:** Distributed Megatron tensor-parallel MLP.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 157: Implementing Rotary Position Embeddings (RoPE) & Grouped-Query Attention (GQA) from scratch CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement complex/sinusoidal Rotary Position Embeddings (`apply_rope` with position offset `start_pos`) and Grouped-Query Attention (where `num_q_heads > num_kv_heads` via zero-copy KV head repetition).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day157/`, and review the mapped book section for `Implementing Rotary Position Embeddings (RoPE) & Grouped-Query Attention (GQA) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `RoPE` and `GroupedQueryAttention` in `src/pymastery/attention.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing Rotary Position Embeddings (RoPE) & Grouped-Query Attention (GQA) from scratch` and save the test log beside `src/pymastery/attention.py` and `tests/test_attention.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing Rotary Position Embeddings (RoPE) & Grouped-Query Attention`

(GQA) from scratch, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in artifacts/day157/failure-notes.md.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask -inf *before* softmax (and only to future positions j > i + past_len).

> **Daily Gate (STOP WHEN):** Verifies relative position invariance (q_m^T k_n depends only on m - n) and GQA head broadcasting.

- **Tangible Deliverable:** src/pymastery/attention.py and tests/test_attention.py
- **STOP WHEN:** Verifies relative position invariance (q_m^T k_n depends only on m - n) and GQA head broadcasting.
- **Do Not Study Yet:** RingAttention multi-host communication.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 158: Implementing Prefill vs Autoregressive Decode with an Explicit KV Cache (KVCache) from scratch CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation ([CORE - 70%]), and 15m static verification, unit testing, and failure diagnosis ([MINOR - 30%]).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build the two-phase LLM execution loop: Prefill processes [B, S_prompt, D] in parallel and populates KVCache; Decode processes [B, 1, D] per step using cached keys/values [B, H_kv, S_past + 1, D_head].
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open py-mastery/, check git status, create artifacts/day158/, and review the mapped book section for Implementing Prefill vs

`Autoregressive Decode with an Explicit KV Cache (KVCache) from scratch`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `KVCache` and `CachedTransformerBlock` in `src/pymastery/kv_cache_engine.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing Prefill vs Autoregressive Decode with an Explicit KV Cache (KVCache) from scratch` and save the test log beside `src/pymastery/kv_cache_engine.py` and `tests/test_kv_cache_engine.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing Prefill vs Autoregressive Decode with an Explicit KV Cache (KVCache) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day158/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** Generating 8 tokens step-by-step with `KVCache` produces the exact same logits (to `1e-5`) as recomputing the full prefix from scratch.

- **Tangible Deliverable:** `src/pymastery/kv_cache_engine.py` and `tests/test_kv_cache_engine.py`
- **STOP WHEN:** Generating 8 tokens step-by-step with `KVCache` produces the exact same logits (to `1e-5`) as recomputing the full prefix from scratch.
- **Do Not Study Yet:** Triton FlashDecoding kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 159: Implementing Token Sampling Pipelines: Greedy, Temperature, Top-K, Top-P (Nucleus) & Repetition Penalty CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write composable, deterministic (seeded) token samplers operating on final-step logits.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day159/`, and review the mapped book section for `Implementing Token Sampling Pipelines: Greedy, Temperature, Top-K, Top-P (Nucleus) & Repetition Penalty`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `sample_next_token` with temperature scaling, repetition penalty, top-k filtering, and top-p cumulative probability masking in `src/pymastery/sampler.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing Token Sampling Pipelines: Greedy, Temperature, Top-K, Top-P (Nucleus) & Repetition Penalty` and save the test log beside `src/pymastery/sampler.py` and `tests/test_sampler.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing Token Sampling Pipelines: Greedy, Temperature, Top-K, Top-P (Nucleus) & Repetition Penalty`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day159/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** `temperature=0.0` is strictly greedy; `top_k=1` matches greedy; `top_p=0.5` zeroes out tail tokens accurately.

- **Tangible Deliverable:** `src/pymastery/sampler.py` and `tests/test_sampler.py`
- **STOP WHEN:** `temperature=0.0` is strictly greedy; `top_k=1` matches greedy; `top_p=0.5` zeroes out tail tokens accurately.
- **Do Not Study Yet:** Grammar-constrained FSM decoding.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 160: Implementing Byte-Pair Encoding (BPE) Tokenizer (`encode` & `decode`) from scratch CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a clean BPE tokenizer that applies merge rules by priority rank, handles special tokens (`<|bos|>`, `<|eos|>`), and decodes token IDs back to UTF-8 strings.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day160/`, and review the mapped book section for `Implementing Byte-Pair Encoding (BPE) Tokenizer (`encode `&` decode`) from scratch`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `BPETokenizer` in `src/pymastery/bpe_tokenizer.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing Byte-Pair Encoding (BPE) Tokenizer (`encode `&` decode`) from scratch` and save the test log beside `src/pymastery/bpe_tokenizer.py` and `tests/test_bpe_tokenizer.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing Byte-Pair Encoding (BPE) Tokenizer (`encode `&` decode`) from scratch`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day160/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** `tokenizer.decode(tokenizer.encode(text)) == text` holds across ASCII, whitespace, and multi-byte UTF-8 inputs.

- **Tangible Deliverable:** `src/pymastery/bpe_tokenizer.py` and `tests/test_bpe_tokenizer.py`
- **STOP WHEN:** `tokenizer.decode(tokenizer.encode(text)) == text` holds across ASCII, whitespace, and multi-byte UTF-8 inputs.
- **Do Not Study Yet:** Distributed corpus BPE training.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 161: Week 23 Gate: Complete Pure-Python Transformer Inference Engine CORE: Advanced Track (Months 5-8)

MINOR: Analytical vs numerical finite-difference gradient check and KV-cache equivalence ¶

> **The Big Picture:** Understand every line of how PyTorch and LLM forward/backward passes work by implementing a reverse-mode automatic differentiation engine and complete KV-cached Transformer in Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Wire `BPETokenizer`, `CachedTransformerBlock` (RoPE + GQA + KV Cache + SwiGLU + RMSNorm), and `Sampler` into a self-contained LLM forward pass.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Analytical vs numerical finite-difference gradient check and KV-cache equivalence — because writing a Transformer, RoPE, GQA, and KV-cache from scratch cements total mastery of both Python and LLM mechanics.
- **Mapped Book & Docs:** Sebastian Raschka, Build a Large Language Model (From Scratch) (Manning) Ch. 3, 4

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day161/`, and review the mapped book section for `Week 23 Gate: Complete Pure-Python Transformer Inference Engine`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run end-to-end prompt prefill and 16-token autoregressive generation with KV-cache verification and per-step timing.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 23 Gate: Complete Pure-Python Transformer Inference Engine` and save the test log beside `gates/week23-gate.md` and `src/pymastery/nano_llm.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Analytical vs numerical finite-difference gradient check and KV-cache equivalence: Verify every operator and attention layer against numerical invariants and confirm KV-cached decode matches full-sequence forward pass to `1e-5`.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 23 Gate: Complete Pure-Python Transformer Inference Engine`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day161/failure-notes.md`.

> **Engineering Trap to Avoid:** Never forget to apply the causal mask `-inf` *before* `softmax` (and only to future positions `j > i + past_len`).

> **Daily Gate (STOP WHEN):** End-to-end generation passes equivalence, causal-masking, and determinism tests.

- **Tangible Deliverable:** `gates/week23-gate.md` and `src/pymastery/nano_llm.py`
- **STOP WHEN:** End-to-end generation passes equivalence, causal-masking, and determinism tests.
- **Do Not Study Yet:** Multi-GPU pipeline parallelism.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 24 - Month 6 Milestone: Paged KV-Cache Block Manager & Continuous Batching Simulation¶

Day 162: Logical vs Physical KV Blocks (`block_size=16`) & the Block Table Allocator CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Eliminate external and internal KV-cache memory fragmentation by mapping each sequence's logical token blocks to non-contiguous physical block IDs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is

written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.

- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day162/`, and review the mapped book section for `Logical vs Physical KV Blocks (block_size=16) & the Block Table Allocator`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `PhysicalBlockAllocator` and `BlockTable` in `src/pymastery/paged_kv/allocator.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Logical vs Physical KV Blocks (block_size=16) & the Block Table Allocator` and save the test log beside `src/pymastery/paged_kv/allocator.py` and `tests/test_block_allocator.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Logical vs Physical KV Blocks (block_size=16) & the Block Table Allocator`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day162/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Allocates new physical blocks only when a sequence crosses a `block_size` boundary and frees all blocks on sequence completion.

- **Tangible Deliverable:** `src/pymastery/paged_kv/allocator.py` and `tests/test_block_allocator.py`
- **STOP WHEN:** Allocates new physical blocks only when a sequence crosses a `block_size` boundary and frees all blocks on sequence completion.
- **Do Not Study Yet:** CUDA block-table gather kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 163: Copy-on-Write (CoW) Block Sharing for Parallel Sampling (`n > 1`) & Shared System Prompts CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Share physical KV blocks across multiple sequences (`ref_count > 1`) and perform Copy-on-Write allocation only when a shared block is mutated in-place.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.
- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day163/`, and review the mapped book section for `Copy-on-Write (CoW) Block Sharing for Parallel Sampling (n > 1) & Shared System Prompts`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Add `fork_sequence` and `append_token_slot` with Copy-on-Write semantics to `src/pymastery/paged_kv/allocator.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Copy-on-Write (CoW) Block Sharing for Parallel Sampling (n > 1) & Shared System Prompts` and save the test log beside `src/pymastery/paged_kv/allocator.py` and `tests/test_cow_blocks.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Copy-on-Write (CoW) Block Sharing for Parallel Sampling (n > 1) & Shared System Prompts`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day163/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Forking a 64-token prompt across 4 sequences allocates 0 new physical blocks until decode diverges into shared tail blocks.

- **Tangible Deliverable:** `src/pymastery/paged_kv/allocator.py` and `tests/test_cow_blocks.py`
- **STOP WHEN:** Forking a 64-token prompt across 4 sequences allocates 0 new physical blocks until decode diverges into shared tail blocks.
- **Do Not Study Yet:** Distributed KV migration.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 164: Automatic Prefix Caching (APC) with Hash-Chained Physical Blocks & Eviction CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Hash `(parent_block_hash, token_tuple)` for full blocks so new requests automatically reuse cached physical KV blocks from earlier completed requests.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.
- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day164/`, and review the mapped book section for `Automatic Prefix Caching (APC) with Hash-Chained Physical Blocks & Eviction`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `PrefixCachingBlockManager` in `src/pymastery/paged_kv/prefix_cache.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Automatic Prefix Caching (APC) with Hash-Chained Physical Blocks & Eviction` and save the test log beside `src/pymastery/paged_kv/prefix_cache.py` and `tests/test_prefix_cache_blocks.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Automatic Prefix Caching (APC) with Hash-Chained Physical Blocks & Eviction`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day164/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Repeated system prompts achieve 100% prefix block reuse and unreferenced cached blocks evict in LRU order when free blocks run out.

- **Tangible Deliverable:** `src/pymastery/paged_kv/prefix_cache.py` and `tests/test_prefix_cache_blocks.py`
- **STOP WHEN:** Repeated system prompts achieve 100% prefix block reuse and unreferenced cached blocks evict in LRU order when free blocks run out.
- **Do Not Study Yet:** Disk SSD KV offloading.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 165: Continuous Batching Scheduler: `WAITING`, `RUNNING`, `SWAPPED` queues & Preemption CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement vLLM's core iteration-level scheduling algorithm: scheduling prefill and decode in one step and preempting (recomputing or swapping) lowest-priority sequences when KV blocks run out mid-decode.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.

- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day165/`, and review the mapped book section for `Continuous Batching Scheduler: WAITING, RUNNING, SWAPPED queues & Preemption`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ContinuousBatchScheduler` in `src/pymastery/paged_kv/scheduler.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Continuous Batching Scheduler: WAITING, RUNNING, SWAPPED queues & Preemption` and save the test log beside `src/pymastery/paged_kv/scheduler.py` and `tests/test_continuous_scheduler.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Continuous Batching Scheduler: WAITING, RUNNING, SWAPPED queues & Preemption`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day165/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Under memory pressure, the scheduler preempts the newest running sequence safely instead of crashing with OOM, and resumes it once blocks free up.

- **Tangible Deliverable:** `src/pymastery/paged_kv/scheduler.py` and `tests/test_continuous_scheduler.py`
- **STOP WHEN:** Under memory pressure, the scheduler preempts the newest running sequence safely instead of crashing with OOM, and resumes it once blocks free up.
- **Do Not Study Yet:** Multi-step speculative decoding.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 166: Chunked Prefill & Token-Budget Scheduling (`max_num_batched_tokens`) CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Prevent long prompts (e.g., 4,096 tokens) from stalling ongoing decode requests (head-of-line blocking) by splitting large prefills into bounded chunks combined with active decode tokens.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.
- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day166/`, and review the mapped book section for `Chunked Prefill & Token-Budget Scheduling (`max_num_batched_tokens`)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Add chunked prefill budgeting to `src/pymastery/paged_kv/scheduler.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Chunked Prefill & Token-Budget Scheduling (`max_num_batched_tokens`)` and save the test log beside `src/pymastery/paged_kv/scheduler.py` and `tests/test_chunked_prefill.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Chunked Prefill & Token-Budget Scheduling (`max_num_batched_tokens`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day166/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Interleaves a 512-token prefill in 128-token chunks alongside active decodes, keeping decode ITL spikes bounded.

- **Tangible Deliverable:** `src/pymastery/paged_kv/scheduler.py` and `tests/test_chunked_prefill.py`
- **STOP WHEN:** Interleaves a 512-token prefill in 128-token chunks alongside active decodes, keeping decode ITL spikes bounded.
- **Do Not Study Yet:** Disaggregated prefill clusters.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 167: Speculative Decoding Draft-and-Verify Loop in Python

CORE: Advanced Track (Months 5-8) MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the speculative decoding verification algorithm (Draft model proposes K tokens, Target model scores all $K+1$ positions in a single forward pass, rejection sampling accepts prefix and resamples at first mismatch).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.
- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day167/`, and review the mapped book section for `Speculative Decoding Draft-and-Verify Loop in Python`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `speculative_verify_step` in `src/pymastery/paged_kv/speculative.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Speculative Decoding Draft-and-Verify Loop in Python` and save the test log beside `src/pymastery/paged_kv/speculative.py` and `tests/test_speculative.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Speculative Decoding Draft-and-Verify Loop in Python`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day167/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** Guarantees always generating >= 1 token per target forward pass and up to $K+1$ tokens when draft matches target.

- **Tangible Deliverable:** `src/pymastery/paged_kv/speculative.py` and `tests/test_speculative.py`
- **STOP WHEN:** Guarantees always generating >= 1 token per target forward pass and up to $K+1$ tokens when draft matches target.
- **Do Not Study Yet:** Medusa tree attention kernels.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 168: Week 24 Gate (Month 6 Review): Memory, Tensors, Autograd & Paged KV Mastery Gate CORE: Advanced Track (Months 5-8)

MINOR: Zero block leaks, reference count invariants, and memory utilization efficiency ¶

> **The Big Picture:** Bridge pure Python systems engineering with modern LLM serving algorithms by implementing PagedAttention's logical-to-physical block table, Copy-on-Write beam/prefix sharing, and preemption.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Verify all Month 6 deliverables: zero-copy `memoryview` arenas, `StridedTensor`, `AutoTensor`, `NanoLLM`, and `ContinuousBatchScheduler` with Paged KV blocks.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero block leaks, reference count invariants, and memory utilization efficiency — because PagedAttention's block manager in vLLM is written in Python—implementing it yourself makes you ready to read and modify vLLM's core scheduler.

- **Mapped Book & Docs:** vLLM PagedAttention Paper (Kwon et al., SOSP 2023) & High Performance Python, 2nd Ed. Ch. 6, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day168/`, and review the mapped book section for `Week 24 Gate (Month 6 Review): Memory, Tensors, Autograd & Paged KV Mastery Gate`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a 1,000-step continuous batching simulation with random prompt/output lengths and assert 100% block accounting conservation.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 24 Gate (Month 6 Review): Memory, Tensors, Autograd & Paged KV Mastery Gate` and save the test log beside `gates/month06-gate.md` and `tests/test_month06_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero block leaks, reference count invariants, and memory utilization efficiency: Assert `num_free_blocks + num_allocated_blocks == total_blocks` after every single scheduler step.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 24 Gate (Month 6 Review): Memory, Tensors, Autograd & Paged KV Mastery Gate`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day168/failure-notes.md`.

> **Engineering Trap to Avoid:** Never free a physical KV block whose reference count is still `> 0` across shared prefixes.

> **Daily Gate (STOP WHEN):** All 24 weeks of tests pass and block conservation invariant holds across 1,000 randomized steps.

- **Tangible Deliverable:** `gates/month06-gate.md` and `tests/test_month06_gate.py`
- **STOP WHEN:** All 24 weeks of tests pass and block conservation invariant holds across 1,000 randomized steps.
- **Do Not Study Yet:** C/C++ compiler extensions (starts tomorrow in Month 7!).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 7: Profiling, Roofline, Free-Threaded Python & Native C/C++/GPU Interop (Weeks 25–28, Days 169–196)¶

Week 25 - Deterministic & Statistical Profiling, Flamegraphs & Roofline Optimization¶

Day 169: Microbenchmarking rigor: `time.perf_counter_ns`, `timeit`, warm-up, GC isolation & confidence intervals CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a statistically rigorous Python microbenchmark harness that controls warm-up, records GC state, measures median/IQR/p95 in nanoseconds, and detects noisy runs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day169/`, and review the mapped book section for `Microbenchmarking rigor: time.perf_counter_ns, timeit, warm-up, GC isolation & confidence intervals`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MicroBench` in `src/pymastery/profiling/bench_harness.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Microbenchmarking rigor: time.perf_counter_ns, timeit, warm-up, GC isolation & confidence intervals` and save the test log beside `src/pymastery/profiling/bench_harness.py` and `tests/test_bench_harness.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Microbenchmarking rigor: time.perf_counter_ns, timeit, warm-up, GC isolation & confidence intervals`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day169/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Produces reproducible nanosecond/microsecond timing comparisons with relative speedup ratios.

- **Tangible Deliverable:** `src/pymastery/profiling/bench_harness.py` and `tests/test_bench_harness.py`
- **STOP WHEN:** Produces reproducible nanosecond/microsecond timing comparisons with relative speedup ratios.
- **Do Not Study Yet:** Hardware performance counters (`perf_event_open`).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 170: Deterministic profiling with `cProfile` & programmatic bottleneck ranking with `pstats` CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Capture deterministic call profiles around specific code regions (`cProfile.Profile()`) and query `pstats.Stats` programmatically by `cumtime` and `tottime`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day170/`, and review the mapped book section for `Deterministic profiling with` cProfile `& programmatic bottleneck ranking with` pstats``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `profile_region` context manager and bottleneck extractor in `src/pymastery/profiling/deterministic_profiler.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Deterministic profiling with` cProfile `& programmatic bottleneck ranking with` pstats` `and save the test log beside` src/pymastery/profiling/deterministic_profiler.py `and` tests/test_deterministic_profiler.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Deterministic profiling with` cProfile `& programmatic bottleneck ranking with` pstats`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day170/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Automatically identifies the top-3 functions by exclusive (`tottime`) and inclusive (`cumtime`) time.

- **Tangible Deliverable:** `src/pymastery/profiling/deterministic_profiler.py` and `tests/test_deterministic_profiler.py`
- **STOP WHEN:** Automatically identifies the top-3 functions by exclusive (`tottime`) and inclusive (`cumtime`) time.
- **Do Not Study Yet:** Kernel ftrace.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 171: Building a Statistical Stack-Sampling Profiler & Folded Flamegraph Generator CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how low-overhead production profilers (`py-spy`, `scalene`) work by sampling `sys._current_frames()` at regular intervals and emitting `.folded` stack traces for flamegraphs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.

- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day171/`, and review the mapped book section for `Building a Statistical Stack-Sampling Profiler & Folded Flamegraph Generator`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `SamplingFlamegraphProfiler` in `src/pymastery/profiling/flamegraph.py` that outputs standard Brendan Gregg folded stack format (`main;scheduler;step 42`) and an interactive SVG/HTML bar view.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Building a Statistical Stack-Sampling Profiler & Folded Flamegraph Generator` and save the test log beside `src/pymastery/profiling/flamegraph.py` and `tests/test_flamegraph.py`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Building a Statistical Stack-Sampling Profiler & Folded Flamegraph Generator`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day171/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Generates valid folded stack traces and attributes >80% of samples to the known hot function.

- **Tangible Deliverable:** `src/pymastery/profiling/flamegraph.py` and `tests/test_flamegraph.py`
- **STOP WHEN:** Generates valid folded stack traces and attributes >80% of samples to the known hot function.
- **Do Not Study Yet:** ptrace out-of-process attach.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 172: Joint CPU + Memory Allocation Profiling (Scalene-style line attribution) CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Correlate CPU time with bytes allocated per line (`tracemalloc` + line timings) to spot functions that bottleneck on allocator churn.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day172/`, and review the mapped book section for `Joint CPU + Memory Allocation Profiling (Scalene-style line attribution)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AllocationHotspotAnalyzer` in `src/pymastery/profiling/alloc_profiler.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Joint CPU + Memory Allocation Profiling (Scalene-style line attribution)` and save the test log beside `src/pymastery/profiling/alloc_profiler.py` and `tests/test_alloc_profiler.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Joint CPU + Memory Allocation Profiling (Scalene-style line attribution)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day172/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Pinpoints a hot loop creating temporary lists/dicts and quantifies the exact MB/sec allocation rate.

- **Tangible Deliverable:** `src/pymastery/profiling/alloc_profiler.py` and `tests/test_alloc_profiler.py`
- **STOP WHEN:** Pinpoints a hot loop creating temporary lists/dicts and quantifies the exact MB/sec allocation rate.
- **Do Not Study Yet:** Jemalloc heap dumps.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 173: Roofline Analysis for Python & ML Workloads: Compute-Bound vs Memory-Bandwidth-Bound vs Overhead-Bound CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Classify workloads into three regimes: Python Interpreter Overhead-Bound, Memory-Bandwidth-Bound (e.g., LLM decode), or Compute-Bound (e.g., LLM prefill), and compute Arithmetic Intensity (`FLOPs / Bytes`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day173/`, and review the mapped book section for `Roofline Analysis for Python & ML Workloads: Compute-Bound vs Memory-Bandwidth-Bound vs Overhead-Bound`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `RooflineModel` calculator and workload classifier in `src/pymastery/profiling/roofline.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Roofline Analysis for Python & ML Workloads: Compute-Bound vs Memory-Bandwidth-Bound vs Overhead-Bound` and save the test log beside `src/pymastery/profiling/roofline.py` and `tests/test_roofline.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Roofline Analysis for Python & ML Workloads: Compute-Bound vs Memory-`

`Bandwidth-Bound vs Overhead-Bound`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day173/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Accurately calculates ridge point (`Peak FLOP/s / Peak GB/s`), arithmetic intensity for batch size B, and bound classification.

- **Tangible Deliverable:** `src/pymastery/profiling/roofline.py` and `tests/test_roofline.py`
- **STOP WHEN:** Accurately calculates ridge point (`Peak FLOP/s / Peak GB/s`), arithmetic intensity for batch size B, and bound classification.
- **Do Not Study Yet:** Nsight Compute warp stall reasons.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 174: Systematic Pure-Python Optimization Case Study (3x–10x Speedup on `StridedTensor` & `BPETokenizer`) CORE: Advanced Track (Months 5-8)

MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Apply profiling evidence to optimize `BPETokenizer` and `StridedTensor` (hoisting loop invariants, pre-computing lookup tables, replacing `O(N^2)` scans with priority queues/dicts, local variable binding).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day174/`, and review the mapped book section for `Systematic Pure-Python Optimization Case Study (3x–10x Speedup on` StridedTensor `&` BPETokenizer`)`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Profile and optimize `BPETokenizer.encode` and `StridedTensor` ops, recording before/after metrics in `artifacts/day174-optimization.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Systematic Pure-Python Optimization Case Study (3x–10x Speedup on` StridedTensor `&` BPETokenizer`)` and save the test log beside `src/pymastery/bpe_tokenizer.py`, `tests/test_bpe_perf.py`, and `artifacts/day174-optimization.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Systematic Pure-Python Optimization Case Study (3x–10x Speedup on` StridedTensor `&` BPETokenizer`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day174/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Achieves >= 4x measured speedup on 10 KB text tokenization while passing 100% of existing correctness tests.

- **Tangible Deliverable:** `src/pymastery/bpe_tokenizer.py`, `tests/test_bpe_perf.py`, and `artifacts/day174-optimization.md`
- **STOP WHEN:** Achieves >= 4x measured speedup on 10 KB text tokenization while passing 100% of existing correctness tests.
- **Do Not Study Yet:** Writing C code (covered next week!).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 175: Week 25 Gate: Profiling & Optimization Evidence Portfolio

CORE: Advanced Track (Months 5-8) **MINOR: Before/after benchmark speedup with 5+ warmup-excluded repetitions** ¶

> **The Big Picture:** Replace performance guesswork with scientific profiling (`cProfile`, `pstats`, `time.perf_counter_ns`, statistical stack sampling, memory-vs-CPU attribution) and targeted optimization.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Demonstrate end-to-end profile -> hypothesis -> optimization -> verification workflow on your continuous batching and tensor engine.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Before/after benchmark speedup with 5+ warmup-excluded repetitions — because optimizing without a profile is the #1 way engineers waste weeks speeding up a function that takes 0.5% of runtime.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. (Gorelick & Ozsvald, O'Reilly) Ch. 1, 2, 7

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day175/`, and review the mapped book section for `Week 25 Gate: Profiling & Optimization Evidence Portfolio`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Produce flamegraph stack output, allocation diff, roofline table, and verified speedup report.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 25 Gate: Profiling & Optimization Evidence Portfolio` and save the test log beside `gates/week25-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Before/after benchmark speedup with 5+ warmup-excluded repetitions: Every optimization must record baseline vs optimized p50/p95 runtime and pass regression correctness tests.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 25 Gate: Profiling & Optimization Evidence Portfolio`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day175/failure-notes.md`.

> **Engineering Trap to Avoid:** Never optimize code before capturing a baseline profile and locking in a correctness test!

> **Daily Gate (STOP WHEN):** Every optimization claim is backed by 5-run before/after measurements and zero regressions.

- **Tangible Deliverable:** `gates/week25-gate.md`
- **STOP WHEN:** Every optimization claim is backed by 5-run before/after measurements and zero regressions.
- **Do Not Study Yet:** Unmeasured micro-tweaks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 26 - The GIL, Free-Threaded Python (PEP 703) & Native C Interop (`ctypes`, `cffi`)¶

Day 176: Inside the GIL (`eval_breaker`, switch interval `sys.getswitchinterval()`) & Thread Contention CORE: Advanced Track (Months 5-8) [MINOR: Pointer safety, explicit argtypes/restype, and zero-copy buffer passing]¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand why CPU-bound Python threads contend on the GIL, how `sys.setswitchinterval()` affects context-switch overhead, and when C extensions release the GIL (`Py_BEGIN_ALLOW_THREADS`).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day176/`, and review the mapped book section for `Inside the GIL (`eval_breaker`, switch interval` sys.getswitchinterval()`) & Thread Contention`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write `src/pymastery/native/gil_analysis.py` measuring thread contention across 1, 2, 4, and 8 threads for pure-Python math vs GIL-releasing native calls.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Inside the GIL (`eval_breaker`, switch interval` sys.getswitchinterval()`) & Thread Contention` and save the test log beside `src/pymastery/native/gil_analysis.py` and `tests/test_gil_analysis.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Inside the GIL (`eval_breaker`, switch interval` sys.getswitchinterval()`) & Thread Contention`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day176/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** Demonstrates why GIL-releasing calls scale linearly across cores while pure-Python loops serialize on the GIL.

- **Tangible Deliverable:** `src/pymastery/native/gil_analysis.py` and `tests/test_gil_analysis.py`
- **STOP WHEN:** Demonstrates why GIL-releasing calls scale linearly across cores while pure-Python loops serialize on the GIL.
- **Do Not Study Yet:** Kernel scheduler patches.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 177: Free-Threaded Python (PEP 703 / Python 3.13t/3.14t Py_GIL_DISABLED) & Thread-Safe Data Design CORE: Advanced Track (Months 5-8) [MINOR: Pointer safety, explicit argtypes/restype, and zero-copy buffer passing]¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write forward-compatible concurrent Python that runs safely under both standard GIL builds and Free-Threaded (`no-gil`) builds: checking `sys._is_gil_enabled()`, avoiding unprotected shared dict/list mutation races, and using fine-grained locks.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day177/`, and review the mapped book section for `Free-Threaded Python (PEP 703 / Python 3.13t/3.14t Py_GIL_DISABLED) & Thread-Safe Data Design`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement thread-safe sharded counters and lock-striped caches in `src/pymastery/native/freethreaded_safe.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Free-Threaded Python (PEP 703 / Python 3.13t/3.14t Py_GIL_DISABLED) & Thread-Safe Data Design` and save the test log beside `src/pymastery/native/freethreaded_safe.py` and `tests/test_freethreaded_safe.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Free-Threaded Python (PEP 703 / Python 3.13t/3.14t Py_GIL_DISABLED) & Thread-Safe Data Design`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day177/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** 16 concurrent threads mutating the 16-shard cache pass 100% invariant checks under high contention.

- **Tangible Deliverable:** `src/pymastery/native/freethreaded_safe.py` and `tests/test_freethreaded_safe.py`
- **STOP WHEN:** 16 concurrent threads mutating the 16-shard cache pass 100% invariant checks under high contention.
- **Do Not Study Yet:** CPython biased reference counting C patches.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 178: `ctypes` fundamentals: loading shared libraries, primitive types, `argtypes`, `restype` & `Structure` CORE: Advanced Track (Months 5-8) `[MINOR: Pointer safety, explicit` argtypes/restype`, and zero-copy buffer passing]`¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Use Python's built-in `ctypes` to define C structs (`class CensorHeader(ctypes.Structure): _fields_ = [...]`), pointers (`POINTER(c_float)`), and safe C library bindings.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.

- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day178/`, and review the mapped book section for `ctypes fundamentals: loading shared libraries, primitive types, `argtypes`, `restype` & `Structure`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/native/ctypes_structs.py` demonstrating C struct memory layout, alignment (`ctypes.sizeof`, `alignment`), and safe C standard library calls.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `ctypes fundamentals: loading shared libraries, primitive types, `argtypes`, `restype` & `Structure and save the test log beside` src/pymastery/native/ctypes_structs.py `and` tests/test_ctypes_structs.py`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `ctypes fundamentals: loading shared libraries, primitive types, `argtypes`, `restype` & `Structure, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day178/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** C struct layout matches `struct.calcsize` and all C calls have explicit `argtypes` and `restype`.

- **Tangible Deliverable:** `src/pymastery/native/ctypes_structs.py` and `tests/test_ctypes_structs.py`
- **STOP WHEN:** C struct layout matches `struct.calcsize` and all C calls have explicit `argtypes` and `restype`.
- **Do Not Study Yet:** COM Windows automation.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 179: Zero-copy pointer extraction from `bytearray` / `array('f')` / `memoryview` into `ctypes` CORE: Advanced Track (Months 5-8) `[MINOR: Pointer safety, explicit` argtypes/restype`, and zero-copy buffer passing]`¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Pass contiguous Python buffers directly to C functions as `float*` (`((ctypes.c_float * n).from_buffer(mv))`) without copying a single byte.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day179/`, and review the mapped book section for `Zero-copy pointer extraction from` bytearray `/` array('f') `/` memoryview `into` ctypes``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `zero_copy_float_ptr(mv)` in `src/pymastery/native/zero_copy_ffi.py` with lifetime pinning and read-only buffer checks.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Zero-copy pointer extraction from` bytearray `/` array('f') `/` memoryview `into` ctypes` `and save the test log beside` src/pymastery/native/zero_copy_ffi.py `and` tests/test_zero_copy_ffi.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Zero-copy pointer extraction from` bytearray `/` array('f') `/` memoryview `into` ctypes`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day179/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** Mutating the array through the `ctypes` pointer immediately updates the Python `array('f')` in place.

- **Tangible Deliverable:** `src/pymastery/native/zero_copy_ffi.py` and `tests/test_zero_copy_ffi.py`
- **STOP WHEN:** Mutating the array through the `ctypes` pointer immediately updates the Python `array('f')` in place.
- **Do Not Study Yet:** JNI Java bridges.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 180: Writing, compiling & binding a custom C kernel (`fast_ops.c`) from Python CORE: Advanced Track (Months 5-8) [MINOR: Pointer safety, explicit argtypes/restype, and zero-copy buffer passing]¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Write a clean C99 source file implementing vectorized/SIMD-friendly `rmsnorm_f32`, `softmax_f32`, and `matmul_f32`, compile it to a shared library (`.dll`/`.so`) or load via C compiler/fallback, and wrap it in a typed Python API.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day180/`, and review the mapped book section for `Writing, compiling & binding a custom C kernel (`fast_ops.c`) from Python`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `src/pymastery/native/fast_ops.c` and `src/pymastery/native/fast_ops_wrapper.py` (with automatic C compiler detection + verified pure-Python/ctypes fallback if no C compiler is on PATH).
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Writing, compiling & binding a custom C kernel (`fast_ops.c`) from Python` and save the test log beside `src/pymastery/native/fast_ops.c`, `fast_ops_wrapper.py`, and `tests/test_fast_ops.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Writing, compiling & binding a custom C kernel (`fast_ops.c`) from Python`, verify

that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day180/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** Python wrapper validates tensor shapes, contiguity, and dtypes before invoking the C kernel, matching reference results to `1e-5`.

- **Tangible Deliverable:** `src/pymastery/native/fast_ops.c`, `fast_ops_wrapper.py`, and `tests/test_fast_ops.py`
- **STOP WHEN:** Python wrapper validates tensor shapes, contiguity, and dtypes before invoking the C kernel, matching reference results to `1e-5`.
- **Do Not Study Yet:** Inline x86 assembly.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 181: Releasing the GIL across native calls & parallelizing C kernels with `ThreadPoolExecutor` CORE: Advanced Track (Months 5-8) `[MINOR: Pointer safety, explicit` argtypes/restype`, and zero-copy buffer passing]`¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Execute native C/FFI functions across multiple threads in `ThreadPoolExecutor` to achieve true multi-core parallelism from Python.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day181/`, and review the mapped book section for `Releasing the GIL across native calls & parallelizing C kernels with` ThreadPoolExecutor``.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement parallel batch matrix-vector multiplication in `src/pymastery/native/parallel_native.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Releasing the GIL across native calls & parallelizing C kernels with` ThreadPoolExecutor` `and save the test log beside` src/pymastery/native/parallel_native.py `and` tests/test_parallel_native.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Releasing the GIL across native calls & parallelizing C kernels with` ThreadPoolExecutor`, verify that` pytest `or` mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day181/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** Multi-threaded execution completes deterministically and matches single-threaded outputs.

- **Tangible Deliverable:** `src/pymastery/native/parallel_native.py` and `tests/test_parallel_native.py`
- **STOP WHEN:** Multi-threaded execution completes deterministically and matches single-threaded outputs.
- **Do Not Study Yet:** OpenMP runtime conflicts.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 182: Week 26 Gate: Zero-Copy Native Kernel Acceleration Suite

CORE: Advanced Track (Months 5-8) `[MINOR: Pointer safety, explicit` **argtypes/restype**`, and zero-copy buffer passing]`¶

> **The Big Picture:** Master how CPython's Global Interpreter Lock (GIL) works, how Python 3.13+ Free-Threading (`--disable-gil`) changes concurrency, and how to call native C shared libraries with zero-copy pointers using `ctypes`.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Plug `fast_ops_wrapper` into your `NanoLLM` Transformer (`RMSNorm`, `Softmax`, `MatMul`) and benchmark the speedup.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing — because Python achieves C/CUDA speed in production by passing raw buffer memory pointers across the FFI boundary.
- **Mapped Book & Docs:** Fluent Python, 2nd Ed. Ch. 19 & High Performance Python, 2nd Ed. Ch. 8, 9

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day182/`, and review the mapped book section for `Week 26 Gate: Zero-Copy Native Kernel Acceleration Suite`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run the Week 23 Transformer test suite with the native C/accelerated backend enabled and compare latency.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 26 Gate: Zero-Copy Native Kernel Acceleration Suite` and save the test log beside `gates/week26-gate.md` and `tests/test_week26_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Pointer safety, explicit `argtypes`/`restype`, and zero-copy buffer passing: Always declare `func.argtypes` and `func.restype` on every `ctypes` function—omitting them causes 64-bit pointer truncation segfaults!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 26 Gate: Zero-Copy Native Kernel Acceleration Suite`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day182/failure-notes.md`.

> **Engineering Trap to Avoid:** Never let a Python garbage-collected buffer be freed while a C function still holds a raw pointer to its memory!

> **Daily Gate (STOP WHEN):** 100% numerical parity with pure-Python baseline and documented latency improvement.

- **Tangible Deliverable:** `gates/week26-gate.md` and `tests/test_week26_gate.py`
- **STOP WHEN:** 100% numerical parity with pure-Python baseline and documented latency improvement.
- **Do Not Study Yet:** CUDA driver C API.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 27 - C++/pybind11/nanobind Architecture, Custom PyTorch Operators, Triton DSL & DLPack¶

Day 183: The DLPack Protocol (__dlpack__, __dlpack_device__) for zero-copy cross-framework tensor exchange CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how PyTorch, NumPy, CuPy, JAX, and vLLM exchange tensors with zero memory copies using the DLPack C-struct capsule specification.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day183/`, and review the mapped book section for `The DLPack Protocol (__dlpack__, __dlpack_device__) for zero-copy cross-framework tensor exchange`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement DLPack protocol support and a zero-copy tensor bridge in `src/pymastery/gpu_interop/dlpack_bridge.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `The DLPack Protocol (__dlpack__, __dlpack_device__) for zero-copy cross-framework tensor exchange` and save the test log beside `src/pymastery/gpu_interop/dlpack_bridge.py` and `tests/test_dlpack_bridge.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `The DLPack Protocol (__dlpack__, __dlpack_device__) for zero-copy cross-framework tensor exchange`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day183/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Exchanges contiguous tensor buffers across wrapper boundaries with identical memory addresses and device tags.

- **Tangible Deliverable:** `src/pymastery/gpu_interop/dlpack_bridge.py` and `tests/test_dlpack_bridge.py`
- **STOP WHEN:** Exchanges contiguous tensor buffers across wrapper boundaries with identical memory addresses and device tags.
- **Do Not Study Yet:** Legacy `PyArrayInterface`.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 184: How `pybind11` & `nanobind` bind C++/CUDA classes and functions to Python (`PYBIND11_MODULE`) CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]`¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Master the architecture of `pybind11`/`nanobind` bindings used in vLLM (`csrc/torch_bindings.cpp`): type casters, `nb::ndarray`, `py::gil_scoped_release`, and `setup.py`/`pyproject.toml` build extensions.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day184/`, and review the mapped book section for `How pybind11 & nanobind bind C++/CUDA classes and functions to Python (PYBIND11_MODULE)`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Write a complete, documented `csrc/paged_attention_binding.cpp` reference binding and Python C-extension stub simulator in `src/pymastery/gpu_interop/cpp_binding_arch.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `How` pybind11 `&` nanobind `bind C++/CUDA classes and functions to Python (`PYBIND11_MODULE`)` and save the test log beside `src/pymastery/gpu_interop/cpp_binding_arch.py` and `tests/test_cpp_binding_arch.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `How` pybind11 `&` nanobind `bind C++/CUDA classes and functions to Python (`PYBIND11_MODULE`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day184/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Validates device placement, contiguity, stride alignment, and dtype contracts before dispatching to native kernels.

- **Tangible Deliverable:** `src/pymastery/gpu_interop/cpp_binding_arch.py` and `tests/test_cpp_binding_arch.py`
- **STOP WHEN:** Validates device placement, contiguity, stride alignment, and dtype contracts before dispatching to native kernels.
- **Do Not Study Yet:** Boost.Python legacy macros.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 185: Custom Operator Registration & Fake/Meta Tensors (`register_op` + `register_fake` for `torch.compile`) CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts]¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand why `torch.compile` and symbolic tracers need a `register_fake` (meta-tensor shape/dtype propagation function) alongside every custom C++/CUDA/Triton kernel.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day185/`, and review the mapped book section for `Custom Operator Registration & Fake/Meta Tensors (`register_op `+` register_fake `for` torch.compile`)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement an extensible `CustomOpRegistry` with `MetaTensor` (shape/dtype/device-only, 0 storage bytes) verification in `src/pymastery/gpu_interop/custom_op_registry.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Custom Operator Registration & Fake/Meta Tensors (`register_op `+` register_fake `for` torch.compile`)` and save the test log beside `src/pymastery/gpu_interop/custom_op_registry.py` and `tests/test_custom_op_registry.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Custom Operator Registration & Fake/Meta Tensors (`register_op `+` register_fake `for` torch.compile`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day185/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Propagates shapes and dtypes through a 12-layer 70B-parameter model graph in < 1ms using 0 MB of weight memory via `MetaTensor`s!

- **Tangible Deliverable:** `src/pymastery/gpu_interop/custom_op_registry.py` and `tests/test_custom_op_registry.py`
- **STOP WHEN:** Propagates shapes and dtypes through a 12-layer 70B-parameter model graph in < 1ms using 0 MB of weight memory via `MetaTensor`s!
- **Do Not Study Yet:** TorchScript legacy JIT.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 186: How `@triton.jit` works in Python: AST inspection, `tl.program_id`, block pointers, masking & autotuning CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]`¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand Triton as a Python-embedded DSL: how `@triton.jit` parses the Python AST, specializes on `tl.constexpr`, computes `pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)`, and applies boundary masks.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day186/`, and review the mapped book section for `How` @triton.jit `works in Python: AST inspection,` tl.program_id`, block pointers, masking & autotuning`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement a pure-Python `TritonSPMDSimulator` (`program_id`, `arange`, `load(ptr, mask, other)`, `store`) and run fused `add_rmsnorm` and `online_softmax` SPMD kernels in `src/pymastery/gpu_interop/triton_spmd_sim.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `How` @triton.jit `works in Python: AST inspection,` tl.program_id`, block pointers, masking &`

`autotuning` and save the test log beside `src/pymastery/gpu_interop/triton_spmd_sim.py` and `tests/test_triton_spmd_sim.py`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `How` @triton.jit `works in Python: AST inspection,` tl.program_id`, block pointers, masking & autotuning`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day186/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Executes 1D and 2D tiled SPMD programs across non-multiple-of-`BLOCK_SIZE` lengths without out-of-bounds reads/writes.

- **Tangible Deliverable:** `src/pymastery/gpu_interop/triton_spmd_sim.py` and `tests/test_triton_spmd_sim.py`
- **STOP WHEN:** Executes 1D and 2D tiled SPMD programs across non-multiple-of-`BLOCK_SIZE` lengths without out-of-bounds reads/writes.
- **Do Not Study Yet:** Raw PTX register assembly.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 187: Operator Fusion & Online Softmax (FlashAttention tiling math in Python) CORE: Advanced Track (Months 5-8) `[MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]`¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the 1-pass Online Softmax and tiled FlashAttention algorithm (maintaining running row max m_i and running denominator l_i across KV tiles without materializing the $N \times N$ attention matrix).

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day187/`, and review the mapped book section for `Operator Fusion & Online Softmax (FlashAttention tiling math in Python)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `tiled_flash_attention_forward` in `src/pymastery/gpu_interop/flash_attention_sim.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Operator Fusion & Online Softmax (FlashAttention tiling math in Python)` and save the test log beside `src/pymastery/gpu_interop/flash_attention_sim.py` and `tests/test_flash_attention_sim.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Operator Fusion & Online Softmax (FlashAttention tiling math in Python)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day187/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Matches standard $O(N^2)$ materialized attention to `1e-5` across arbitrary tile sizes while never allocating an $N \times N$ matrix.

- **Tangible Deliverable:** `src/pymastery/gpu_interop/flash_attention_sim.py` and `tests/test_flash_attention_sim.py`
- **STOP WHEN:** Matches standard $O(N^2)$ materialized attention to `1e-5` across arbitrary tile sizes while never allocating an $N \times N$ matrix.
- **Do Not Study Yet:** Hopper WGMMA warp-group instructions.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 188: Asynchronous GPU Execution Model in Python: Streams, Events, Host-Device Pinning & CUDA Graphs CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how Python orchestrates asynchronous GPU work: why `cudaStreamSynchronize` stalls Python, how CUDA Graphs record a sequence of kernel launches into a single replayable C call to eliminate Python CPU overhead.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day188/`, and review the mapped book section for `Asynchronous GPU Execution Model in Python: Streams, Events, Host-Device Pinning & CUDA Graphs`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `AsyncStreamAndGraphSimulator` in `src/pymastery/gpu_interop/cuda_graph_sim.py` modeling static buffer capture, replay, and launch-overhead reduction.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Asynchronous GPU Execution Model in Python: Streams, Events, Host-Device Pinning & CUDA Graphs` and save the test log beside `src/pymastery/gpu_interop/cuda_graph_sim.py` and `tests/test_cuda_graph_sim.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Asynchronous GPU Execution Model in Python: Streams, Events, Host-Device Pinning & CUDA Graphs`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day188/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** Enforces static buffer address invariants during graph replay and detects illegal dynamic allocations inside graph capture.

- **Tangible Deliverable:** `src/pymastery/gpu_interop/cuda_graph_sim.py` and `tests/test_cuda_graph_sim.py`
- **STOP WHEN:** Enforces static buffer address invariants during graph replay and detects illegal dynamic allocations inside graph capture.
- **Do Not Study Yet:** Driver-level CUgraph API.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 189: Week 27 Gate: MetaTensor Shape Verifier, Tiled FlashAttention & CUDA Graph Runner CORE: Advanced Track (Months 5-8) [MINOR: Zero-copy tensor exchange (__dlpack__) and custom operator registration contracts]¶

> **The Big Picture:** Understand how modern AI engines (vLLM, PyTorch, SGLang, FlashAttention) bind C++, CUDA, and Triton kernels into Python using `pybind11`/`nanobind`, `torch.library.custom_op`, `@triton.jit`, and the zero-copy DLPack protocol.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `MetaTensor` shape propagation, `tiled_flash_attention_forward`, and `CUDA Graph` static decode capture into a unified execution pipeline.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts — because this week connects your deep Python mastery directly to GPU kernels, PyTorch `torch.compile`, and vLLM C++/Triton extensions.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 8 & PyTorch Custom Operators / Triton Official Documentation

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day189/`, and review the mapped book section for `Week 27 Gate: MetaTensor Shape Verifier, Tiled FlashAttention & CUDA Graph Runner`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Verify a 70B model configuration using `MetaTensor`s in 0 MB RAM and run tiled FlashAttention parity tests.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 27 Gate: MetaTensor Shape Verifier, Tiled FlashAttention & CUDA Graph Runner` and save the test log beside `gates/week27-gate.md` and `tests/test_week27_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Zero-copy tensor exchange (`__dlpack__`) and custom operator registration contracts: Every custom operator must specify a meta/shape-inference function (`register_fake`) so graph compilers know output shapes without running GPU kernels.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 27 Gate: MetaTensor Shape Verifier, Tiled FlashAttention & CUDA Graph Runner`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day189/failure-notes.md`.

> **Engineering Trap to Avoid:** Never convert a GPU `torch.Tensor` to a CPU `numpy` array or Python `list` (`tensor.tolist()`) in the middle of an inference forward pass—it forces a synchronous host-device stall!

> **Daily Gate (STOP WHEN):** All shape inference, FlashAttention tiling, and static graph replay tests pass.

- **Tangible Deliverable:** `gates/week27-gate.md` and `tests/test_week27_gate.py`
- **STOP WHEN:** All shape inference, FlashAttention tiling, and static graph replay tests pass.
- **Do Not Study Yet:** Multi-datacenter WAN training.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 28 - Month 7 Milestone: High-Performance Hybrid Python/Native Runtime & Profiler¶

Day 190: Unified Backend Dispatch System (`pure_python`, `tiled_spmd`, `native_c`) CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Wire your `KernelBackendBase` registry so `NanoLLM` can switch transparently between pure Python, tiled FlashAttention, and native C kernels via config.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day190/`, and review the mapped book section for `Unified Backend Dispatch System (pure_python, tiled_spmd, native_c)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/runtime/backend_dispatcher.py` and `tests/test_backend_dispatcher.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Unified Backend Dispatch System (pure_python, tiled_spmd, native_c)` and save the test log beside `src/pymastery/runtime/backend_dispatcher.py` and `tests/test_backend_dispatcher.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Unified Backend Dispatch System (pure_python, tiled_spmd, native_c)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day190/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** All three backends produce identical token outputs within `1e-5` logit tolerance.

- **Tangible Deliverable:** `src/pymastery/runtime/backend_dispatcher.py` and `tests/test_backend_dispatcher.py`
- **STOP WHEN:** All three backends produce identical token outputs within `1e-5` logit tolerance.
- **Do Not Study Yet:** Third-party plugin registries.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 191: Static-Buffer Decode Step Runner (CUDA-Graph-Ready Execution Pattern) CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Refactor the decode step of `NanoLLM` so inputs, token positions, and outputs are copied into fixed-address pre-allocated buffers before execution—the exact pattern required by vLLM/SGLang CUDA Graphs.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day191/`, and review the mapped book section for `Static-Buffer Decode Step Runner (CUDA-Graph-Ready Execution Pattern)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `StaticDecodeRunner` in `src/pymastery/runtime/static_decode.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Static-Buffer Decode Step Runner (CUDA-Graph-Ready Execution Pattern)` and save the test log beside `src/pymastery/runtime/static_decode.py` and `tests/test_static_decode.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Static-Buffer Decode Step Runner (CUDA-Graph-Ready Execution Pattern)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day191/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** Buffer memory addresses remain 100% constant across 500 consecutive decode steps and `tracemalloc` reports 0 bytes allocated per step.

- **Tangible Deliverable:** `src/pymastery/runtime/static_decode.py` and `tests/test_static_decode.py`
- **STOP WHEN:** Buffer memory addresses remain 100% constant across 500 consecutive decode steps and `tracemalloc` reports 0 bytes allocated per step.
- **Do Not Study Yet:** Dynamic graph recompilation.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 192: Automated Bottleneck Diagnostic CLI (`pymastery-profile`)

CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a CLI command that runs a target workload under `deterministic_profiler`, `SamplingFlamegraphProfiler`, `AllocationHotspotAnalyzer`, and `RooflineModel`, and outputs a unified diagnostic Markdown report.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day192/`, and review the mapped book section for `Automated Bottleneck Diagnostic CLI (pymastery-profile)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/runtime/profile_cli.py` and `tests/test_profile_cli.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Automated Bottleneck Diagnostic CLI (pymastery-profile)` and save the test log beside `src/pymastery/runtime/profile_cli.py` and `tests/test_profile_cli.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Automated Bottleneck Diagnostic CLI (pymastery-profile)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day192/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** Produces a complete Markdown report with top CPU functions, top allocation lines, folded flamegraph, and roofline regime.

- **Tangible Deliverable:** `src/pymastery/runtime/profile_cli.py` and `tests/test_profile_cli.py`
- **STOP WHEN:** Produces a complete Markdown report with top CPU functions, top allocation lines, folded flamegraph, and roofline regime.
- **Do Not Study Yet:** Proprietary SaaS APM agents.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 193: Memory Footprint & KV Capacity Planner using `MetaTensor`s

CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a dry-run capacity planner that traces any Transformer config (`num_layers`, `hidden_size`, `num_q_heads`, `num_kv_heads`, `dtype`) using `MetaTensor`s to compute exact weight GB, activation peak MB, and max concurrent KV tokens for a target GPU memory budget (e.g., 24 GB / 80 GB).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day193/`, and review the mapped book section for `Memory Footprint & KV Capacity Planner using` MetaTensors.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `CapacityPlanner` in `src/pymastery/runtime/capacity_planner.py`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Memory Footprint & KV Capacity Planner using` MetaTensors and save the test log beside `src/pymastery/runtime/capacity_planner.py` and `tests/test_capacity_planner.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Memory Footprint & KV Capacity Planner using` MetaTensors, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day193/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** Matches hand-calculated Llama-3-8B and Llama-3-70B weight and KV-cache byte formulas 100%.

- **Tangible Deliverable:** `src/pymastery/runtime/capacity_planner.py` and `tests/test_capacity_planner.py`
- **STOP WHEN:** Matches hand-calculated Llama-3-8B and Llama-3-70B weight and KV-cache byte formulas 100%.
- **Do Not Study Yet:** Trial-and-error OOM crashing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 194: Benchmarking & Comparing All 3 Backends Across Prefill and Decode Workloads CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Run a controlled 5-repetition benchmark matrix comparing `pure_python`, `tiled_spmd`, and `native_c` across batch sizes and sequence lengths.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge

from high-level Python to low-level C/C++/GPU systems.

- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day194/`, and review the mapped book section for `Benchmarking & Comparing All 3 Backends Across Prefill and Decode Workloads`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Execute the matrix and save `artifacts/month07-backend-comparison.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Benchmarking & Comparing All 3 Backends Across Prefill and Decode Workloads` and save the test log beside `artifacts/month07-backend-comparison.md` and `tests/test_backend_matrix.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Benchmarking & Comparing All 3 Backends Across Prefill and Decode Workloads`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day194/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** Benchmark table records p50/p95 prefill TTFT, decode ITL, memory peak, and numerical max-abs-diff.

- **Tangible Deliverable:** `artifacts/month07-backend-comparison.md` and `tests/test_backend_matrix.py`
- **STOP WHEN:** Benchmark table records p50/p95 prefill TTFT, decode ITL, memory peak, and numerical max-abs-diff.
- **Do Not Study Yet:** Uncontrolled single-run timing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 195: Closed-Book Month 7 Drill: Zero-Copy FFI & Stride Debugging

CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Diagnose and fix 4 seeded low-level bugs: a non-contiguous transposed tensor passed to a C pointer, a missing `ctypes.argtypes` declaration, a GC-collected backing buffer, and a softmax overflow.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day195/`, and review the mapped book section for `Closed-Book Month 7 Drill: Zero-Copy FFI & Stride Debugging`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Fix all 4 bugs in `src/pymastery/drills/month07_bug_hunt.py` and verify with tests.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Closed-Book Month 7 Drill: Zero-Copy FFI & Stride Debugging` and save the test log beside `src/pymastery/drills/month07_bug_hunt.py` and `tests/test_month07_bug_hunt.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Closed-Book Month 7 Drill: Zero-Copy FFI & Stride Debugging`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day195/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** All 4 regression tests pass deterministically.

- **Tangible Deliverable:** `src/pymastery/drills/month07_bug_hunt.py` and `tests/test_month07_bug_hunt.py`

- **STOP WHEN:** All 4 regression tests pass deterministically.
- **Do Not Study Yet:** Adding new features.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 196: Week 28 Gate (Month 7 Review): Profiling & Native Interop Mastery Gate CORE: Advanced Track (Months 5-8)

MINOR: End-to-end throughput/latency measurement and zero host-device synchronization traps ¶

> **The Big Picture:** Synthesize Month 7 by integrating your profiler, roofline analyzer, native `ctypes`/C kernels, `MetaTensor` verifier, and tiled FlashAttention into your inference engine.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Complete the Month 7 gate across Weeks 25–28 (Days 169–196).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end throughput/latency measurement and zero host-device synchronization traps — because Month 7 completes your bridge from high-level Python to low-level C/C++/GPU systems.
- **Mapped Book & Docs:** High Performance Python, 2nd Ed. Ch. 1–9, 11

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day196/`, and review the mapped book section for `Week 28 Gate (Month 7 Review): Profiling & Native Interop Mastery Gate`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run full repository tests, `mypy --strict`, and `ruff check`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 28 Gate (Month 7 Review): Profiling & Native Interop Mastery Gate` and save the test log beside `gates/month07-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end throughput/latency measurement and zero host-device synchronization traps: Verify every benchmark compares pure Python vs vectorized/tiled vs native C/FFI execution.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 28 Gate (Month 7 Review): Profiling & Native Interop Mastery Gate`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day196/failure-notes.md`.

> **Engineering Trap to Avoid:** Never guess whether a workload is CPU-launch-bound or compute-bound—let the profiler and roofline calculator prove it.

> **Daily Gate (STOP WHEN):** All 196 days of tests pass cleanly.

- **Tangible Deliverable:** `gates/month07-gate.md`
- **STOP WHEN:** All 196 days of tests pass cleanly.
- **Do Not Study Yet:** Distributed multi-worker RPC (starts tomorrow in Month 8!).
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Month 8: Distributed IPC, Observability, vLLM Internals & Advanced Capstone (Weeks 29–32, Days 197–224)¶

Week 29 - Distributed Systems, Multi-Process IPC, Prometheus Telemetry & Fault Tolerance¶

Day 197: High-throughput Multi-Process RPC (`asyncio` Unix/TCP sockets + SharedMemory descriptor passing) CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build the control-plane/data-plane split used by vLLM v1 (`AsyncLLMEngine` -> `EngineCore` process): small control messages over framed async sockets, large tensor/token payloads over `SharedMemory`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day197/`, and review the mapped book section for `High-throughput Multi-Process RPC (asyncio Unix/TCP sockets + SharedMemory descriptor passing)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `EngineCoreIPCClient` and `EngineCoreIPCServer` in `src/pymastery/distributed/engine_ipc.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `High-throughput Multi-Process RPC (asyncio Unix/TCP sockets + SharedMemory descriptor passing)` and save the test log beside `src/pymastery/distributed/engine_ipc.py` and `tests/test_engine_ipc.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `High-throughput Multi-Process RPC (asyncio Unix/TCP sockets + SharedMemory descriptor passing)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day197/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Streams token chunks from the `EngineCore` process back to the async frontend with < 0.5ms IPC overhead.

- **Tangible Deliverable:** `src/pymastery/distributed/engine_ipc.py` and `tests/test_engine_ipc.py`
- **STOP WHEN:** Streams token chunks from the `EngineCore` process back to the async frontend with < 0.5ms IPC overhead.
- **Do Not Study Yet:** CORBA/SOAP XML.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 198: Single-Controller Multi-Worker Coordination (Tensor-Parallel Worker Group Simulation) CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Coordinate N parallel worker processes (simulating Tensor Parallelism ranks `tp_rank = 0..N-1`) from a single Python controller using lockstep broadcast commands and collective barrier/all-reduce verification.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day198/`, and review the mapped book section for `Single-Controller Multi-Worker Coordination (Tensor-Parallel Worker Group Simulation)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MultiRankWorkerGroup` and simulated `all_reduce_sum` in `src/pymastery/distributed/tp_workers.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Single-Controller Multi-Worker Coordination (Tensor-Parallel Worker Group Simulation)` and save the test log beside `src/pymastery/distributed/tp_workers.py` and `tests/test_tp_workers.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Single-Controller Multi-Worker Coordination (Tensor-Parallel Worker Group Simulation)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day198/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** 2-rank tensor-parallel sharded matmul (`ColumnParallelLinear` + `RowParallelLinear` + `all_reduce`) matches single-rank unsharded matmul to `1e-5`!

- **Tangible Deliverable:** `src/pymastery/distributed/tp_workers.py` and `tests/test_tp_workers.py`
- **STOP WHEN:** 2-rank tensor-parallel sharded matmul (`ColumnParallelLinear` + `RowParallelLinear` + `all_reduce`) matches single-rank unsharded matmul to `1e-5`!
- **Do Not Study Yet:** InfiniBand verbs C driver.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 199: Implementing a Prometheus Metrics Engine from Scratch (`Counter`, `Gauge`, `Histogram`, Text Exposition Format) CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a zero-dependency, thread/async-safe Prometheus metrics library that enforces label-cardinality rules and renders the standard `# HELP`, `# TYPE`, and `_bucket{le="..."}` text exposition format.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day199/`, and review the mapped book section for `Implementing a Prometheus Metrics Engine from Scratch (Counter, Gauge, Histogram, Text Exposition Format)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `MetricsRegistry`, `Counter`, `Gauge`, and `Histogram` in `src/pymastery/distributed/prometheus_engine.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Implementing a Prometheus Metrics Engine from Scratch (Counter, Gauge, Histogram, Text Exposition Format)` and save the test log beside `src/pymastery/distributed/prometheus_engine.py` and `tests/test_prometheus_engine.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Implementing a Prometheus Metrics Engine from Scratch (Counter, Gauge,`

Histogram, `Text Exposition Format`), verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day199/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Output matches Prometheus 0.0.4 text format specification and rejects high-cardinality forbidden labels (`request_id`).

- **Tangible Deliverable:** `src/pymastery/distributed/prometheus_engine.py` and `tests/test_prometheus_engine.py`
- **STOP WHEN:** Output matches Prometheus 0.0.4 text format specification and rejects high-cardinality forbidden labels (`request_id`).
- **Do Not Study Yet:** Prometheus TSDB disk blocks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 200: Circuit Breakers, Bulkheads, Exponential Backoff with Jitter & Load Shedding CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Protect services from cascading failures using state-machine Circuit Breakers (`CLOSED` -> `OPEN` -> `HALF_OPEN`), per-route Bulkhead semaphores, and deadline-aware load shedding.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day200/`, and review the mapped book section for `Circuit Breakers, Bulkheads, Exponential Backoff with Jitter & Load Shedding`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `CircuitBreaker` and `AdaptiveLoadShedder` in `src/pymastery/distributed/resilience.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Circuit Breakers, Bulkheads, Exponential Backoff with Jitter & Load Shedding` and save the test log beside `src/pymastery/distributed/resilience.py` and `tests/test_resilience.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Circuit Breakers, Bulkheads, Exponential Backoff with Jitter & Load Shedding`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day200/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Trips to `OPEN` after K consecutive failures, fails fast in O(1), probes recovery in `HALF_OPEN`, and resets to `CLOSED` on success.

- **Tangible Deliverable:** `src/pymastery/distributed/resilience.py` and `tests/test_resilience.py`
- **STOP WHEN:** Trips to `OPEN` after K consecutive failures, fails fast in O(1), probes recovery in `HALF_OPEN`, and resets to `CLOSED` on success.
- **Do Not Study Yet:** Service mesh sidecar proxies.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 201: Consistent Hashing & Prefix-Aware Load Balancing Router (Multi-Instance Routing) CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Route incoming LLM requests across multiple engine replicas using a weighted trade-off between **Prefix Cache Affinity** (routing shared prefixes to the replica that already holds the KV cache) and **Queue Load Balance**.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day201/`, and review the mapped book section for `Consistent Hashing & Prefix-Aware Load Balancing Router (Multi-Instance Routing)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `PrefixAwareLoadBalancer` in `src/pymastery/distributed/router.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Consistent Hashing & Prefix-Aware Load Balancing Router (Multi-Instance Routing)` and save the test log beside `src/pymastery/distributed/router.py` and `tests/test_router.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Consistent Hashing & Prefix-Aware Load Balancing Router (Multi-Instance Routing)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day201/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Achieves >85% cache hit affinity on repeated prefixes while preventing replica queue overload when one prefix is hot.

- **Tangible Deliverable:** `src/pymastery/distributed/router.py` and `tests/test_router.py`
- **STOP WHEN:** Achieves >85% cache hit affinity on repeated prefixes while preventing replica queue overload when one prefix is hot.
- **Do Not Study Yet:** Hardware L4 switches.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 202: Deterministic Chaos Testing Framework (`FaultInjector` middleware & process chaos) CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build a programmable chaos injection harness capable of injecting configurable latency spikes, dropped connections, simulated OOM block exhaustion, and worker process kills.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day202/`, and review the mapped book section for `Deterministic Chaos Testing Framework (`FaultInjector `middleware & process chaos)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `ChaosInjector` in `src/pymastery/distributed/chaos.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Deterministic Chaos Testing Framework (`FaultInjector `middleware & process chaos)` and save the test log beside `src/pymastery/distributed/chaos.py` and `tests/test_chaos.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as `request_id` or raw prompts) as Prometheus metric label values!
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Deterministic Chaos Testing Framework (`FaultInjector `middleware & process chaos)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day202/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Reproducibly triggers each failure mode from a deterministic seed and verifies recovery telemetry.

- **Tangible Deliverable:** `src/pymastery/distributed/chaos.py` and `tests/test_chaos.py`
- **STOP WHEN:** Reproducibly triggers each failure mode from a deterministic seed and verifies recovery telemetry.
- **Do Not Study Yet:** Random unseeded production faults.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 203: Week 29 Gate: Distributed Tensor-Parallel Worker Group & Prefix Router CORE: Advanced Track (Months 5-8)

MINOR: Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery ¶

> **The Big Picture:** Build the distributed systems and production observability primitives used by real inference clusters: multi-process ZeroMQ/socket framing, tensor-parallel worker coordination, Prometheus exposition format from scratch, and circuit breakers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Combine `PrefixAwareLoadBalancer`, `EngineCoreIPC`, `MultiRankWorkerGroup`, `MetricsRegistry`, and `CircuitBreaker`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery — because in production, an inference engine is a distributed multi-process system that must expose metrics and survive partial failures.
- **Mapped Book & Docs:** Python Concurrency with asyncio (Fowler) & Designing Data-Intensive Applications / Prometheus Docs

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day203/`, and review the mapped book section for `Week 29 Gate: Distributed Tensor-Parallel Worker Group & Prefix Router`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run a multi-replica routing and fault-recovery test suite.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 29 Gate: Distributed Tensor-Parallel Worker Group & Prefix Router` and save the test log beside `gates/week29-gate.md` and `tests/test_week29_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Wire protocol resilience, Prometheus label cardinality bounds, and worker health recovery: Never allow unbounded dynamic strings (such as

`request_id` or raw prompts) as Prometheus metric label values!

- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 29 Gate: Distributed Tensor-Parallel Worker Group & Prefix Router`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day203/failure-notes.md`.

> **Engineering Trap to Avoid:** Never block the API event loop waiting on a dead worker process without a heartbeat timeout and circuit breaker.

> **Daily Gate (STOP WHEN):** Survives simulated replica failure via circuit breaker failover and preserves exact Prometheus telemetry.

- **Tangible Deliverable:** `gates/week29-gate.md` and `tests/test_week29_gate.py`
- **STOP WHEN:** Survives simulated replica failure via circuit breaker failover and preserves exact Prometheus telemetry.
- **Do Not Study Yet:** Multi-region DNS.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 30 - Reading & Modifying Real Inference Engine Architecture (vLLM / SGLang Patterns)¶

Day 204: Architecture Map: How vLLM Separates API Server (`AsyncLLM`) -> `EngineCore` -> `Scheduler` -> `ModelRunner` CORE: Advanced Track (Months 5-8) MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Trace the complete data and control flow of an OpenAI `/v1/chat/completions` request through tokenization, IPC queue, continuous batching `Scheduler.schedule()`, `ModelRunner.execute_model()`, sampler, and detokenizer.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.

- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day204/`, and review the mapped book section for `Architecture Map: How vLLM Separates API Server (AsyncLLM) -> EngineCore -> Scheduler -> ModelRunner``.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Implement the unified `vllm_mini` architectural skeleton in `src/pymastery/vllm_mini/architecture.py` with exact interface contracts.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Architecture Map: How vLLM Separates API Server (AsyncLLM) -> EngineCore -> Scheduler -> ModelRunner` and save the test log beside` src/pymastery/vllm_mini/architecture.py and tests/test_vllm_mini_arch.py`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Architecture Map: How vLLM Separates API Server (AsyncLLM) -> EngineCore -> Scheduler -> ModelRunner`, verify that` pytest or mypy --strict `catches it immediately, restore the fix, and log the symptom in` artifacts/day204/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** Every stage boundary (`EngineCoreRequest`, `SchedulerOutput`, `ModelRunnerOutput`, `RequestOutput`) is strictly typed and tested.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/architecture.py` and `tests/test_vllm_mini_arch.py`
- **STOP WHEN:** Every stage boundary (`EngineCoreRequest`, `SchedulerOutput`, `ModelRunnerOutput`, `RequestOutput`) is strictly typed and tested.
- **Do Not Study Yet:** Legacy vLLM v0 deprecated internals.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 205: Streaming Detokenizer (`IncrementalDetokenizer`) for Token-by-Token Text Emission CORE: Advanced Track (Months 5-8)

MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Solve the subword token boundary problem during streaming: holding back incomplete UTF-8 byte tokens and emitting clean string deltas without duplicate spaces.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day205/`, and review the mapped book section for `Streaming Detokenizer (`IncrementalDetokenizer`) for Token-by-Token Text Emission`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `IncrementalDetokenizer` in `src/pymastery/vllm_mini/detokenizer.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Streaming Detokenizer (`IncrementalDetokenizer`) for Token-by-Token Text Emission` and save the test log beside `src/pymastery/vllm_mini/detokenizer.py` and `tests/test_detokenizer.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Streaming Detokenizer (`IncrementalDetokenizer`) for Token-by-Token Text Emission`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day205/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** Concatenating all streamed text deltas equals the full-sequence decoded string 100% of the time, even with split multi-byte emojis.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/detokenizer.py` and `tests/test_detokenizer.py`
- **STOP WHEN:** Concatenating all streamed text deltas equals the full-sequence decoded string 100% of the time, even with split multi-byte emojis.
- **Do Not Study Yet:** Regex post-processing hacks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 206: OpenAI-Compatible `/v1/completions` & `/v1/chat/completions` Protocol (Sync & SSE Streaming) CORE: Advanced Track (Months 5-8)

MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement the exact OpenAI wire schema (`ChatCompletionRequest`, `ChatCompletionChunk`, `ChoiceDelta`, `Usage`, `data: [DONE]`) using `NanoSchema` and `NanoServe`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day206/`, and review the mapped book section for `OpenAI-Compatible` /v1/completions `&` /v1/chat/completions `Protocol (Sync & SSE Streaming)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/vllm_mini/openai_server.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `OpenAI-Compatible` /v1/completions `&` /v1/chat/completions `Protocol (Sync & SSE Streaming)` and save the test log beside `src/pymastery/vllm_mini/openai_server.py` and `tests/test_openai_server.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `OpenAI-Compatible` /v1/completions `&` /v1/chat/completions `Protocol (Sync & SSE Streaming)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day206/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** Compatible with standard OpenAI client expectations for both `stream=False` and `stream=True`.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/openai_server.py` and `tests/test_openai_server.py`
- **STOP WHEN:** Compatible with standard OpenAI client expectations for both `stream=False` and `stream=True`.
- **Do Not Study Yet:** Proprietary non-standard JSON APIs.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 207: Guided / Structured Decoding via Finite-State / JSON Grammar Masking (`LogitsProcessor`) CORE: Advanced Track (Months 5-8)

MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how structured outputs (JSON mode / regex / schema-constrained decoding) work in Python: tracking parser state per sequence and setting disallowed token logits to `-inf` before sampling.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day207/`, and review the mapped book section for `Guided / Structured Decoding via Finite-State / JSON Grammar Masking (`LogitsProcessor`)`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `JSONSchemaLogitsMasker` in `src/pymastery/vllm_mini/guided_decoding.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Guided / Structured Decoding via Finite-State / JSON Grammar Masking (`LogitsProcessor`)` and save the test log beside `src/pymastery/vllm_mini/guided_decoding.py` and `tests/test_guided_decoding.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Guided / Structured Decoding via Finite-State / JSON Grammar Masking (`LogitsProcessor`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day207/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** 100% of sampled sequences under `JSONSchemaLogitsMasker` parse as valid JSON matching the target schema.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/guided_decoding.py` and `tests/test_guided_decoding.py`
- **STOP WHEN:** 100% of sampled sequences under `JSONSchemaLogitsMasker` parse as valid JSON matching the target schema.
- **Do Not Study Yet:** Retry-on-invalid-JSON loops.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 208: LoRA Multi-Adapter Routing & Dynamic Adapter LRU Cache Manager CORE: Advanced Track (Months 5-8)

MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Understand how a single base model serves dozens of LoRA adapters concurrently by managing an active GPU/RAM LoRA slot cache and tagging each request in a batch with its `lora_int_id`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day208/`, and review the mapped book section for `LoRA Multi-Adapter Routing & Dynamic Adapter LRU Cache Manager`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `LoRACacheManager` and per-token LoRA batch routing in `src/pymastery/vllm_mini/lora_manager.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `LoRA Multi-Adapter Routing & Dynamic Adapter LRU Cache Manager` and save the test log beside `src/pymastery/vllm_mini/lora_manager.py` and `tests/test_lora_manager.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `LoRA Multi-Adapter Routing & Dynamic Adapter LRU Cache Manager`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day208/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** Mixed-adapter batches compute accurate base + low-rank (`x @ A @ B * scale`) outputs and evict cold adapters in LRU order.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/lora_manager.py` and `tests/test_lora_manager.py`
- **STOP WHEN:** Mixed-adapter batches compute accurate base + low-rank (`x @ A @ B * scale`) outputs and evict cold adapters in LRU order.
- **Do Not Study Yet:** Fine-tuning gradient updates.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 209: Engine Observability Hook: Real-Time TTFT, ITL, KV-Cache Usage %, Preemption Count & Goodput CORE: Advanced Track (Months 5-8)

MINOR: End-to-end architectural traceability and zero-regression modification ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Wire your `MetricsRegistry` directly into the scheduler and engine loop to expose `vllm:num_requests_running`, `vllm:num_requests_waiting`, `vllm:gpu_cache_usage_perc`, `vllm:time_to_first_token_seconds`, and `vllm:time_per_output_token_seconds`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day209/`, and review the mapped book section for `Engine Observability Hook: Real-Time TTFT, ITL, KV-Cache Usage %, Preemption Count & Goodput`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `EngineMetricsCollector` in `src/pymastery/vllm_mini/metrics.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Engine Observability Hook: Real-Time TTFT, ITL, KV-Cache Usage %, Preemption Count & Goodput` and save the test log beside `src/pymastery/vllm_mini/metrics.py` and `tests/test_vllm_mini_metrics.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Engine Observability Hook: Real-Time TTFT, ITL, KV-Cache Usage %, Preemption Count & Goodput`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day209/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** `GET /metrics` exposes accurate real-time gauges, counters, and latency histograms during load.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/metrics.py` and `tests/test_vllm_mini_metrics.py`
- **STOP WHEN:** `GET /metrics` exposes accurate real-time gauges, counters, and latency histograms during load.
- **Do Not Study Yet:** External log scraping.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 210: Week 30 Gate: Complete `vllm_mini` Integration Test

CORE: Advanced Track (Months 5-8) **MINOR: End-to-end architectural traceability and zero-regression modification** ¶

> **The Big Picture:** Apply your 7+ months of Python mastery to the exact code patterns used inside vLLM and SGLang: `AsyncLLMEngine`, `Scheduler`, `BlockManager`, `ModelRunner`, and OpenAI-compatible API servers.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Run an end-to-end integration test of `vllm_mini` covering OpenAI chat streaming, prefix caching, guided JSON decoding, LoRA routing, and `/metrics`.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** End-to-end architectural traceability and zero-regression modification — because this week bridges your hand-built components directly to reading and contributing to vLLM and SGLang.
- **Mapped Book & Docs:** vLLM Official Architecture Docs & Source Patterns + Fluent Python, 2nd Ed.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day210/`, and review the mapped book section for `Week 30 Gate: Complete` vllm_mini `Integration Test`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Execute `tests/test_week30_gate.py`.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 30 Gate: Complete` vllm_mini `Integration Test` and save the test log beside `gates/week30-gate.md` and `tests/test_week30_gate.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** End-to-end architectural traceability and zero-regression modification: Verify every architectural component has a clear interface (`Protocol`), unit tests, and metric hooks.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 30 Gate: Complete` vllm_mini `Integration Test`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day210/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not treat vLLM or SGLang as black boxes—they are built from the exact Python patterns (dataclasses, asyncio queues, ZeroMQ/IPC, Paged KV dicts, custom ops) you just built!

> **Daily Gate (STOP WHEN):** All features work together seamlessly under `mypy --strict` and `pytest`.

- **Tangible Deliverable:** `gates/week30-gate.md` and `tests/test_week30_gate.py`
- **STOP WHEN:** All features work together seamlessly under `mypy --strict` and `pytest`.
- **Do Not Study Yet:** New unverified features.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 31 - Month 8 Advanced Mastery Capstone — Part 1: Full-Stack `Mini-vLLM` Production Engine¶

Day 211: Capstone Integration: Multi-Process `AsyncLLMEngine` + `EngineCore` + `PagedKVScheduler` CORE: Advanced Track (Months 5-8)

MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Connect the `NanoServe` OpenAI HTTP/SSE frontend to the background `EngineCore` process via `EngineCoreIPC`, driving the

`ContinuousBatchScheduler` and `PrefixCachingBlockManager`.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day211/`, and review the mapped book section for `Capstone Integration: Multi-Process AsyncLLMEngine + EngineCore + PagedKVScheduler``.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Finalize `src/pymastery/vllm_mini/engine.py` and `tests/test_vllm_mini_engine.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Integration: Multi-Process AsyncLLMEngine + EngineCore + PagedKVScheduler` and save the test log beside src/pymastery/vllm_mini/engine.py and tests/test_vllm_mini_engine.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Integration: Multi-Process` AsyncLLMEngine + EngineCore + PagedKVScheduler`, verify that pytest or mypy --strict catches it immediately, restore the fix, and log the symptom in` artifacts/day211/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Supports 64 concurrent streaming requests with automatic prefix caching, chunked prefill, and preemption.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/engine.py` and `tests/test_vllm_mini_engine.py`
- **STOP WHEN:** Supports 64 concurrent streaming requests with automatic prefix caching, chunked prefill, and preemption.
- **Do Not Study Yet:** Unbounded memory growth.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 212: Capstone Execution Backend: Hybrid `NanoLLM` with Quantization (INT8/INT4), FlashAttention & Native C Kernels CORE: Advanced Track (Months 5-8)

MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Wire the `ModelRunner` to execute your KV-cached `NanoLLM` using INT8/INT4 quantized weights, tiled FlashAttention, and native C/FFI kernels with static decode buffers.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day212/`, and review the mapped book section for `Capstone Execution Backend: Hybrid` NanoLLM `with Quantization (INT8/INT4), FlashAttention & Native C Kernels`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Finalize `src/pymastery/vllm_mini/model_runner.py` and `tests/test_vllm_mini_runner.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Execution Backend: Hybrid` NanoLLM `with Quantization (INT8/INT4), FlashAttention & Native C Kernels` and save the test log beside `src/pymastery/vllm_mini/model_runner.py` and `tests/test_vllm_mini_runner.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Execution Backend: Hybrid` NanoLLM `with Quantization (INT8/INT4), FlashAttention & Native C Kernels`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day212/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Passes quality parity gate between FP32, INT8, and INT4 modes and allocates 0 bytes per static decode step.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/model_runner.py` and `tests/test_vllm_mini_runner.py`
- **STOP WHEN:** Passes quality parity gate between FP32, INT8, and INT4 modes and allocates 0 bytes per static decode step.
- **Do Not Study Yet:** External black-box engines.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 213: Capstone Cluster Layer: 2-Replica Prefix-Aware Router, Circuit Breaker & Health Draining CORE: Advanced Track (Months 5-8)

MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Launch two `Mini-vLLM` engine replicas behind your `PrefixAwareLoadBalancer` with automatic circuit breaking, `/readyz` health probes, and graceful SIGTERM draining.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day213/`, and review the mapped book section for `Capstone Cluster Layer: 2-Replica Prefix-Aware Router, Circuit Breaker & Health Draining`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/vllm_mini/cluster.py` and `tests/test_vllm_mini_cluster.py`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Cluster Layer: 2-Replica Prefix-Aware Router, Circuit Breaker & Health Draining` and save the test log beside `src/pymastery/vllm_mini/cluster.py` and `tests/test_vllm_mini_cluster.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Cluster Layer: 2-Replica Prefix-Aware Router, Circuit Breaker & Health Draining`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day213/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Killing replica 1 mid-benchmark triggers automatic failover to replica 2 with zero unhandled exceptions.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/cluster.py` and `tests/test_vllm_mini_cluster.py`
- **STOP WHEN:** Killing replica 1 mid-benchmark triggers automatic failover to replica 2 with zero unhandled exceptions.
- **Do Not Study Yet:** Kubernetes YAML manifests.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 214: Capstone Benchmark & SLO Goodput Suite (`vllm-mini-bench`)

CORE: Advanced Track (Months 5-8) **MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience** ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Run a comprehensive benchmark matrix across concurrency levels (1, 4, 8, 16, 32), prompt/output lengths, and quantization modes, measuring TTFT, ITL, p50/p95/p99, KV cache utilization, prefix hit rate, and SLO Goodput (requests/sec meeting both TTFT < X and ITL < Y).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day214/`, and review the mapped book section for `Capstone Benchmark & SLO Goodput Suite (`vllm-mini-bench`)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Implement `src/pymastery/vllm_mini/benchmark_suite.py` and persist results to SQLite + Markdown.

- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Benchmark & SLO Goodput Suite (`vllm-mini-bench`)` and save the test log beside `src/pymastery/vllm_mini/benchmark_suite.py` and `artifacts/week31-benchmark-matrix.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Benchmark & SLO Goodput Suite (`vllm-mini-bench`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day214/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Generates complete 5-repetition benchmark tables, Pareto quantization tradeoffs, and roofline classifications.

- **Tangible Deliverable:** `src/pymastery/vllm_mini/benchmark_suite.py` and `artifacts/week31-benchmark-matrix.md`
- **STOP WHEN:** Generates complete 5-repetition benchmark tables, Pareto quantization tradeoffs, and roofline classifications.
- **Do Not Study Yet:** Single-number vanity metrics.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 215: Capstone Profiling & Optimization Pass (Flamegraph + Allocation Audit + Speedup Verification) CORE: Advanced Track (Months 5-8)

MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Profile `Mini-vLLM` under high concurrency, generate a folded flamegraph and `tracemalloc` allocation report, and implement one measured optimization.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day215/`, and review the mapped book section for `Capstone Profiling & Optimization Pass (Flamegraph + Allocation Audit + Speedup Verification)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Save `artifacts/week31-flamegraph.folded` and `artifacts/week31-profile-report.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Profiling & Optimization Pass (Flamegraph + Allocation Audit + Speedup Verification)` and save the test log beside `artifacts/week31-flamegraph.folded` and `artifacts/week31-profile-report.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Profiling & Optimization Pass (Flamegraph + Allocation Audit + Speedup Verification)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day215/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Before/after profile proves reduced CPU or allocation overhead on the hot path.

- **Tangible Deliverable:** `artifacts/week31-flamegraph.folded` and `artifacts/week31-profile-report.md`
- **STOP WHEN:** Before/after profile proves reduced CPU or allocation overhead on the hot path.
- **Do Not Study Yet:** Unverified code changes.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 216: Capstone Chaos & Post-Mortem Suite (3 Production Failure Scenarios) CORE: Advanced Track (Months 5-8)

MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Execute 3 automated chaos scenarios (1: KV Block Exhaustion & Preemption Storm; 2: Slow/Disconnected Streaming Clients; 3: Worker Process Crash & Circuit-Breaker Failover) and generate structured incident post-mortems.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day216/`, and review the mapped book section for `Capstone Chaos & Post-Mortem Suite (3 Production Failure Scenarios)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Run `tests/test_vllm_mini_chaos.py` and write `docs/INCIDENT_POSTMORTEMS.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Capstone Chaos & Post-Mortem Suite (3 Production Failure Scenarios)` and save the test log beside `tests/test_vllm_mini_chaos.py` and `docs/INCIDENT_POSTMORTEMS.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Capstone Chaos & Post-Mortem Suite (3 Production Failure Scenarios)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day216/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** All 3 chaos tests pass deterministically and post-mortems link directly to Prometheus metrics and logs.

- **Tangible Deliverable:** `tests/test_vllm_mini_chaos.py` and `docs/INCIDENT_POSTMORTEMS.md`

- **STOP WHEN:** All 3 chaos tests pass deterministically and post-mortems link directly to Prometheus metrics and logs.
- **Do Not Study Yet:** Manual ad-hoc testing.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 217: Week 31 Gate: Advanced Capstone Feature Freeze

CORE: Advanced Track (Months 5-8) MINOR: Full-system integration, strict typing, zero-copy paths, and chaos resilience ¶

> **The Big Picture:** Assemble your 8-month crowning achievement: a complete, self-implemented, production-grade `Mini-vLLM` Inference Engine & Distributed Serving Stack written from scratch in quality Python.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Freeze all code across the entire 8-month `pymastery` repository and run a full static, unit, property, and chaos test audit.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Full-system integration, strict typing, zero-copy paths, and chaos resilience — because this capstone proves you have mastered Python at a Senior/Staff AI Inference Systems Engineer level.
- **Mapped Book & Docs:** All Core Books (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day217/`, and review the mapped book section for `Week 31 Gate: Advanced Capstone Feature Freeze`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Verify 100% `mypy --strict`, zero `ruff` warnings, and 100% test pass rate.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Week 31 Gate: Advanced Capstone Feature Freeze` and save the test log beside `gates/week31-gate.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Full-system integration, strict typing, zero-copy paths, and chaos resilience: Every component must be your own clean, typed, tested Python code.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Week 31 Gate: Advanced Capstone Feature Freeze`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day217/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not cut corners on error handling, cancellation cleanup, or benchmark reproducibility.

> **Daily Gate (STOP WHEN):** Complete repository builds and passes all tests in one command.

- **Tangible Deliverable:** `gates/week31-gate.md`
- **STOP WHEN:** Complete repository builds and passes all tests in one command.
- **Do Not Study Yet:** Last-minute feature creep.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Week 32 - Month 8 Advanced Mastery Capstone — Part 2: Final Defense, Portfolio & Certification¶

Day 218: Clean-Room Repository Audit, Packaging & Zero-Dependency Verification CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Build the final `.whl` distribution, install it in an isolated environment, and run the complete test and benchmark verification suite.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day218/`, and review the mapped book section for `Clean-Room Repository Audit, Packaging & Zero-Dependency Verification`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Execute `scripts/final_clean_room_audit.py` and save `artifacts/day218-clean-build.log`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Clean-Room Repository Audit, Packaging & Zero-Dependency Verification` and save the test log beside `artifacts/day218-clean-build.log` and `REPRODUCE.md`.

- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Clean-Room Repository Audit, Packaging & Zero-Dependency Verification`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day218/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** A fresh terminal reproduces the entire build, test suite, and smoke benchmark from `REPRODUCE.md` without errors.

- **Tangible Deliverable:** `artifacts/day218-clean-build.log` and `REPRODUCE.md`
- **STOP WHEN:** A fresh terminal reproduces the entire build, test suite, and smoke benchmark from `REPRODUCE.md` without errors.
- **Do Not Study Yet:** Undocumented environment hacks.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 219: Comprehensive Architecture Book & Operator Runbook (`ARCHITECTURE.md` & `RUNBOOK.md`) CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Document the full 8-month system architecture: `NanoSchema`/`NanoServe`, `StridedTensor`/`AutoTensor`, `NanoLLM`, `PagedKVScheduler`, `EngineCoreIPC`, native C/FFI kernels, and operational troubleshooting.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.

- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day219/`, and review the mapped book section for `Comprehensive Architecture Book & Operator Runbook (`ARCHITECTURE.md `&` RUNBOOK.md`)`.
- ☐ **[CORE] Task 2: From-Scratch Implementation (25m):** Write `docs/ARCHITECTURE.md` and `docs/RUNBOOK.md` with Mermaid diagrams.
- ☐ **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Comprehensive Architecture Book & Operator Runbook (`ARCHITECTURE.md `&` RUNBOOK.md`)` and save the test log beside `docs/ARCHITECTURE.md` and `docs/RUNBOOK.md`.
- ☐ **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- ☐ **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Comprehensive Architecture Book & Operator Runbook (`ARCHITECTURE.md `&` RUNBOOK.md`)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day219/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** Covers architecture, memory layout, concurrency model, metrics dictionary, and step-by-step incident playbooks.

- **Tangible Deliverable:** `docs/ARCHITECTURE.md` and `docs/RUNBOOK.md`
- **STOP WHEN:** Covers architecture, memory layout, concurrency model, metrics dictionary, and step-by-step incident playbooks.
- **Do Not Study Yet:** Boilerplate filler text.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 220: Timed 60-Minute Senior Systems Python Coding Challenge (Closed-Book) CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Implement a zero-copy strided KV-block ring buffer with async batching and custom descriptor validation from a blank file in 60 minutes without external help.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day220/`, and review the mapped book section for `Timed 60-Minute Senior Systems Python Coding Challenge (Closed-Book)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Complete `src/pymastery/drills/day220_final_coding_exam.py` and tests.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Timed 60-Minute Senior Systems Python Coding Challenge (Closed-Book)` and save the test log beside `src/pymastery/drills/day220_final_coding_exam.py` and `tests/test_day220_final_exam.py`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Timed 60-Minute Senior Systems Python Coding Challenge (Closed-Book)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day220/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** Passes `mypy --strict`, `tracemalloc` zero-leak check, and all unit tests within 60 minutes.

- **Tangible Deliverable:** `src/pymastery/drills/day220_final_coding_exam.py` and `tests/test_day220_final_exam.py`
- **STOP WHEN:** Passes `mypy --strict`, `tracemalloc` zero-leak check, and all unit tests within 60 minutes.
- **Do Not Study Yet:** Copy-pasting old code.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 221: Closed-Book Advanced Python Oral/Written Defense (30 Senior/Staff Interview Questions) CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Answer 30 deep questions spanning `PyObject` memory layout, descriptor lookup precedence, metaclass `__prepare__`, `asyncio TaskGroup` cancellation, `mmap`/`memoryview` zero-copy, tensor strides/broadcasting, GIL/PEP 703, DLPack, `@triton.jit`, and PagedAttention scheduling.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day221/`, and review the mapped book section for `Closed-Book Advanced Python Oral/Written Defense (30 Senior/Staff Interview Questions)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Record answers in `gates/month08-final-defense.md` without notes and verify against your codebase.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Closed-Book Advanced Python Oral/Written Defense (30 Senior/Staff Interview Questions)` and save the test log beside `gates/month08-final-defense.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Closed-Book Advanced Python Oral/Written Defense (30 Senior/Staff Interview`

`Questions)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day221/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** Can explain every topic clearly in under 3 minutes with concrete code examples.

- **Tangible Deliverable:** `gates/month08-final-defense.md`
- **STOP WHEN:** Can explain every topic clearly in under 3 minutes with concrete code examples.
- **Do Not Study Yet:** Vague hand-waving.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 222: Performance & Quality Portfolio Summary (10-Minute Technical Presentation) CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Synthesize your 8-month progression, benchmark speedups (Pure Python vs Vectorized vs Native C), memory savings (`__slots__`, `array`, `memoryview`, Paged KV, INT4), and chaos resilience into a concise technical presentation.
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day222/`, and review the mapped book section for `Performance & Quality Portfolio Summary (10-Minute Technical Presentation)`.

- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `docs/PORTFOLIO_PRESENTATION.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Performance & Quality Portfolio Summary (10-Minute Technical Presentation)` and save the test log beside `docs/PORTFOLIO_PRESENTATION.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Performance & Quality Portfolio Summary (10-Minute Technical Presentation)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day222/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** Every chart and table links to a reproducible script and artifact in the repository.

- **Tangible Deliverable:** `docs/PORTFOLIO_PRESENTATION.md`
- **STOP WHEN:** Every chart and table links to a reproducible script and artifact in the repository.
- **Do Not Study Yet:** Unbacked claims.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 223: Bridge Mapping to the AI Inference Engineering Curriculum (Track 1 & Track 2) CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Map every module in your `pymastery` repository directly to the 16 weeks of Track 1 (`vLLM`, Docker, PyTorch, FastAPI, Prometheus, Nsight) and Track 2 (`C/C++`, `CUDA`, `Triton`, `vLLM Internals`) of the AI Inference Engineering Curriculum.

- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day223/`, and review the mapped book section for `Bridge Mapping to the AI Inference Engineering Curriculum (Track 1 & Track 2)`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Create `docs/BRIDGE_TO_AI_INFERENCE_CURRICULUM.md`.
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Bridge Mapping to the AI Inference Engineering Curriculum (Track 1 & Track 2)` and save the test log beside `docs/BRIDGE_TO_AI_INFERENCE_CURRICULUM.md`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Bridge Mapping to the AI Inference Engineering Curriculum (Track 1 & Track 2)`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day223/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224—carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** Provides a direct reusability map showing how your Python mastery accelerates every week of the AI Inference Engineering book.

- **Tangible Deliverable:** `docs/BRIDGE_TO_AI_INFERENCE_CURRICULUM.md`
- **STOP WHEN:** Provides a direct reusability map showing how your Python mastery accelerates every week of the AI Inference Engineering book.
- **Do Not Study Yet:** Re-learning basic Python.
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

Day 224: Day 224 — Final Gate: 8-Month Python Systems & AI Inference Mastery Certification CORE: Advanced Track (Months 5-8)

MINOR: Clean-room build, architectural runbook, and closed-book senior engineering defense ¶

> **The Big Picture:** Complete the 8-Month (224-Day, 224-Hour) Python Mastery Program: package your work, verify clean-room reproducibility, defend every systems concept from memory layout to async schedulers, and transition seamlessly into the AI Inference Engineering Curriculum.
>
> Today is a focused **1.0-hour engineering session**: 15m concept retrieval, 30m from-scratch implementation (`[CORE - 70%]`), and 15m static verification, unit testing, and failure diagnosis (`[MINOR - 30%]`).

- **Primary Focus [CORE - 70% Weight | 42 mins]:** Final sign-off on all 224 days (112 hours Base Concepts + 112 hours Advanced Concepts).
- **Secondary Focus [MINOR - 30% Weight | 18 mins]:** Clean-room build, architectural runbook, and closed-book senior engineering defense — because you now possess deep, from-scratch mastery of Python for AI Inference Engineering.
- **Mapped Book & Docs:** Complete Book Shelf Review (Ramalho, Gorelick & Ozsvald, Fowler, Viafore, Okken, Shaw)

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Reading (10m):** Open `py-mastery/`, check `git status`, create `artifacts/day224/`, and review the mapped book section for `Day 224 — Final Gate: 8-Month Python Systems & AI Inference Mastery Certification`.
- [ ] **[CORE] Task 2: From-Scratch Implementation (25m):** Complete the final competency checklist, tag git release `v2.0.0-advanced-mastery`, and celebrate completing the entire 8-month curriculum!
- [ ] **[CORE] Task 3: Automated Unit Tests (10m):** Write `pytest` unit and edge-case tests verifying `Day 224 — Final Gate: 8-Month Python Systems & AI Inference Mastery Certification` and save the test log beside `gates/month08-final-certification.md` and git tag `v2.0.0-advanced-mastery`.
- [ ] **[MINOR] Task 4: Quality & Tooling Audit (8m):** Clean-room build, architectural runbook, and closed-book senior engineering defense: Every claim in your final portfolio must point to a runnable test, benchmark CSV/SQLite record, or profile artifact.
- [ ] **Task 5: Failure & Diagnosis Test (7m):** Deliberately break one invariant or type/boundary assumption in `Day 224 — Final Gate: 8-Month Python Systems & AI Inference Mastery Certification`, verify that `pytest` or `mypy --strict` catches it immediately, restore the fix, and log the symptom in `artifacts/day224/failure-notes.md`.

> **Engineering Trap to Avoid:** Do not stop writing quality, strictly-typed, tested code after Day 224— carry these exact habits into every day of the AI Inference Engineering Curriculum!

> **Daily Gate (STOP WHEN):** All 224 daily lessons complete, all 32 weekly gates passed, and both Month 4 and Month 8 Capstones certified!

- **Tangible Deliverable:** `gates/month08-final-certification.md` and git tag `v2.0.0-advanced-mastery`
- **STOP WHEN:** All 224 daily lessons complete, all 32 weekly gates passed, and both Month 4 and Month 8 Capstones certified!
- **Do Not Study Yet:** Nothing—you have mastered Python for AI Inference Engineering!
- **Estimated Time:** 1.0 Hour total (Full Session: 60 mins | Core-Only Sprint: 42 mins)

---

5. Topic Boundary Cards (Months 1–8)¶

Card 1: Modern Packaging, Strict Typing & Data Modeling (Month 1)¶

- **Engineering Purpose:** Eliminate whole classes of `NoneType`, schema mismatch, and environment drift bugs before runtime.
- **Required Depth:** Level 4 (From-Scratch Implementation & Strict Enforcement).
- **Must Implement Yourself:** `pyproject.toml` (PEP 621), `dataclass(frozen=True, slots=True)`, `Enum`/`StrEnum`, `Protocol`, `Generic[T]`, `ParamSpec`, `TypeGuard`, and `Result[T, E]`.
- **STOP WHEN:** Every module in `src/pymastery/` passes `mypy --strict` and `ruff check` with zero warnings and 0 uses of `Any`.

Card 2: Context Safety, Testing Craftsmanship & Lazy Generators (Month 1)¶

- **Engineering Purpose:** Guarantee deterministic resource cleanup under exceptions and O(1) memory footprint on multi-gigabyte data streams.
- **Required Depth:** Level 4 (Implementation + Memory Verification).
- **Must Implement Yourself:** Custom exception trees with `__cause__`, `ExceptionGroup`/`except*`, `__enter__`/`__exit__`, `ExitStack`, custom `Iterable`/`Iterator` classes, and lazy generator pipelines verified with `tracemalloc`.
- **STOP WHEN:** Streaming 250,000 JSONL records uses < 5 MB peak RAM and leaves zero leaked file descriptors under fault injection.

Card 3: The Python Data Model, Descriptors & Algorithmic Structures (Month 2)¶

- **Engineering Purpose:** Master object internals, attribute lookup, and O(1)/O(log N) data structures used in inference caches and schedulers.
- **Required Depth:** Level 4 (Implementation + Complexity Benchmarking).
- **Must Implement Yourself:** Dunder methods (`__repr__`, `__hash__`, `__eq__`, `__getitem__` slicing), `__slots__`, `weakref` registries, typed decorators, `__get__`/`__set__`/`__set_name__` descriptors, `FairRequestScheduler` (`heapq`), O(1) `LRUCache`, `TokenPrefixTrie`, and `mmap` `.safetensors` parser.
- **STOP WHEN:** Custom structures pass randomized property tests and empirical Big-O benchmarks.

Card 4: Deep `asyncio`, Dynamic Batching, Wire Protocols & Multiprocessing (Month 3)¶

- **Engineering Purpose:** Build high-concurrency streaming servers and multi-process worker bridges without deadlocks, task leaks, or event-loop stalls.
- **Required Depth:** Level 4 (Systems Implementation).
- **Must Implement Yourself:** `TaskGroup` supervisors, cancellation/timeout shields, `AsyncDynamicBatcher`, length-prefixed TCP streams, `SSEParser`, raw ASGI streaming server with graceful drain, and `multiprocessing` (`spawn`) + `SharedMemory` zero-copy IPC.
- **STOP WHEN:** 100 concurrent streaming requests batch, stream, handle disconnects, and recover from simulated worker crashes cleanly.

Card 5: Metaprogramming, AST, Bytecode & Framework Authoring (Month 5)¶

- **Engineering Purpose:** Demystify how PyTorch (`nn.Module`, `torch.compile`), Triton (`@triton.jit`), Pydantic, and FastAPI work internally.
- **Required Depth:** Level 4 (Framework Authoring from Scratch).
- **Must Implement Yourself:** `__init_subclass__` plugin registries, metaclasses, `MiniModule` parameter tracker, AST linter/transformer, `dis` bytecode analyzer, symbolic DAG tracer, `NanoSchema` (with codegen validator), and `NanoServe` (ASGI + DI + OpenAPI).
- **STOP WHEN:** Your hand-built `NanoServe` + `NanoSchema` framework powers your inference server with zero external web dependencies.

Card 6: CPython Memory, Buffer Protocol, Strided Tensors, Autograd & Paged KV (Month 6)¶

- **Engineering Purpose:** Bridge high-level Python to low-level memory arenas, tensor strides, numerical quantization, and vLLM scheduling.
- **Required Depth:** Level 4 (From-Scratch Numerical & Engine Implementation).
- **Must Implement Yourself:** Zero-copy `memoryview` slab allocator, `StridedTensor` (zero-copy `transpose`/`broadcast_to`), `einsum`, IEEE-754 stable `softmax`, INT8/INT4 quantizer, `AutoTensor` autograd, `NanoLLM` (RoPE + GQA + KV Cache + BPE), and `ContinuousBatchScheduler` with PagedAttention Copy-on-Write & Prefix Caching.
- **STOP WHEN:** KV-cached decode matches full-sequence logits to `1e-5` and 1,000 continuous batching steps conserve 100% of physical KV blocks.

Card 7: Profiling, Roofline, Free-Threading & Native C/C++/GPU Interop (Month 7)¶

- **Engineering Purpose:** Diagnose CPU/memory/GPU bottlenecks scientifically and cross the Python-to-C/CUDA boundary with zero copies.
- **Required Depth:** Level 4 (Systems Tuning & Native FFI).
- **Must Implement Yourself:** `MicroBench`, `cProfile/pstats` analyzer, `SamplingFlamegraphProfiler`, `AllocationHotspotAnalyzer`, `RooflineModel`, thread-safe PEP 703 structures, zero-copy `ctypes` C kernels (`fast_ops.c`), DLPack bridge, `MetaTensor` 0-byte verifier, and Tiled FlashAttention SPMD simulator.
- **STOP WHEN:** Every optimization has a 5-run before/after profile and native C kernels match reference outputs to `1e-5`.

Card 8: Distributed IPC, Prometheus Telemetry & `Mini-vLLM` Capstone (Month 8)¶

- **Engineering Purpose:** Integrate all 8 months of Python mastery into a fault-tolerant, observable, multi-process inference engine control plane.
- **Required Depth:** Level 4 (Production Systems Capstone).
- **Must Implement Yourself:** `EngineCoreIPC`, `MultiRankWorkerGroup` (Tensor Parallelism simulation), `MetricsRegistry` (Prometheus text exposition), `CircuitBreaker`, `PrefixAwareLoadBalancer`, `IncrementalDetokenizer`, `JSONSchemaLogitsMasker`, `LoRACacheManager`, and the full `Mini-vLLM` capstone.
- **STOP WHEN:** Clean-room build, benchmark matrix, flamegraph profile, and 3 chaos post-mortems pass final review.

---

6. Authoritative Book Recommendations (by Author Name & Publisher Name)¶

Rather than juggling 15 scattered tutorials, use these **6 canonical engineering books** (4 Primary Core + 2 Specialized Reference). Every single week of the 8-month curriculum maps directly to exact chapters in these books:

6.1 Primary Core Books (Must-Have Shelf)¶

Priority	Book Title & Edition	Author Name(s)	Publisher Name	Target Curriculum Phase & Why It Is Essential
#1 Primary Bible (Base + Advanced)	***Fluent Python: Clear, Concise, and Effective Programming* (2nd Edition, 2022)**	**Luciano Ramalho**	**O'Reilly Media**	**Months 1, 2, 3 & 5 (Weeks 1–11, 17–20):** The undisputed gold standard for Python's Data Model (dunder methods, `__slots__`), type hints (`Protocol`, generics), closures, decorators, descriptors, iterators/generators, `asyncio`, concurrency, and metaprogramming (`__init_subclass__`, metaclasses).
#2 Primary Typing & Architecture (Base)	***Robust Python: Write Clean and Maintainable Code* (1st Edition, 2021)**	**Patrick Viafore**	**O'Reilly Media**	**Months 1 & 4 (Weeks 1–4, 13–15):** Deep, practical coverage of strict static typing (`mypy`), `dataclass`, `Enum`, `Protocol`, `TypedDict`, custom collection types, context managers, and extensible design without inheritance bloat.
#3 Primary Async & Streaming Systems (Base + Advanced)	***Python Concurrency with asyncio* (1st Edition, 2022)**	**Matthew Fowler**	**Manning Publications**	**Months 3 & 8 (Weeks 9–12, 29):** The best engineering book ever written on `asyncio` internals, event loops, Tasks, timeouts, cancellation, raw async sockets, streaming HTTP, queues, `multiprocessing` bridges, and avoiding event-loop stalls.
#4 Primary Performance, Memory & C Interop (Advanced)	***High Performance Python: Practical Performant Programming for Humans* (2nd Edition, 2020)**	**Micha Gorelick and Ian Ozsvald**	**O'Reilly Media**	**Months 2, 6 & 7 (Weeks 7, 12, 21–28):** Masterclass on `cProfile`, line/memory profiling, Python memory layout, `array`/`memoryview` zero-copy buffers, vectorization, `multiprocessing`/`shared_memory`, the GIL, and compiling C/C++ extensions (`ctypes`, `cffi`, Cython/native).

6.2 Specialized Reference Books (Internals, Testing & LLM Mechanics)¶

Role	Book Title & Edition	Author Name(s)	Publisher Name	Target Curriculum Phase & Why It Is Essential
CPython VM & Bytecode Reference	***CPython Internals: Your Guide to the Python 3 Interpreter* (1st Edition, 2021)**	**Anthony Shaw**	**Real Python (Dan Bader / Real Python Press)**	**Months 5 & 6 (Weeks 18, 21):** Explains how CPython parses ASTs, compiles bytecode (`dis`), executes the evaluation frame loop, manages `PyObject` reference counts, and runs the generational cyclic garbage collector.
Testing Craftsmanship Reference	***Python Testing with pytest: Simple, Rapid, Effective, and Scalable* (2nd Edition, 2022)**	**Brian Okken**	**The Pragmatic Bookshelf (Pragmatic Programmers)**	**Months 1–4 (Week 3 & Daily Task 3):** Concise guide to writing professional, fast, parametrized `pytest` suites, fixtures, `tmp_path`, `monkeypatch`, and testing CLI/package boundaries.
Clean Architecture Reference	***Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices* (1st Edition, 2020)**	**Harry Percival and Bob Gregory**	**O'Reilly Media**	**Months 1 & 4 (Weeks 4, 13):** Shows how to separate pure domain models from infrastructure/I/O adapters using `Protocol` ports and lightweight dependency injection.
Best-Practice Idioms Reference	***Effective Python: 125 Specific Ways to Write Better Python* (3rd Edition, 2024)**	**Brett Slatkin**	**Addison-Wesley Professional (Pearson)**	**Months 1–8 Daily Polish:** Actionable rules on modern Python 3.12/3.13 idioms, concurrency pitfalls, robustness, and API design.
From-Scratch LLM Mechanics Bridge	***Build a Large Language Model (From Scratch)* (1st Edition, 2024)**	**Sebastian Raschka**	**Manning Publications**	**Month 6 (Week 23):** Clear reference for BPE tokenization, causal self-attention, RoPE, layer normalization, and autoregressive token generation loops.

---

7. Value-Added Engineering Additions (What Most Prompts Miss)¶

To make sure your 8-month journey produces **real, senior-level engineering skill** rather than passive reading, this curriculum includes five critical systems mechanisms you might not have thought to ask for:

7.1 The "Zero AI Autocomplete / Blank-File" Implementation Rule¶

When doing **Task 2 (25m From-Scratch Implementation)** each day:

- **Turn off AI code-completion (Copilot/inline ghost text) during your 25-minute implementation block.**
- You may read the book, Python docs, or `help()`, and you can ask an AI *conceptual* questions ("How does `__set_name__` receive the attribute name?") or ask it to **review** your code *after* you write it—but your fingers must type every class, signature, loop, and test from a blank file. Muscle memory for strict typing, error handling, and async cleanup only forms when you resolve the compiler/type-checker errors yourself.

7.2 Your Standard `pyproject.toml` Quality Gate Template (Use from Day 1)¶

Every code file you write across all 224 days lives inside a single cohesive mono-package (`py-mastery/`) configured with strict verification from Day 1:

```
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "pymastery"
version = "0.1.0"
description = "From-scratch Systems Python & AI Inference Engineering Mastery"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "mypy>=1.10",
    "ruff>=0.5",
]

[tool.mypy]
python_version = "3.12"
strict = true
warn_unreachable = true
disallow_any_unimported = true
disallow_any_generics = true

[tool.ruff]
target-version = "py312"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "N", "SIM", "RUF", "C90"]

[tool.pytest.ini_options]
testpaths = ["tests"]
filterwarnings = ["error"]
```

7.3 The 5-Point "Quality Code" Self-Review Rubric¶

Before checking off any day's final task box, verify these 5 invariants: 1. **Types Tell the Whole Truth:** Could another engineer understand the inputs, outputs, and failure modes of your function from its signature alone without reading its body? 2. **Invalid States Are Unrepresentable:** Did you use `@dataclass(frozen=True, slots=True)`, `Enum`, and `NewType`/`TypeGuard` instead of passing untyped `dict`s or raw `int`/`str` flags? 3. **O(1) Memory & Deterministic Cleanup:** Does your code process streams lazily (`Iterator`/`AsyncIterator`) and close every file/socket/task/shared-memory block inside a `with`/`async with`/`finally` block? 4. **Complexity Awareness:** Did you avoid hidden $O(N^2)$ string concatenations, `list.pop(0)`, or linear `x in list` scans inside loops? 5. **Failure-Tested:** Did you witness your test suite fail when you broke an invariant in Task 5, and pass when you restored it?

7.4 How This Curriculum Connects Directly to Your AI Inference Engineering Curriculum¶

You do **not** have to wait 8 full months before touching `AI_INFERENCE_ENGINEERING_CURRICULUM_BOOK.html`:

- **Option A (Sequential Foundation First):** Complete **Months 1–4 (Base Concepts, 1 hr/day)** first to achieve rock-solid Python & `asyncio` mastery, then start **Track 1 of the AI Inference Curriculum** while doing **Months 5–8 (Advanced Python, 1 hr/day)** in parallel! Every topic in

Months 5–8 (tensors, KV cache, PagedAttention scheduler, profiling, C/CUDA interop, Prometheus) directly supercharges the corresponding weeks of the AI Inference Engineering Curriculum.

- **Option B (Parallel Daily Pair):** Spend **1 hour/day** on this Python Mastery Curriculum as your morning "warm-up & deep language craft" block right before your AI Inference Engineering lab block.

---

8. Final Mastery Checklists¶

8.1 Month 4 Base Concepts Gate Checklist (Day 112)¶

- [ ] I can create a clean `src/` Python package with `pyproject.toml`, `ruff`, `mypy --strict`, and `pytest` from memory.
- [ ] I write frozen, slotted dataclasses, `Protocol` interfaces, `Generic[T]` containers, and `ParamSpec` decorators without `Any`.
- [ ] I use `ExitStack`, custom context managers, chained exceptions, and `ExceptionGroup` so resources never leak on error.
- [ ] I build lazy O(1)-memory generator pipelines and verify peak heap memory with `tracemalloc`.
- [ ] I implement Python Data Model dunders (`__repr__`, `__eq__`, `__hash__`, `__getitem__` slicing), `__slots__`, `weakref`, and the descriptor protocol (`__get__`, `__set__`, `__set_name__`) from scratch.
- [ ] I have implemented an O(1) `LRUCache`, a `FairRequestScheduler` (`heapq`), a `TokenPrefixTrie`, and an `mmap` `.safetensors` binary reader by hand.
- [ ] I have implemented an `asyncio` `AsyncDynamicBatcher`, length-prefixed TCP RPC, an `SSEParser`, a raw ASGI streaming server with graceful drain, and a `multiprocessing.shared_memory` worker bridge (`pyserve-core`).

8.2 Month 8 Advanced Concepts Gate Checklist (Day 224)¶

- [ ] I can use `__init_subclass__`, metaclasses, `ast.NodeTransformer`, `dis` bytecode inspection, and `sys.monitoring` safely.
- [ ] I have built `NanoSchema` (with code-generated validators) and `NanoServe` (ASGI router + DI + SSE + OpenAPI) from scratch.
- [ ] I can explain CPython `PyObject` layout, refcounting, cyclic GC generations, `gc.freeze()`, and zero-copy `memoryview` slicing/casting.
- [ ] I have built a `StridedTensor` (with zero-copy `transpose` and `stride=0` broadcasting), an INT8/INT4 quantizer, a reverse-mode `AutoTensor` autograd engine, and a KV-cached Transformer (`NanoLLM` with RoPE, GQA, SwiGLU, RMSNorm, and BPE) from scratch.
- [ ] I have implemented a PagedAttention `PhysicalBlockAllocator` (with Copy-on-Write and Automatic Prefix Caching), `ContinuousBatchScheduler` (with chunked prefill and preemption), and speculative decoding verification loop in pure Python.
- [ ] I can profile CPU and memory using `cProfile`, stack-sampling flamegraphs, and roofline arithmetic-intensity models.
- [ ] I can bind native C kernels (`ctypes`/`cffi`/`pybind11` architecture) with zero-copy buffer pointers, verify 70B shapes in 0 MB with `MetaTensor`s, and explain DLPack, `@triton.jit`, and CUDA Graphs.
- [ ] I have built and defended the full `Mini-vLLM` multi-process capstone with Prometheus metrics, prefix-aware routing, circuit breakers, and chaos post-mortems.